

Contents

1	AdHoc	3	3.13 MergeSortTree	18	
1.1	Bit Hacks	3	3.14 SegTreeBeats	18	
1.2	FastIO	3	3.15 Segment Tree (2D)	19	
1.3	Gray Code	3	3.16 Segment Tree (Iterative)	20	
1.4	Infinite Grid	4	3.17 Segment Tree (Persistent)	20	
1.5	Josephus Problem	4	3.18 Segment Tree (Recursive)	21	
1.6	Knight Distance	4	3.19 Segment Tree (Sparse)	21	
1.7	RandomShuffle	4	3.20 Wavelet Tree	22	
1.8	U128	4	3.21 Sliding RMQ	23	
1.9	__int128	6	3.22 SparseTable (1D)	23	
2	DP	6	3.23 SparseTable (Rectangle Query)	23	
2.1	Alien Trick	6	3.24 SparseTable (Square Query)	24	
2.2	CHT (Dynamic)	6	3.25 Treap (Implicit)	24	
2.3	CHT (Offline)	7	3.26 Treap (Persistent)	25	
2.4	Divide & Conquer Optimization	8	3.27 Trie (Basic)	26	
2.5	Knuth Optimization	8	3.28 Trie (Persistent)	26	
2.6	Li-Chao Segment Tree	8	4	Game Theory	27
2.7	Li-Chao Tree (Sparse)	9	4.1	Green Hackenbush	27
2.8	SOS DP	10	4.2	Red-Blue Hackenbush	28
2.9	Sequence DP	10	5	Geometry	28
2.10	LCS (Faster)	11	5.1	2D Geo	28
2.11	SteinerTree	11	5.2	3D Geo	34
3	DS	12	5.3	Area of Union Of Circles	37
3.1	BIT (1D)	12	5.4	Integer Tricks	38
3.2	BIT (2D - Point)	12	5.5	IntegerGeo	38
3.3	BIT (2D - Range)	12	5.6	Radial Sweep	39
3.4	BIT (3D)	13	5.7	RotationMatrix	39
3.5	Closest Pair	13	6	Graph	40
3.6	HashTable	14	6.1	ArticulationBridge	40
3.7	KD Tree	14	6.2	ArticulationPoint	40
3.8	MO (Basic)	15	6.3	Block-CutVertexTree	40
3.9	MO on Tree	15	6.4	LinkCutTree(rooted)	41
3.10	MOs algo with updates	16			

6.5	LinkCutTree	42	6.42	LCA, $O(V \log V + Q)$	67
6.6	Dinic	43	6.43	MO on Tree	68
6.7	FasterMaxflowButWeird	44	6.44	Path Path Intersection	68
6.8	GlobalMinCut	47	6.45	0-1BFS	69
6.9	Gomuri-Hu Tree	47	6.46	BellmanFord	69
6.10	Kth Minimum Cut	47	6.47	Dijkstra(V^2)	69
6.11	MinCostMaxFlow	48	6.48	Dijkstra($V \log V + E$)	69
6.12	MinimumCostCirculation	49	6.49	FloydWarshall	70
6.13	___FlowNotes	50	6.50	JohnsonAlgorithm	70
6.14	Bipartite Matching (Hopcroft Karp)	52	7 Math	71	
6.15	Bipartite Matching(Kuhn)	53	7.1	Fibonacci Combination Sums	71
6.16	General Matching (Edmonds Blossom)	53	7.2	Partition Function	72
6.17	StableMarriage	55	7.3	Stirling Number (via NTT)	72
6.18	Weighted Bipartite Matching(Hungarian)	55	7.4	$nCr \bmod M$	72
6.19	Weighted Bipartite Matching2(Hungarian)	56	7.5	Gaussian Elimination	74
6.20	Weighted General Matching	56	7.6	Matrix	74
6.21	DMST	57	7.7	Maximum Xor Subset	76
6.22	DMSTwithSolution	57	7.8	Simplex	76
6.23	ManhattanMST	58	7.9	Thomas Algorithm	77
6.24	PrimsBorukuvaMST	59	7.10	Baby Step Giant Step (Any modulo)	77
6.25	ChromaticNumber	59	7.11	Baby Step Giant Step (Only Prime Modulo)	78
6.26	CycleEnumeration	60	7.12	Continued Fraction	79
6.27	EdgeColoring	60	7.13	Diophantine Equation	79
6.28	EulerWalk	61	7.14	Fast Fibonacci	80
6.29	MatrixTreeTheorem	62	7.15	Faulhaber's Formula	80
6.30	MaxClique	62	7.16	Fraction Binary Search	80
6.31	MinimumMeanCycle	62	7.17	General CRT	81
6.32	TwoSat	63	7.18	Geometric Sum	81
6.33	Centroid Decomposition on Edge	63	7.19	Integration (Romberg)	81
6.34	Centroid Decomposition	64	7.20	Integration (Simpson)	81
6.35	CompressTree	64	7.21	Interpolation (Lagrange - Optimized)	82
6.36	DSU	64	7.22	Interpolation (Lagrange)	82
6.37	DominatorTree (for Any Directed Graph)	65	7.23	Interpolation (Newton)	83
6.38	DominatorTree	65	7.24	Lehmer Prime Counting	83
6.39	$F_x \text{ times } x \text{ le } N$ Trick	66	7.25	Maximum Divisors in Range	84
6.40	HLD	66	7.26	Pollard's Rho	84
6.41	LCA (using SparseTable)	67	7.27	Primitive root for Prime	85
			7.28	Range Congruence	86
			7.29	Schreier-Sims Group Test	86

ACM ICPC C++ Reference Notebook

Al-Muzahid Seyam | BUET

October 3, 2026

7.30 Sieve (Linear) 87

7.31 FFT (Faster) 88

7.32 FFT (Slow, but short) 88

7.33 FWHT 89

7.34 KaratSuva 90

7.35 NTT Applications 91

7.36 NTT 91

7.37 Online FFT Trick 92

7.38 Polynomial Root Check 92

7.39 Polynomial 93

8 Matroid 95

8.1 colorGraphicMatroid 95

8.2 colorLinearMatroid 96

8.3 graphicGraphicMatroid 98

9 Notes 99

9.1 Bernoulli 99

9.2 Catalan 99

9.3 Eulerian 99

9.4 Stirling 100

10 String 100

10.1 Aho Corasick 100

10.2 Dynamic Aho-Corasick 101

10.3 Hashing (1D) 102

10.4 Hashing (2D) 103

10.5 KMP Fail + Z-Algo 103

10.6 Minimum Lexicographical Rotation 104

10.7 Palindromic Tree 104

10.8 Suffix Array (O(n logn)) 105

10.9 Suffix Automata 106

1 AdHoc

1.1 Bit Hacks

```
1 #define ll long long int
2
3 // Only for non-negative integers
4 // Returns the immediate next number
5 // with same count of one bits, -1 on failure
6 ll NextSamePop(ll n){
7     if(n == 0) return -1; ll x = (n & -n);
8     ll left = (x + n); ll right = ((n ^ left) / x) >> 2;
9     ll res = (left | right); return res;
10 }
11
12 // Returns the immediate previous number
13 // with same count of one bits, -1 on failure
14 ll PrevSamePop(ll n){
15     if (n == 0 || n == 1) return -1;
16     ll res = ~NextSamePop(~n); return (res == 0) ? -1 : res;
17 }
18
19 unsigned int reverse_bits(unsigned int v){
20     v = ((v >> 1) & 0x55555555) | ((v & 0x55555555) << 1);
21     v = ((v >> 2) & 0x33333333) | ((v & 0x33333333) << 2);
22     v = ((v >> 4) & 0x0F0F0F0F) | ((v & 0x0F0F0F0F) << 4);
23     v = ((v >> 8) & 0x00FF00FF) | ((v & 0x00FF00FF) << 8);
24     return ((v >> 16) | (v << 16));
25 }
```

1.2 FastIO

```
1 inline int getint(){
2     char ch = getchar(); int x = 0;
3     while(ch < '0' || ch > '9') ch = getchar();
4     while(ch >= '0' && ch <= '9'){
5         x = x * 10 + ch - '0'; ch = getchar();
6     }
7     return x;
8 }
```

1.3 Gray Code

```
1 #define ll long long int
2 ll gray_code(ll x){return x ^ (x >> 1);}
3 ll inverse_gray_code(ll x){
4     ll h = 1, res = 0;
```

```

7     x >>= 1, h = (h << 1) + 1;
8 } while (x);
9 return res;
10 }

```

1.4 Infinite Grid

```

1 //infinite grid formed by replication of initial n*m grid
2 //All co-ordinates are 0-indexed
3 //Answer rectangular queries (x1,y1,x2,y2)
4
5 int n, m;
6 ll F(int x,int y){
7     if(x < 0 || y < 0) return 0;
8
9     //12
10    //34
11    int r = x / n; int c = y / m;
12    int X = x % n; int Y = y % m;
13
14    //Case 1 : r * c Full blocks)
15    if(r && c){
16        //Add contribution of g(0,0,n-1,m-1)
17        //in range row[0,r-1] and col[0,c-1]
18    }
19
20    //Case 2 : c partial (row-wise) blocks
21    if(c){
22        //Add contribution of g(0,0,X,m-1)
23        //in range col[0,c-1]
24    }
25
26    //Case 3 : r partial (col-wise) blocks
27    if(r){
28        //Add contribution of g(0,0,n-1,Y)
29        //in range row[0,r-1]
30    }
31
32    //Case 4 : only partial (both row and col-wise) block(r,c)
33    //Add contribution of g(0,0,X,Y) for block(r,c)
34 }
35
36 ll Solve(int x1,int y1,int x2,int y2){
37     return F(x2,y2) - F(x1-1,y2) - F(x2,y1-1) + F(x1-1,y1-1);
38 }

```

1.5 Josephus Problem

```

1 // Tested : Gym 101955K - Let the Flames Begin
2

```

```

3 #include <bits/stdc++.h>
4 using namespace std;
5 #define ll long long int
6 // Complexity : O(min(k log(n), m))
7 // People 1 to n in a circle, counting starts from 1, every k'th
8 // people dies, returns the position of the m'th killed people
9 ll Josephus(ll n, ll k, ll m){
10     m = n - m;
11     if (k <= 1) return n - m;
12     ll i = m;
13     while (i < n){
14         ll r = (i - m + k - 2) / (k - 1);
15         if ((i + r) > n) r = n - i;
16         else if (!r) r = 1;
17         i += r;
18         m = (m + (r * k)) % i;
19     }
20     return m + 1;
21 }

```

1.6 Knight Distance

```

1 #define ll long long int
2 // Minimum number of knight moves from (x,y) to
3 // (0,0) in non-negative infinite chessboard
4 ll knight_move(ll x, ll y){
5     ll cnt = max({(x+1)/2, (y+1)/2, (x+y+2)/3});
6     while((cnt%2) != (x+y)%2) cnt++;
7     if(x == 1 && !y) return 3;
8     if(y == 1 && !x) return 3;
9     if(x == y && x == 2) return 4;
10    return cnt;
11 }

```

1.7 RandomShuffle

```

1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2 vector<int> permutation(N);
3 shuffle(permutation.begin(), permutation.end(), rng);

```

1.8 U128

```

1 typedef unsigned long long int U64;
2 struct U128{
3     U64 lo, hi;
4     static const U64 bmax = -1;
5     static const size_t sz = 128;
6     static const size_t hsz = 64;

```

```

7   inline U128() : lo(0), hi(0) {}
8   inline U128(unsigned long long v) : lo(v), hi(0) {}
9   inline U128 operator-() const {return ~U128(*this) + 1;}
10  inline U128 operator~() const {
11  U128 t(*this);t.lo = ~t.lo;t.hi = ~t.hi;return t;
12  }
13  inline U128 &operator +=(const U128 &b) {
14      if (lo > bmax - b.lo) ++hi;lo += b.lo;hi += b.hi;return *this;
15  }
16  inline U128 &operator -= (const U128 &b){return *this += -b;}
17  inline U128 &operator *= (const U128 &b) {
18      if (*this == 0 || b == 1) return *this;
19      if (b == 0){lo = hi = 0;return *this;}
20      U128 a(*this);U128 t = b;lo = hi = 0;
21      for (size_t i = 0; i < sz; i++) {
22          if((t & 1) != 0){*this += (a << i);} t >>= 1;
23      }
24      return *this;
25  }
26  inline U128 &operator /= (const U128 &b) {
27  U128 rem;divide(*this, b, *this, rem);return *this;
28  }
29  inline U128 &operator %=(const U128 &b) {
30  U128 quo;divide(*this, b, quo, *this);return *this;
31  }
32  inline static void divide(const U128 &num, const U128 &den, U128 &quo, U128 &
    rem) {
33      if(den == 0) {int a = 0;quo = U128(a / a);}
34      U128 n = num, d = den, x = 1, ans = 0;
35      while((n >= d) && (((d >> (sz - 1)) & 1) == 0)) {
36          x <<= 1;d <<= 1;
37      }
38      while(x != 0) {
39          if(n >= d) { n -= d;ans |= x;} x >>= 1, d >>= 1;
40      }
41      quo = ans, rem = n;
42  }
43  inline U128 &operator&=(const U128 &b) {
44      hi &= b.hi;lo &= b.lo;return *this;
45  }
46  inline U128 &operator|=(const U128 &b) {
47      hi |= b.hi;lo |= b.lo;return *this;
48  }
49  inline U128 &operator<=(const U128& rhs) {
50  size_t n = rhs.to_int();
51      if (n >= sz){lo = hi = 0;return *this;}
52      if(n >= hsz){n -= hsz;hi = lo;lo = 0;}
53      if(n != 0) {
54          hi <<= n;const U64 mask(~(U64(-1) >> n));
55          hi |= (lo & mask) >> (hsz - n);lo <<= n;
56      }
57      return *this;
58  }

```

```

59  inline U128 &operator>=(const U128& rhs) {
60      size_t n = rhs.to_int();
61      if (n >= sz){lo = hi = 0;return *this;}
62      if(n >= hsz){n -= hsz;lo = hi; hi = 0;}
63      if(n != 0) {
64          lo >>= n;const U64 mask(~(U64(-1) << n));
65          lo |= (hi & mask) << (hsz - n);hi >>= n;
66      }
67      return *this;
68  }
69  inline int to_int() const { return static_cast<int> (lo); }
70  inline U64 to_U64() const { return lo; }
71  inline bool operator == (const U128 &b) const {
72      return hi == b.hi && lo == b.lo;
73  }
74  inline bool operator != (const U128 &b) const {
75      return !(*this == b);
76  }
77  inline bool operator < (const U128 &b) const {
78      return (hi == b.hi) ? lo < b.lo : hi < b.hi;
79  }
80  inline bool operator >= (const U128 &b) const {
81      return ! (*this < b);
82  }
83  inline U128 operator & (const U128 &b) const {
84      U128 a(*this); return a &= b;
85  }
86  inline U128 operator << (const U128 &b) const {
87      U128 a(*this); return a <<= b;
88  }
89  inline U128 operator >> (const U128 &b) const {
90      U128 a(*this); return a >>= b;
91  }
92  inline U128 operator * (const U128 &b) const {
93      U128 a(*this); return a *= b;
94  }
95  inline U128 operator + (const U128 &b) const {
96      U128 a(*this); return a += b;
97  }
98  inline U128 operator - (const U128 &b) const {
99      U128 a(*this); return a -= b;
100  }
101  inline U128 operator % (const U128 &b) const {
102      U128 a(*this); return a %= b;
103  }
104  inline void print(){
105      U128 x = *this;char str[128];int i, j, len = 0;
106      do{str[len++]=(x % 10).lo+48;x/=10;}while (x != 0);
107      reverse(str, str + len);str[len] = 0;puts(str);
108  }
109  };
110  inline U128 gcd(U128 a, U128 b){
111      if (b == 0) return a;return gcd(b, a % b);

```

```

112 }
113 inline U128 expo(U128 b, U128 e){
114     U128 res = 1;
115     while (e != 0){if ((e & 1) != 0) res *= b;e >>= 1, b *= b;}
116     return res;
117 }
118 inline U128 expo(U128 x, U128 n, U128 m){
119     U128 res = U128(1);
120     while (n != 0){
121         if ((n & 1) != 0){res *= x;res %= m;}
122         x *= x;x %= m;n >>= 1;
123     }
124     return res % m;
125 }
126 struct Rational{
127     U128 p, q;
128     inline Rational(){p = 0, q = 1;}
129     inline Rational(U128 P, U128 Q) : p(P), q(Q){simplify();}
130     inline void simplify() {U128 g = gcd(p, q);p /= g;q /= g;}
131     inline Rational operator+ (const Rational &f) const {
132         return Rational(p * f.q + q * f.p, q * f.q);
133     }
134     inline Rational operator- (const Rational &f) const {
135         return Rational(p * f.q - q * f.p, q * f.q);
136     }
137     inline Rational operator* (const Rational &f) const {
138         return Rational(p * f.p, q * f.q);
139     }
140     inline Rational operator/ (const Rational &f) const {
141         return Rational(p * f.q, q * f.p);
142     }
143 };
144 int main(){
145     U128 X = U128(9178291938173ULL);
146     U128 Y = U128(123456789123456ULL);
147     U128 M = U128(1000000000000000000ULL);
148     for(int i = 0; i < 10000; i++) U128 R = expo(X, Y, M);
149 }

```

1.9 __int128

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     int n = 100;
6     __int128 Ans = n;
7
8     vector<char> vec;
9     vec.clear();
10    while(Ans) vec.push_back(Ans % 10 | 48), Ans /= 10;
11    reverse(vec.begin(), vec.end());

```

```

12    for(char x : vec) printf("%c",x);
13    printf("\n");
14 }

```

2 DP

2.1 Alien Trick

1 Lets say we need to divide n elements into k groups and
2 minimize the sum of a specific function for each group.
3 Normally this kind of problem can be solved by CHT or D&C.
4 Alien trick comes into use when N, K ≤ 10⁵.

5
6 We can use alien trick if the following conditions holds:
7 Here, Fn(K) = The value of dividing N elements into K groups
8 1. Fn(K) ≥ Fn(K + 1) (FOR MINIMIZE , For maximum it's reverse)
9 It means its always optimal to use (K+1) groups than K groups
10 2. Fn(K - 1) - Fn(K) ≥ Fn(K) - Fn(K + 1) (FOR Minimize)

11
12 It means with the increase of K, ans becomes more optimal,
13 but the rate of becoming optimal slows down.

14
15 If a function satisfies the above 2 conditions : then
16 we define G(n) = Fn(k) + kC
17 That means we need to add an extra C for each group we use.
18 Now this G(n) function will follow a convex property.

19
20 So now we can do binary search on C. Inside the binary search.
21 For each mid, we find the optimal value of G(n) and the
22 corresponding k for that value. This k decreases when C increases.

23
24 We find the lowest C for which corresponding k is ≤ given K.
25 This trick may apply for problems other than DP as well.

2.2 CHT (Dynamic)

```

1 //Dynamic CHT
2 //Tested : CF 455E - Function
3
4 #define ll long long int
5 #define ld long double
6 ll inf = 9e18 + 5;
7
8 struct HullDynamic{ // Max Query
9     struct line {
10         ll m, b; ld x;
11         ll val; bool isQuery;
12         line(ll _m = 0, ll _b = 0) :
13             m(_m), b(_b), val(0), x(-inf), isQuery(false) {}

```

```

14     ll eval(ll x) const { return m * x + b; }
15     bool parallel(const line &l) const { return m == l.m; }
16     ld intersect(const line &l) const {
17         return parallel(l) ? inf : 1.0 * (l.b - b) / (m - l.m);
18     }
19 }
20 bool operator < (const line &l) const {
21     if(l.isQuery) return x < l.val;
22     else return m < l.m;
23 }
24 };
25
26 set<line> hull;
27 typedef set<line> :: iterator iter;
28
29 bool cPrev(iter it) { return it != hull.begin(); }
30 bool cNext(iter it) { return it != hull.end() && next(it) != hull.end(); }
31
32 bool bad(const line &l1, const line &l2, const line &l3) {
33     return l1.intersect(l3) <= l1.intersect(l2);
34 }
35 bool bad(iter it) {
36     return cPrev(it) && cNext(it) && bad(*prev(it), *it, *next(it));
37 }
38
39 iter update(iter it) {
40     if(!cPrev(it)) return it;
41     ld x = it -> intersect(*prev(it));
42     line tmp(*it); tmp.x = x;
43     it = hull.erase(it);
44     return hull.insert(it, tmp);
45 }
46
47 void Add(ll m, ll b) {
48     line l(m, b);
49     iter it = hull.lower_bound(l);
50     if(it != hull.end() && l.parallel(*it)) {
51         if(it -> b < b) it = hull.erase(it);
52         else return;
53     }
54
55     it = hull.insert(it, l);
56     if(bad(it)) return (void) hull.erase(it);
57
58     while(cPrev(it) && bad(prev(it))) hull.erase(prev(it));
59     while(cNext(it) && bad(next(it))) hull.erase(next(it));
60
61     it = update(it);
62     if(cPrev(it)) update(prev(it));
63     if(cNext(it)) update(next(it));
64 }
65
66 ll Query(ll x) const {

```

```

67     if(hull.empty()) return -inf;
68     line q; q.val = x, q.isQuery = 1;
69     iter it = --hull.lower_bound(q);
70     return it -> eval(x);
71 }
72 };

```

2.3 CHT (Offline)

```

1 //Offline CHT
2 struct Line{
3     ll m,c;
4     Line(ll x,ll y){m=x; c=y;}
5     ll Get(ll x) {return m*x+c;}
6     bool operator<(const Line &other) const {return m < other.m;}
7 };
8
9 bool Bad(Line &P,Line &C,Line &N){
10     return (P.c-C.c) * 1.0L * (N.m-P.m) > (P.c-N.c) * 1.0L * (C.m - P.m);
11 }
12
13 //This Convex Hull always maintains lower convex hull
14 //m1 >= m2 >= m3 .... >= mk
15 //For Min Query : Add(m,c)
16 //For Max Query : Add(-m,-c)
17
18 struct ConvexHull{
19     vector<Line>hull;
20
21     void Add(ll m,ll c){
22         //always maintaining the minimum c in case multiple equal m
23         if(hull.size()>0 && hull.back().m==m) {
24             if(hull.back().c>c) hull.pop_back();
25             else return;
26         }
27
28         hull.push_back(Line(m, c));
29         int sz=hull.size();
30         while(sz>2 && Bad(hull[sz-3],hull[sz-2],hull[sz-1])){
31             swap(hull[sz-2],hull[sz-1]); hull.pop_back(); sz--;
32         }
33     }
34
35     ll Query(ll x){
36         int lo=-1;
37         int hi=hull.size()-1;
38         while(hi-lo>1) {
39             int mid=(lo+hi)/2;
40             if(hull[mid].Get(x)>=hull[mid+1].Get(x)) lo=mid;
41             else hi=mid;
42         }
43         if(hi<0 || hi==hull.size()) return INF;

```

```

44     return hull[hi].Get(x);
45 }
46 };

```

2.4 Divide & Conquer Optimization

```

1 // Complexity : O(n log n)
2 // dp[i][j] = min(dp[i-1][k-1] + C[k][j]) [ k <= j ]
3 // To use D&Q, its sufficient to prove the following :
4 // Cost(L + 1 , j + 1) - Cost(L + 1, j) <= Cost(k + 1, j + 1) - Cost(k + 1, j)
   for any(L < k < j) For Max Query
5 // Cost(L + 1 , j + 1) - Cost(L + 1, j) >= Cost(k + 1, j + 1) - Cost(k + 1, j)
   for any(L < k < j) For Min Query
6
7 #include<bits/stdc++.h>
8 using namespace std;
9 #define ll long long int
10 const int MAX = 5005;
11
12 int ara[MAX];
13 ll dp[2][MAX];
14 ll C[MAX][MAX];
15
16 void compute(int K, int L, int R, int OptL, int OptR){
17     if(L > R) return;
18     int mid = (L + R) / 2;
19
20     int optNow = -1;
21     dp[K & 1][mid] = 0;
22     for(int i=OptL; i<=min(OptR, mid); i++){
23         ll tmp = dp[(K & 1) ^ 1][i - 1] + C[i][mid];
24         if(tmp >= dp[K & 1][mid]){
25             dp[K & 1][mid] = tmp;
26             optNow = i;
27         }
28     }
29     compute(K, L, mid - 1, OptL, optNow);
30     compute(K, mid + 1, R, optNow, OptR);
31 }
32
33 int main(){
34     int t;
35     scanf("%d",&t);
36
37     while(t--){
38         int n, K;
39         scanf("%d %d", &n, &K);
40         for(int i=1; i<=n; i++) scanf("%d",&ara[i]);
41
42         for(int i=1; i<=n; i++){
43             for(int j=i; j<=n; j++){
44                 if(i==j) C[i][j] = ara[j];

```

```

45         else C[i][j] = C[i][j-1] | ara[j];
46     }
47 }
48
49 for(int i=1; i<=n; i++) dp[1][i] = C[1][i];
50 for(int i=2; i<=K; i++) compute(i, 1, n, 1, n);
51 printf("%lld\n", dp[K & 1][n]);
52 }
53 }

```

2.5 Knuth Optimization

```

1 //Complexity : O(n^2) for any k <= n
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define MAX 5005
5 #define ll long long int
6
7 ll M[MAX];
8 const ll INVALID = LLONG_MIN;
9 ll C[MAX][MAX], dp[MAX][MAX], Opt[MAX][MAX];
10 //Recurrence : dp[i][j] = min/max i <= k <= j (dp[i-1][k-1] + C[k][j])
11 //Condition : Opt[i-1][j] <= Opt[i][j] <= Opt[i][j+1]
12
13 int main(){
14     int N,K;
15     scanf("%d %d",&N,&K);
16     for(int i=1;i<=N;i++) scanf("%lld",&M[i]);
17     for(int i=1;i<=N;i++) for(int j=i;j<=N;j++) C[i][j]=C[i][j-1] | M[j];
18
19     for(int i=0;i<=K;i++) dp[i][0]=0;
20     for(int i=0;i<=K;i++) for(int j=1;j<=N;j++) dp[i][j]=INVALID;
21     for(int i=1;i<=N;i++) {Opt[0][i]=1; Opt[i][N+1]=N;}
22
23     for(int i=1;i<=K;i++){
24         for(int j=N;j>=1;j--){
25             for(int k=Opt[i-1][j];k<=Opt[i][j+1];k++){
26                 if(dp[i-1][k-1]== INVALID) continue;
27                 if(dp[i][j]<dp[i-1][k-1]+C[k][j]){
28                     dp[i][j]=dp[i-1][k-1]+C[k][j];
29                     Opt[i][j]=k;
30                 }
31             }
32         }
33     }
34     printf("%lld\n",dp[K][N]);
35 }

```

2.6 Li-Chao Segment Tree

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define ll long long int
4 const int MAX_X = 100005;
5 //domain of f(x) = [1, MAX_X-1]
6
7 struct Poly{
8     vector<ll> co;
9     Poly(){}
10    void add(ll c) {co.push_back(c);}
11    ll f(ll x){
12        ll ans = 0;
13        ll mul = 1;
14        for(ll c : co) {ans += mul * c; mul = mul * x;}
15        return ans;
16    }
17 };
18
19 #define Left (node*2)
20 #define Right (node*2+1)
21 #define mid ((lo+hi)/2)
22 Poly tree[4 * MAX_X];
23
24 void build(int node, int lo, int hi){
25     tree[node].co.clear();
26     tree[node].add(LLONG_MAX);
27     if(lo == hi) return;
28     build(Left, lo, mid);
29     build(Right, mid+1, hi);
30 }
31
32 void addLine(int node, int lo, int hi, Poly upd){
33     bool bame = upd.f(lo) < tree[node].f(lo);
34     bool majhe = upd.f(mid) < tree[node].f(mid);
35     if(majhe) swap(tree[node], upd);
36
37     if(lo == hi) return;
38     if(bame != majhe) addLine(Left, lo, mid, upd);
39     else addLine(Right, mid+1, hi, upd);
40 }
41
42 ll query(int node, int lo, int hi, ll x){
43     ll Curr = tree[node].f(x);
44     if(lo == hi) return Curr;
45     if(x <= mid) return min(Curr, query(Left, lo, mid, x));
46     else return min(Curr, query(Right, mid+1, hi, x));
47 }
48
49 // Comments :
50 // 1. Applies directly to lines in any domain space
51 // 2. In positive domain space, polynomials behave
52 // like lines after a certain threshold value MAGIC.
53 // Add a function as addLine(1,Magic,MAX_X-1, cur_poly)

```

```

54 // For query x < MAGIC, brute-force to calculate minimum
55 // For query x >= MAGIC, return query(1,Magic,MAX_X-1,x)

```

2.7 Li-Chao Tree (Sparse)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define ll long long int
4
5 const int maxk = 105;
6 const int maxn = 10005;
7 const int maxX = 1000005;
8 const ll inf = 1e12;
9
10 struct Poly{
11     vector<ll> co;
12     Poly() {}
13     void add(ll c) {co.push_back(c);}
14     ll f(ll x){
15         if(co.size() == 0) return inf;
16
17         ll ans = 0;
18         ll mul = 1;
19         for(ll c : co) {ans += mul * c; mul = mul * x;}
20         return ans;
21     }
22 };
23
24 struct lc{
25     Poly S;
26     lc *lft, *rgt;
27
28     lc() {lft = rgt = NULL; S = Poly();}
29
30     void Del(){
31         if(lft) lft->Del();
32         if(rgt) rgt->Del();
33         delete this;
34     }
35
36     void add(int lo, int hi, Poly upd){
37         if(upd.f(lo) > S.f(lo) && upd.f(hi) > S.f(hi)) return;
38
39         int mid = (lo + hi) / 2;
40         bool bame = upd.f(lo) < S.f(lo);
41         bool majhe = upd.f(mid) < S.f(mid);
42         if(majhe) swap(S, upd);
43         if(lo == hi) return;
44
45         if(bame != majhe){
46             if(!lft) lft = new lc();
47             lft->add(lo, mid, upd);

```

```

48     }
49     else{
50         if(!rgt) rgt = new lc();
51         rgt->add(mid+1, hi, upd);
52     }
53 }
54
55 ll query(int lo, int hi, ll x){
56     ll Curr = S.f(x);
57     if(lo == hi) return Curr;
58     int mid = (lo + hi) / 2;
59     if(x <= mid) return min(Curr, lft ? lft->query(lo, mid, x) : inf);
60     else return min(Curr, rgt ? rgt->query(mid+1, hi, x) : inf);
61 }
62 };

```

2.8 SOS DP

```

1 //SOS DP
2
3 for(int mask = 0; mask < (1<<N); mask++){
4     //handle base case separately (leaf states)
5     dp[mask][0] = A[mask];
6     for(int b = 0; b < N; b++){
7         if(mask & (1<<b)) dp[mask][b] = dp[mask][b-1] + dp[mask^(1<<b)][b-1];
8         else dp[mask][b] = dp[mask][b-1];
9     }
10    F[mask] = dp[mask][N-1];
11 }

```

2.9 Sequence DP

```

1 //Number of subsequence
2 Formula 1 :  $2^n$ 
3 Formula 2 :  $C(n,0) + C(n,1) + \dots + C(n,n)$ 
4 Formula 3 :  $f(n) = 2*f(n-1)$  //take s[n] or don't take
5
6
7 //Number of distinct subsequence
8 Formula 1 :  $f(n) = 2*f(n-1) - f(m)$  //here, m=index of previous occurrence of s[n]
9     ]
10     //m=0 if s[n] not found previously
11
12 //Longest Common Subsequence
13 Formula 1 :  $f(i,j)=0$  if  $i==0$ 
14     ||  $j==0$ 
15      $f(i,j)=1+f(i-1,j-1)$  if  $X[i]==Y[j]$ 
16      $f(i,j)=\max(f(i,j-1), f(i-1,j))$  if  $X[i]!=Y[j]$ 

```

```

15 Space Optimization trick: [2][N]
16 Time Optimization trick: use LIS in  $O(n \log n)$  if at most one string contains
    repetitions of characters
17
18 //Longest Repeating Subsequence
19 //Longest Subsequence that occurs at least twice in a string without overlapping
20 Formula 1 :  $f(i,j)=0$  if  $i==0$ 
21     ||  $j==0$ 
22      $f(i,j)=1+f(i-1,j-1)$  if  $s[i]==s[j]$  and  $i!=j$ 
23      $f(i,j)=\max(f(i,j-1), f(i-1,j))$  if  $s[i]!=s[j]$  or  $i==j$ 
24
25 //Edit Distance
26 //Operations: Insert, Delete, Replace
27 Formula 1 :  $f(i,j)=j$  /* insert Y[1...j] in X */ if  $i==0$ 
28      $f(i,j)=i$  /* insert X[1...i] in Y */ if  $j==0$ 
29      $f(i,j)=f(i-1,j-1)$  if  $X[i]==Y[j]$ 
30
31      $f(i,j)=1+\min(f(i,j-1), f(i-1,j), f(i-1,j-1))$  if  $X[i]!=Y[j]$ 
32
33 //Edit Distance
34 //Operations: Insert, Delete
35 Formula 1 :  $f(i,j)=\text{Size}(x) + \text{Size}(Y) - \text{LCS}(X,Y)$ 
36
37
38 //Number of palindromic subsequence
39 Formula 1 :  $f(i,j)=1$  if  $i==j$ 
40      $f(i,j)=f(i+1,j)+f(i,j-1)-f(i+1,j-1)+[f(i+1,j-1)+1]$  if  $s[i]==s[j]$ 
41
42      $f(i,j)=f(i+1,j)+f(i,j-1)-f(i+1,j-1)$  if  $s[i]!=s[j]$ 
43
44 //Number of distinct palindromic subsequence
45 Formula 1 :  $f(i,j,x)=0$  if  $i==j$ 
46     and  $s[i]!=x$ 
47      $f(i,j,x)=1$  if  $i==j$ 
48     and  $s[i]=x$ 
49      $f(i,j,x)=f(i+1,j,x)+f(i,j-1,x)-f(i+1,j-1,x)$  if  $s[i]!=x$  or  $s[j]!=x$ 
50      $f(i,j,x)=2 + \text{sum of all } y \text{ in } a-z (f(i+1,j-1,y))$  if  $s[i]==x$  and  $s[j]==x$ 
51     //2 added for xx and x[longest sequence]x
52
53 //Longest Palindromic Subsequence
54 Formula 1 :  $f(i,j)=0$  if  $i>j$ 
55      $f(i,j)=1$  if  $i==j$ 

```

```

56         f(i,j)=f(i+1,j-1)+2
57     s[j])
58         f(i,j)=max(f(i+1,j),f(i,j-1))
59     s[j])
60 //Longest Common Substring
61 Formula 1 :   f(i,j)=0
62               || j==0)
63     Y[j])
64         f(i,j)=1+f(i-1,j-1)
65     Y[j])
66         f(i,j)=0
67     Y[j])
68         ans=max of all f(i,j)
69
70 //Longest Palindromic Substring
71 Formula 1 :   isPal(i,j)=0
72               isPal(i,j)=1
73               isPal(i,j)=2+isPal(i+1,j-1)
74 and s[i]==s[j])
75               isPal(i,j)=0
76 and s[i]!=s[j])

```

2.10 LCS (Faster)

```

1 //Complexity : O(n*m / 64)
2
3 #define MAX 100010
4 #define ull unsigned long long
5
6 bool flag[MAX];
7 char A[MAX], B[MAX], S[2][MAX];
8
9 int lcs(char* A, char* B){
10     int n, m, res = 0;
11     ull mask[128] = {0};
12
13     memset(flag, 0, sizeof(flag));
14     for(n = 0; A[n]; n++) S[0][n] = A[n];
15     for(m = 0; B[m]; m++) S[1][m] = B[m];
16
17     for(int i = 0; (i * 64) < m; i++){
18         memset(mask, 0, sizeof(mask));
19         for(int k = 0; k < 64 && (i * 64 + k) < m; k++){
20             mask[S[1][i * 64 + k]] |= (1ULL << k);
21         }
22         ull x = 0;
23         for(int j = 0; j < n; j++){
24             ull t = mask[S[0][j]] & ~x;
25             x |= t;
26             ull v = flag[j];

```

```

if(s[i]==27
28
29 if(s[i]!=29
30
31 if(v) x &= ~1ULL;
32
33 res += __builtin_popcountll(x);
34
35 return res;
36 }
}
if(X[i]!=

```

```

if(i>j)
if(i==j)
if(i<j
if(i<j

```

```

ull q = x - (t << 1) - v;
ull y = (q & ~x) | t;
flag[j] = y >> 63;
x &= ~(y << 1);
if(v) x &= ~1ULL;
}
res += __builtin_popcountll(x);
}
return res;
}

```

2.11 SteinerTree

```

1 struct edge{
2     int t , v;
3     edge(int t=0 ,int v=0):t(t),v(v){}
4 };
5 // f[u][v] = shortest Path between u to v
6 // D[r][mask] = minimum cost stainer tree with root r and covering all the nodes
7 // in the mask.
8
9 struct StreinerTree{
10     int n;
11     int f[maxn][maxn] , d[maxn][1<<maxk];
12     void init(int n) {
13         this->n = n;
14         for(int i=1;i<=n;i++) for(int j=1;j<=n;j++) f[i][j] = INF;
15     }
16     void addEdge(int ff , int t , int v) {
17         f[ff][t] = min(v , f[ff][t]);
18         f[t][ff] = min(v , f[t][ff]);
19     }
20
21     void floyd() {
22         for(int k=1;k<=n;k++) for(int i=1;i<=n;i++) for(int j=1;j<=n;j++)
23             f[i][j] = min(f[i][j] , f[i][k]+f[k][j]);
24     }
25
26     void mincostST(vector<int> s) {
27         for(int i=1;i<=n;i++) for(int j=0;j<(1<<s.size());j++) d[i][j] = INF;
28         for(int i=0;i<s.size();i++) d[s[i]][1<<i] = 0;
29         for(int i=1;i<(1<<s.size());i++) {
30             for(int j=1;j<=n;j++) for(int ii = (i-1)&i;ii>0;ii = (ii-1)&i)
31                 d[j][ii] = min(d[j][ii] , d[j][ii]+d[j][i-ii]);
32             for(int j=1;j<=n;j++) for(int k=1;k<=n;k++) d[j][i] = min(d[j][i] , d
33                 [k][i]+f[j][k]);
34         }
35     };

```

3 DS

3.1 BIT (1D)

```

1 //Tested : LightOJ 1164 - Horrible Queries
2
3 #include<bits/stdc++.h>
4 using namespace std;
5 #define ll long long int
6 const int MAXN = 1000005;
7
8 ll BIT[2][MAXN];
9 void update(int cs, int indx, ll val){
10     while(indx < MAXN){
11         BIT[cs][indx] += val;
12         indx += (indx & -indx);
13     }
14 }
15
16 ll sum(int cs, int indx){
17     ll ans = 0;
18     while(indx != 0) {
19         ans += BIT[cs][indx];
20         indx -= (indx & -indx);
21     }
22     return ans;
23 }
24
25 void updateRange(int l, int r, ll val){
26     update(0,l,val);      update(0,r+1,-val);
27     update(1,l,val*(l-1)); update(1,r+1,-val*r);
28 }
29
30 ll sumRange(int indx) {return sum(0,indx)*indx - sum(1,indx);}
31 ll QueryRange(int l, int r) {return sumRange(r)-sumRange(l-1);}
32
33 const int LOGN = 20;
34 int LowerBound(int cs, ll v){
35     ll sum = 0;
36     int indx = 0;
37     for(int i = LOGN; i >= 0; i--){
38         int nPos = indx + (1<<i);
39         if(nPos < MAXN && sum + BIT[cs][nPos] < v){
40             sum += BIT[cs][nPos];
41             indx = nPos;
42         }
43     }
44     //pos = maximal x such that Sum(x) < v
45     return indx + 1; //+1 for LowerBound
46 }

```

3.2 BIT (2D - Point)

```

1 //Tested : LightOJ 1266 - Points in Rectangle
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 #define ll long long int
6 const int MAXN = 1005;
7 int BIT[MAXN][MAXN];
8
9 //A[x][y] += val
10 void update(int x, int y, int val){
11     int xx = x;
12     while(xx < MAXN){
13         int yy = y;
14         while(yy < MAXN){
15             BIT[xx][yy] += val;
16             yy += (yy & -yy);
17         }
18         xx += (xx & -xx);
19     }
20 }
21
22 //return sum over all A[i][j], where 1 <= i <= x and 1 <= j <= y
23 int sum(int x, int y){
24     int ans = 0;
25     int xx = x;
26     while(xx != 0){
27         int yy = y;
28         while(yy != 0){
29             ans += BIT[xx][yy];
30             yy -= (yy & -yy);
31         }
32         xx -= (xx & -xx);
33     }
34     return ans;
35 }

```

3.3 BIT (2D - Range)

```

1 //Tested : https://www.spoj.com/problems/USUBQSUB
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 #define ll long long int
6 const int MAXN = 1005;
7
8 struct BIT2D{
9     ll tree[4][MAXN][MAXN];
10     BIT2D() {memset(tree,0,sizeof(tree));}
11 }

```

```

12 //Add v to submatrix [i,j] to [inf,inf]
13 void update(int p, int q, ll v){
14     if((p <= 0) || (q <= 0) || (p >= MAXN) || (q >= MAXN)) return;
15
16     int i = p, c = p - 1, d = q - 1;
17     while(i < MAXN){
18         int j = q;
19         while(j < MAXN){
20             tree[0][i][j] += v;          tree[1][i][j] += (v * d);
21             tree[2][i][j] += (v * c);    tree[3][i][j] += (v * c * d);
22             j += (j & -j);
23         }
24         i += (i & -i);
25     }
26 }
27
28 //returns Sum over submatrix [1,1] to [p,q]
29 ll query(int p, int q){
30     int i, j;
31     ll x, y, z, c, d, res;
32
33     i = p, x = y = z = 0;
34     while(i != 0){
35         j = q, c = d = 0;
36         while(j != 0){
37             c += tree[0][i][j];          d += tree[1][i][j];
38             y += tree[2][i][j];          z += tree[3][i][j];
39             j ^= (j & (-j));
40         }
41         i ^= (i & -i);
42         x += (c * q - d);
43     }
44     res = (x * p) - (y * q) + z;
45     return res;
46 }
47
48 //Add v to submatrix [i,j] to [k,l]
49 void update(int i, int j, int k, int l, ll v){
50     update(i, j, v);          update(k + 1, j, -v);
51     update(k + 1, l + 1, v);  update(i, l + 1, -v);
52 }
53
54 //returns Sum over submatrix [i,j] to [k,l]
55 ll query(int i, int j, int k, int l){
56     if (i > k || j > l) return 0;
57     ll res = query(k, l) - query(i - 1, l) + query(i - 1, j - 1) - query(k, j
58     - 1);
59     return res;
60 }
61 };

```

3.4 BIT (3D)

```

1 //Point Update and Range Query similar as 2D BIT
2
3 void update(int x1, int y1, int z1, int x2, int y2, int z2){
4     update(x2, y2, z2, 1),          update(x1 - 1, y1 - 1, z2, 1);
5     update(x1 - 1, y2, z1 - 1, 1),  update(x2, y1 - 1, z1 - 1, 1);
6     update(x1 - 1, y2, z2, -1),     update(x2, y1 - 1, z2, -1);
7     update(x2, y2, z1 - 1, -1),     update(x1 - 1, y1 - 1, z1 - 1, -1);
8 }

```

3.5 Closest Pair

```

1 // Tested : UVa 10245 - The Closest Pair Problem
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 #define ll long long int
6
7 struct Point{
8     ll x, y;
9     Point() {}
10     Point(ll _x, ll _y) {x = _x; y = _y;}
11     bool operator < (const Point &p) const {return x == p.x ? y < p.y : x < p.x;}
12     Point operator - (Point p) {return Point(x - p.x, y - p.y);}
13 };
14 ll getDot(Point a, Point b) {return a.x * b.x + a.y * b.y;}
15 ll dist(Point p, Point q) {return getDot(p-q, p-q);}
16
17 vector<Point> p;
18 ll solve(int l, int r) {
19     if(r - l <= 3) {
20         ll ret = LLONG_MAX;
21         for(int i = l; i <= r; i++)
22             for(int j = i + 1; j <= r; j++)
23                 ret = min(ret, dist(p[i], p[j]));
24         return ret;
25     }
26
27     int mid = (l + r) / 2;
28     ll d = min(solve(l, mid), solve(mid+1, r));
29
30     vector<Point> t;
31     for(int i = l; i <= r; i++){
32         ll dx = p[mid].x - p[i].x;
33         if(dx * dx <= d) t.push_back({p[i].y, p[i].x});
34     }
35
36     sort(t.begin(), t.end());
37     for(int i = 0; i < t.size(); i++) {
38         for(int j = i+1; j < t.size() && j <= i + 15; j++)

```

```

39         d = min(d, dist(t[i], t[j]));
40     }
41     return d;
42 }
43
44 ll closestPair() {
45     sort(p.begin(), p.end());
46     return solve(0, p.size()-1);
47 }

```

3.6 HashTable

```

1 #include <bits/stdc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 using namespace std;
4 using namespace __gnu_pbds;
5
6 const int RANDOM = chrono::high_resolution_clock::now().time_since_epoch().count
    ();
7 unsigned hash_f(unsigned x) {
8     x = ((x >> 16) ^ x) * 0x45d9f3b;
9     x = ((x >> 16) ^ x) * 0x45d9f3b;
10    x = (x >> 16) ^ x;
11    return x;
12 }
13
14 unsigned hash_combine(unsigned a, unsigned b) { return a * 31 + b; }
15 struct chash {int operator()(int x) const { return hash_f(x); }};
16 typedef gp_hash_table<int, int, chash>gp;
17 gp table;

```

3.7 KD Tree

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 const int N = 200555, K = 2;
4 typedef long long ll;
5 #define sqr(x) ((x) * (x))
6
7 int k = 2, idx;
8 // idx = current dimension, k = total dimension
9
10 struct Point{
11     int pr, id;
12     ll x[K];
13     bool operator < (const Point &u) const {return x[idx] < u.x[idx];}
14 }po[N];
15
16 typedef pair < ll , Point > tp;
17 priority_queue < tp > nq;

```

```

18
19 struct KDtree{
20     Point pt[N<<2];
21     int son[N<<2], mn[N<<2];
22
23     void init(){
24         memset(son, 0, sizeof(son));
25         memset(mn, 0, sizeof(mn));
26     }
27
28     void build(int l, int r, int node = 1, int dep = 0)    {
29         if(l > r) return;
30         son[node] = r - 1;
31         son[node * 2] = son[node * 2 + 1] = -1;
32         idx = dep % k;
33         int mid = (l + r) / 2;
34         nth_element(po + l, po + mid, po + r + 1);
35         pt[node] = po[mid];
36         mn[node] = po[mid].pr;
37
38         build(l, mid - 1, node * 2, dep + 1);
39         build(mid + 1, r, node * 2 + 1, dep + 1);
40
41         mn[node] = min(mn[node], min(mn[node * 2], mn[node * 2 + 1]));
42     }
43
44     void query(Point p, int m, int node = 1, int dep = 0){
45         if(son[node] == -1) return;
46         tp nd(0, pt[node]);
47         for(int i = 0; i < k; i++) nd.first += sqr(nd.second.x[i] - p.x[i]);
48         int dim = dep % k;
49         int lft = 2 * node, rgh = 2 * node + 1, fg = 0;
50         if(p.x[dim] >= pt[node].x[dim]) swap(lft, rgh);
51
52         if(nd.second.pr > p.pr) nd.first = 1e18;
53         if(~son[lft] && mn[lft] <= p.pr) query(p, m, lft, dep + 1);
54
55         if(nq.size() < m) nq.push(nd), fg = 1;
56         else{
57             if(nd.first < nq.top().first) nq.pop(), nq.push(nd);
58             else if(nd.first == nq.top().first && nd.second.id < nq.top().second.
59 id) nq.pop(), nq.push(nd);
60             if(sqr(p.x[dim] - pt[node].x[dim]) <= nq.top().first) fg = 1;
61         }
62         if(~son[rgh] && fg && mn[rgh] <= p.pr) query(p, m, rgh, dep + 1);
63     }kdt;
64 int T, n, m, root;
65
66 int main(){
67     cin >> T;
68
69     while(T--){

```

```

70     scanf("%d %d", &n, &m);
71     for(int i = 1; i <= n; i++){
72         scanf("%lld %lld %lld", &po[i].x[0], &po[i].x[1], &po[i].pr);
73         po[i].id = i;
74     }
75
76     kdt.build(1, n);
77     while(m--){
78         Point p;
79         scanf("%lld %lld %lld", &p.x[0], &p.x[1], &p.pr);
80         kdt.query(p, 1);
81         if(nq.empty()){
82             printf("-1\n");
83             continue;
84         }
85         printf("%lld %lld %d\n", nq.top().second.x[0], nq.top().second.x[1],
86         nq.top().second.pr);
87         while(!nq.empty()) nq.pop();
88     }
89 }

```

3.8 MO (Basic)

```

1 namespace MO{
2     const int MAXN = 100005;
3     const int MAXQ = 100005;
4
5     int Sz;
6     int A[MAXN];
7     int blkId[MAXN];
8     bool vis[MAXN];
9
10    struct Query{
11        int L,R,id;
12        Query(){
13            Query(int x,int y,int i){L=x;R=y;id=i;}
14        bool operator<(const Query other) const{
15            int a = blkId[L]; int b = blkId[other.L];
16            return a == b ? (a & 1 ? (R > other.R) : (R < other.R)) : a < b;
17        }
18    }qry[MAXQ];
19    int perQ[MAXQ];
20
21    int MaxFreq = 0;
22    int CoC[MAXN];
23    int Count[MAXN];
24    void Check(int x){
25        if(!vis[x]){
26            vis[x]=1;
27            if(Count[A[x]] CoC[Count[A[x]]]--;
28            Count[A[x]]++; CoC[Count[A[x]]]++;

```

```

29            if(CoC[MaxFreq + 1]) MaxFreq++;
30        }
31        else{
32            vis[x]=0;
33            CoC[Count[A[x]]]--; Count[A[x]]--;
34            if(Count[A[x]] CoC[Count[A[x]]]++;
35            if(CoC[MaxFreq] == 0) MaxFreq--;
36        }
37    }
38 }
39 using namespace MO;
40
41 int main(){
42     int N,Q;
43     scanf("%d %d", &N, &Q);
44
45     //Initiate Global
46     Sz=sqrt(N);
47     memset(vis,0,sizeof(vis));
48     memset(Count,0,sizeof(Count));
49     for(int i=0;i<=N;i++) blkId[i] = i/Sz;
50
51     for(int i=1;i<=N;i++) scanf("%d",&A[i]);
52     for(int i=1;i<=Q;i++){
53         int l,r;
54         scanf("%d %d",&l,&r);
55         qry[i] = Query(l,r,i);
56     }
57     sort(qry+1,qry+Q+1);
58
59     int left = qry[1].L;
60     int right = left-1;
61
62     for(int i=1;i<=Q;i++){
63         Query now = qry[i];
64         while(left<now.L) Check(left++);
65         while(left>now.L) Check(--left);
66         while(right<now.R) Check(++right);
67         while(right>now.R) Check(right--);
68         perQ[now.id] = MaxFreq;
69     }
70 }

```

3.9 MO on Tree

```

1 //MO on Tree
2 int Time = 0;
3 int Euler[2*MAXN];
4 vector<int> adj[MAXN];
5 int Par[MAXN], D[MAXN], st[MAXN], en[MAXN];
6
7 void dfs(int u,int p){

```

```

8   st[u] = ++Time; Euler[Time] = u;
9   Par[u] = p;
10  D[u] = D[p] + 1;
11
12  for(int v : adj[u]){
13      if(v == p) continue;
14      dfs(v,u);
15  }
16  en[u] = ++Time; Euler[Time] = u;
17 }
18 using namespace MO;
19 using namespace LCA;
20
21 int main(){
22     int N,Q;
23     scanf("%d %d",&N,&Q);
24
25     //Input
26     Time = 0;
27     dfs(1,0);
28     BuildSparse(N);
29
30     for(int i=1;i<=Q;i++){
31         int u,v;
32         scanf("%d %d",&u,&v);
33
34         if(st[u]>st[v]) swap(u,v);
35         int lca = LCA(u,v);
36         if(lca == u) qry[i] = Query(st[u],st[v],i);    //Case 1 : range(st[u],st[v]
37     })
38         else qry[i] = Query(en[u],st[v],i);           //Case 2 : range(en[u],st[v]
39     }) and LCA(u,v)
40     }
41
42     Sz = sqrt(N);
43     sort(qry+1,qry+Q+1);
44
45     int left = qry[1].L;
46     int right = left-1;
47     for(int i=1;i<=Q;i++){
48         Query now = qry[i];
49         while(left<now.L) Check(Euler[left++]);
50         while(left>now.L) Check(Euler[--left]);
51         while(right<now.R) Check(Euler[++right]);
52         while(right>now.R) Check(Euler[right--]);
53
54         int lca = LCA(Euler[left],Euler[right]);
55         if(Euler[left]!=lca && Euler[right]!=lca) Check(lca); //Adding LCA(u,v)
56         for Case 2
57         Ans[now.id]=MaxFreq;
58         if(Euler[left]!=lca && Euler[right]!=lca) Check(lca); //Discarding the
59         Addition
60     }

```

```

57 }

```

3.10 MOs algo with updates

```

1  #define ll long long int
2  const int MAXN = 300005;
3  const int MAXQ = 300005;
4
5  int Sz;
6  ll currAns;
7  struct Query{
8      int L,R,t,id;
9      Query() {}
10     Query(int _L,int _R,int _upd,int _id) {L=_L; R=_R; t=_upd; id=_id;}
11     bool operator < (const Query o){
12         if(L/Sz != o.L/Sz) return L < o.L;
13         if(R/Sz != o.R/Sz) return R < o.R;
14         return t < o.t;
15     }
16 }OnlyQry[MAXQ];
17
18 ll ans[MAXQ];
19 tuple<int,int,int> AllQ[MAXQ], OnlyUpd[MAXQ];
20
21 vector<int> forsort;
22 unordered_map<int, int> ID;
23
24 bool vis[MAXN];
25 int A[MAXN], Count [MAXN];
26 int dummy[MAXN], ValBeforeUpd[MAXN];
27
28 void DoUpdate(int idx){
29     int i = get<1>(OnlyUpd[idx]);
30     int Prv = get<2>(OnlyUpd[idx]);
31     int New = A[i]; A[i] = Prv;
32     if(!vis[i]) return;
33     if(New != -1) {Count[New]--; if(Count[New] == 0) currAns -= forsort[New -
34     1];}
35     if(Prv != -1) {Count[Prv]++; if(Count[Prv] == 1) currAns += forsort[Prv -
36     1];}
37 }
38
39 void UnDoUpdate(int idx){
40     int i = get<1>(OnlyUpd[idx]);
41     int Prv = ValBeforeUpd[idx];
42     int New = A[i]; A[i] = Prv;
43     if(!vis[i]) return;
44     if(New != -1) {Count[New]--; if(Count[New] == 0) currAns -= forsort[New -
45     1];}
46     if(Prv != -1) {Count[Prv]++; if(Count[Prv] == 1) currAns += forsort[Prv -
47     1];}
48 }

```

```

45 void Add(int x){
46     if(A[x] == -1) return;
47     if(Count[A[x]]==0) currAns += forsort[A[x]-1]; Count[A[x]]++;
48 }
49
50 void Remove(int x){
51     if(A[x] == -1) return;
52     Count[A[x]]--; if(Count[A[x]]==0) currAns -= forsort[A[x]-1];
53 }
54
55 void Check(int x){
56     if(!vis[x]) Add(x);
57     else Remove(x);
58     vis[x] ^= 1;
59 }
60
61 int main(){
62     int N,Q;
63     scanf("%d %d",&N,&Q);
64     Sz = N/cbrt(N);
65
66     for(int i=1;i<=N;i++) {
67         scanf("%lld",&A[i]);
68         if(ID[A[i]]) continue;
69         ID[A[i]] = 1;
70         forsort.push_back(A[i]);
71     }
72
73     int UpdNo = 0;
74     int QryNo = 0;
75     for(int i=1;i<=Q;i++){
76         int tp,a,b;
77         scanf("%d %d %d",&tp,&a,&b);
78         AllQ[i] = make_tuple(tp,a,b);
79
80         if(tp == 0){
81             OnlyUpd[++UpdNo] = AllQ[i];
82             if(ID[b]) continue;
83             ID[b] = 1;
84             forsort.push_back(b);
85         }
86         else OnlyQry[++QryNo] = Query(a,b,UpdNo,QryNo);
87     }
88
89     sort(forsort.begin(), forsort.end());
90     int Cnt = 0;
91     for(int x : forsort){
92         Cnt++;
93         if(x % 3) ID[x] = -1;
94         else ID[x] = Cnt;
95     }
96
97

```

```

98     for(int i=1;i<=N;i++) A[i] = ID[A[i]], dummy[i] = A[i];
99     for(int i=1;i<=Q;i++) if(get<0>(AllQ[i]) == 0) get<2>(AllQ[i]) = ID[get<2>(
100     AllQ[i])];
101     for(int i=1;i<=UpdNo;i++) get<2>(OnlyUpd[i]) = ID[get<2>(OnlyUpd[i])];
102
103     for(int i=1;i<=UpdNo;i++){
104         ValBeforeUpd[i] = dummy[get<1>(OnlyUpd[i])];
105         dummy[get<1>(OnlyUpd[i])] = get<2>(OnlyUpd[i]);
106     }
107     sort(OnlyQry+1, OnlyQry+QryNo+1);
108
109     int left = 0;
110     int right = left - 1;
111     int currUpd = 0;
112     for(int i=1;i<=QryNo;i++){
113         int L = OnlyQry[i].L;
114         int R = OnlyQry[i].R;
115         int Upd = OnlyQry[i].t;
116
117         while(currUpd < Upd) DoUpdate(++currUpd);
118         while(currUpd > Upd) UnDoUpdate(currUpd--);
119
120         while(left<L) Check(left++);
121         while(left>L) Check(--left);
122         while(right<R) Check(++right);
123         while(right>R) Check(right--);
124         ans[OnlyQry[i].id] = currAns;
125     }
126     for(int i=1;i<=QryNo;i++) printf("%lld\n",ans[i]);

```

3.11 OrderedSet

```

1 #include<bits/stdc++.h>
2 #include <ext/pb_ds/assoc_container.hpp> // Common file
3 #include <ext/pb_ds/tree_policy.hpp> // Including
4     tree_order_statistics_node_update
5 #include <ext/pb_ds/detail/standard_policies.hpp>
6 using namespace std;
7 using namespace __gnu_pbds;
8
9 typedef tree<
10     int,null_type,less<int>,
11     rb_tree_tag,
12     tree_order_statistics_node_update>
13     ordered_set;
14 ordered_set xx;

```

3.12 Parallel Binary Search

```

1 //Tested : Atcoder - Stamp Rally
2
3 #include<bits/stdc++.h>
4 using namespace std;
5
6 const int MAXN = 100005;
7 const int MAXM = 100005;
8 const int MAXQ = 100005;
9
10 TYPE ds; //Size MAXN
11 TYPE Update[MAXM];
12 TYPE Query[MAXQ];
13 vector<int> Check[MAXM];
14 int lo[MAXQ], hi[MAXQ];
15
16 // For each qry in Query, find minimum upd such that condistions
17 // for qry are satisfied after applying Update[1...upd] to ds
18 void Solve(int n, int m, int q){
19     for(int qry=1; qry<=q; qry++) lo[qry] = 1, hi[qry] = m;
20
21     bool Changed = true;
22     while(Changed){
23         Changed = false;
24         // Clear and Re-initialize ds upto size n
25         for(int upd=1; upd<=m; upd++) Check[upd].clear();
26
27         for(int qry=1; qry<=q; qry++){
28             if(lo[qry] == hi[qry]) continue;
29             Changed = true;
30             int mid = (lo[qry] + hi[qry]) / 2;
31             Check[mid].push_back(qry);
32         }
33
34         for(int upd=1; upd<=m; upd++){
35             //Apply Update[upd]
36             for(int qry : Check[upd]){
37                 bool ok = [conditions for Query[qry] is satisfied]
38                 if(ok) hi[qry] = upd;
39                 else lo[qry] = upd + 1;
40             }
41         }
42     }
43 }

```

3.13 MergeSortTree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MAXN = 100005;
4
5 int A[MAXN];
6 vector<int> tree[MAXN*5];

```

```

7
8 #define Left (node*2)
9 #define Right (node*2+1)
10 #define mid ((lo+hi)/2)
11
12 void build(int node, int lo, int hi){
13     if(lo==hi) {tree[node].push_back(A[lo]); return;}
14     build(Left,lo,mid);
15     build(Right,mid+1,hi);
16     merge(tree[Left].begin(),tree[Left].end(),tree[Right].begin(),tree[Right].end()
17           (),back_inserter(tree[node]));
18 }
19
20 int query(int node, int lo, int hi, int i, int j, int x){
21     if(i>hi || j<lo) return 0;
22     if(lo>=i && hi<=j) return lower_bound(tree[node].begin(),tree[node].end(),x)
23     - tree[node].begin();
24     int p1 = query(Left,lo,mid,i,j,x);
25     int p2 = query(Right,mid+1,hi,i,j,x);
26     return p1+p2;
27 }

```

3.14 SegTreeBeats

```

1 // Tested : HDU - 5306
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 const int maxn = 1000005;
6 #define ll long long int
7
8 int A[maxn];
9 struct data{
10     int fmx, smx, fmcnt, ll sum;
11 };
12
13 data Merge(data A, data B){
14     data curr;
15
16     curr.sum = A.sum + B.sum;
17     curr.fmx = max(A.fmx, B.fmx);
18     curr.smx = max(A.smx, B.smx);
19     if(A.fmx != B.fmx) curr.smx = max(curr.smx, min(A.fmx, B.fmx));
20
21     curr.fmcnt = 0;
22     if(curr.fmx == A.fmx) curr.fmcnt += A.fmcnt;
23     if(curr.fmx == B.fmx) curr.fmcnt += B.fmcnt;
24     return curr;
25 }
26
27 data tree[maxn*5];
28 #define Left (node * 2)

```

```

29 #define Right ((node << 1) ^ 1)
30 #define mid ((lo+hi) >> 1)
31
32 void build(int node, int lo, int hi){
33     if (lo==hi) {tree[node] = {A[lo], -1, 1, A[lo]}; return;}
34     build(Left,lo,mid);
35     build(Right,mid+1,hi);
36     tree[node] = Merge(tree[Left], tree[Right]);
37 }
38
39 void lazyPropagation(int node,int lo,int hi){
40     if(lo == hi) return;
41
42     int val = tree[node].fmx;
43     if(tree[Left].fmx > val){
44         tree[Left].sum -= (tree[Left].fmx - val) * 1LL * tree[Left].fmcnt;
45         tree[Left].fmx = val;
46     }
47
48     if(tree[Right].fmx > val){
49         tree[Right].sum -= (tree[Right].fmx - val) * 1LL * tree[Right].fmcnt;
50         tree[Right].fmx = val;
51     }
52 }
53
54 void updateRange(int node, int lo, int hi, int i, int j, int val){
55     if(tree[node].fmx <= val) return;
56     if(lo>=i && hi<=j && tree[node].smx < val){
57         tree[node].sum -= (tree[node].fmx - val) * 1LL * tree[node].fmcnt;
58         tree[node].fmx = val;
59         return;
60     }
61     if(lo == hi) return;
62
63     lazyPropagation(node,lo,hi);
64     if(i <= mid) updateRange(Left,lo,mid,i,j,val);
65     if(j > mid) updateRange(Right,mid+1,hi,i,j,val);
66     tree[node] = Merge(tree[Left], tree[Right]);
67 }
68
69 ll querySum(int node,int lo,int hi,int i,int j){
70     if(lo>=i && hi <= j) return tree[node].sum;
71
72     ll ret = 0;
73     lazyPropagation(node,lo,hi);
74     if(i <= mid) ret += querySum(Left,lo,mid,i,j);
75     if(j > mid) ret += querySum(Right,mid+1,hi,i,j);
76     return ret;
77 }
78
79 int queryMax(int node,int lo,int hi,int i,int j){
80     if(lo>=i && hi <= j) return tree[node].fmx;
81

```

```

82     int ret = -1;
83     lazyPropagation(node,lo,hi);
84     if(i <= mid) ret = max(ret, queryMax(Left,lo,mid,i,j));
85     if(j > mid) ret = max(ret, queryMax(Right,mid+1,hi,i,j));
86     return ret;
87 }

```

3.15 Segment Tree (2D)

```

1 // 2D Segment Tree
2 // Memory Complexity : O(16nm)
3 // Be careful writing lx, rx, ly, ry
4
5 const int MAXN = 1030;
6 int tree[4*MAXN][4*MAXN];
7
8 int n, m;
9 void updateY(int ndx, int lx, int rx, int ndy, int ly, int ry, int y, int val) {
10     if(ly == ry) {
11         if(lx == rx) tree[ndx][ndy] = val;
12         else tree[ndx][ndy] = tree[ndx*2][ndy] + tree[ndx*2+1][ndy];
13         return;
14     }
15     int mid = (ly + ry) >> 1;
16     if(y <= mid) updateY(ndx, lx, rx, ndy*2, ly, mid, y, val);
17     else updateY(ndx, lx, rx, ndy*2+1, mid+1, ry, y, val);
18     tree[ndx][ndy] = tree[ndx][ndy*2] + tree[ndx][ndy*2+1];
19 }
20
21 void updateX(int ndx, int lx, int rx, int x, int y, int val) {
22     if(lx != rx) {
23         int mid = (lx + rx) >> 1;
24         if(x <= mid) updateX(ndx*2, lx, mid, x, y, val);
25         else updateX(ndx*2+1, mid+1, rx, x,y, val);
26     }
27     updateY(ndx, lx, rx, 1, 1, m, y,val);
28 }
29
30 int queryY(int ndx, int ndy, int ly, int ry, int y1, int y2) {
31     if(ry < y1 || ly > y2) return 0;
32     if(y1 <= ly && ry <= y2) return tree[ndx][ndy];
33
34     int mid = (ly + ry) >> 1;
35     return queryY(ndx, ndy*2, ly, mid, y1, y2) + queryY(ndx, ndy*2+1, mid+1, ry, y1, y2);
36 }
37
38 int queryX(int ndx, int lx, int rx, int x1, int y1, int x2, int y2) {
39     if(rx < x1 || lx > x2) return 0;
40     if(x1 <= lx && rx <= x2) return queryY(ndx, 1, 1, m, y1, y2);
41
42     int mid = (lx + rx) >> 1;

```

```

43     return queryX(ndx*2, lx, mid, x1,y1,x2,y2) + queryX(ndx*2+1, mid+1, rx, x1,y1
44 }, x2,y2);

```

3.16 Segment Tree (Iterative)

```

1  int n;
2  int tree[MAX*2];
3
4  //point update, range query
5  //initial elements at tree[n]...tree[2*n-1]
6  void build(){
7      for(int i=n-1;i>=1;i--){
8          tree[i]=max(tree[i<<1],tree[i<<1|1]);
9      }
10
11 void update(int p,int value){
12     for(tree[p+=n]=value; p>1; p>>=1){
13         tree[p>>1]=max(tree[p],tree[p^1]);
14     }
15
16 //outputs max(l,r-1)
17 int query(int l,int r){
18     int res=0;
19     for(l+=n, r+=n; l<r; l>>=1, r>>=1){
20         if(l&1) res=max(res,tree[l++]);
21         if(r&1) res=max(res,tree[--r]);
22     }
23     return res;
24 }

```

3.17 Segment Tree (Persistent)

```

1  //Persistent Segment Tree
2  struct node{
3      node *left,*right; int val;
4      node(){val=0;left=NULL;right=NULL;}
5      node(int a,node *b,node *c){val=a;left=b;right=c;}
6
7      node *update(int lo,int hi,int i,int v){
8          node* ret = new node(val,left,right);
9
10         if(lo==hi){ret->val += v; return ret;}
11         int mid=(lo+hi)/2;
12
13         if(i<=mid){
14             if(!left) left = new node();
15             ret->left = left->update(lo,mid,i,v);
16         }
17         else{

```

```

18         if(!right) right = new node();
19         ret->right = right->update(mid+1,hi,i,v);
20     }
21
22     ret->val=0;
23     if(ret->left) ret->val += ret->left->val;
24     if(ret->right) ret->val += ret->right->val;
25
26     return ret;
27 }

```

```

28
29 int query(int lo,int hi,int i,int j){
30     if(hi<i || lo>j) return 0;
31     if(i<=lo && hi<=j) return val;
32     int mid=(lo+hi)/2;
33
34     int ret = 0;
35     if(left) ret += left->query(lo,mid,i,j);
36     if(right) ret += right->query(mid+1,hi,i,j);
37     return ret;
38 }
39 };

```

```

40
41 inline int Val(node* x) {return x ? x->val : 0;}
42 inline node* Left(node* x) {return x ? x->left : NULL;}
43 inline node* Right(node* x) {return x ? x->right : NULL;}
44
45 //Searching kth minimum element in sorted order
46 int Search(node *a,node *b,node *c,node *d,int l,int r,int k){
47     if(l==r) return l;
48     int Count=Val(Left(a))+Val(Left(b))-Val(Left(c))-Val(Left(d));
49
50     int mid=(l+r)/2;
51     if(Count>=k) return Search(Left(a),Left(b),Left(c),Left(d),l,mid,k);
52     else return Search(Right(a),Right(b),Right(c),Right(d),mid+1,r,k-Count);
53 }
54
55 void dfs(int u,int p,int d){
56     .....
57     root[u]=root[p]->update(1,Size,w[u],1);
58     .....
59 }
60
61 int main(){
62     .....
63     dfs(1,0,0);
64     Size=Map.size(); //number of distinct elements after co-ordinate compression
65     root[0]=new node();
66     .....
67 }

```

3.18 Segment Tree (Recursive)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MAXN = 100005;
4
5 int A[MAXN];
6 int tree[MAXN*5];
7
8 #define Left (node*2)
9 #define Right (node*2+1)
10 #define mid ((lo+hi)/2)
11
12 void build(int node, int lo, int hi){
13     if (lo==hi) {tree[node] = A[lo]; return;}
14     build(Left,lo,mid);
15     build(Right,mid+1,hi);
16     tree[node] = tree[Left] + tree[Right];
17 }
18
19 int query(int node, int lo, int hi, int i, int j){
20     if (i>hi || j<lo) return 0;
21     if (lo>=i && hi<=j) return tree[node];
22     int p1 = query(Left,lo,mid,i,j);
23     int p2 = query(Right,mid+1,hi,i,j);
24     return p1 + p2;
25 }
26
27 void update(int node, int lo, int hi, int i, int val){
28     if (i>hi || i<lo) return;
29     if (lo==hi) {tree[node] = val; return;}
30     update(Left,lo,mid,i,val);
31     update(Right,mid+1,hi,i,val);
32     tree[node] = tree[Left] + tree[Right];
33 }
34
35 /*
36 Lazy Propagation
37 1. Lets define a range update operation on segment tree OP. If there is a lazy
38 value x in some node u from some updates, its guaranteed that this lazy value
39 has been already propagated from each node v in the path from root to u. If
40 there is still some lazy value at in a node v in the path from root to u, then
41 that is from some later updates which has not been propagated up-to u yet.
42
43 2. Let there be multiple type of range updates. Build a struct called LazyStruct
44 and keep a lazy variable for each type of update. Note that,
45 a) for ASSIGN update, clear each other type of lazy variables.
46 b) while handling ADD/SUB/COUNT lazy in a node, first propagate down any ASSIGN
47 lazy it already has. This ensures the correct order of the updates.
48 */
49
50 int lazy[MAXN*5];
51 void lazyPropagation(int node,int lo,int hi){

```

```

52     if(lazy[node]){
53         tree[node] += (hi-lo+1)*lazy[node];
54         if(lo != hi) {lazy[Left] += lazy[node]; lazy[Right] += lazy[node];}
55         lazy[node] = 0;
56     }
57 }
58
59 void updateRange(int node, int lo, int hi, int i, int j, int val){
60     lazyPropagation(node,lo,hi);
61     if(lo>hi) return;
62     else if(lo>j || hi<i) return;
63     if(lo>=i && hi<=j) {lazy[node] += val; lazyPropagation(node,lo,hi); return;}
64     updateRange(Left,lo,mid,i,j,val);
65     updateRange(Right,mid+1,hi,i,j,val);
66     tree[node] = tree[Left] + tree[Right];
67 }
68
69 int queryRange(int node,int lo,int hi,int i,int j){
70     if(lo>hi) return 0;
71     else if(lo>j || hi<i) return 0;
72     lazyPropagation(node,lo,hi);
73     if(lo>=i && hi <= j) return tree[node];
74     int p1 = queryRange(Left,lo,mid,i,j);
75     int p2 = queryRange(Right,mid+1,hi,i,j);
76     return p1 + p2;
77 }

```

3.19 Segment Tree (Sparse)

```

1 #define MAX 1000000005
2 #define mid (lo+(hi-lo)/2) //Be careful about overflow
3
4 struct item{
5     int value, lazy;
6     item *Left, *Right ;
7     item(){value=0; lazy=0; Left=Right=NULL;}
8 };
9 item* root;
10
11 int query(item* node, int lo, int hi, int i, int j){
12     if (!node || i>hi || j<lo) return 0;
13     if (lo>=i && hi<=j) return node->value;
14     int p1=query(node->Left,lo,mid,i,j);
15     int p2=query(node->Right,mid+1,hi,i,j);
16     return p1+p2;
17 }
18
19 void update(item* node, int lo, int hi, int i, int val){
20     if (i>hi || i<lo) return;
21     if (lo==hi) {node->value=val; return;}
22     if(lo!=hi && !node->Left) node->Left=new item();
23     if(lo!=hi && !node->Right) node->Right=new item();

```

```

24     update(node->Left, lo, mid, i, val);
25     update(node->Right, mid+1, hi, i, val);
26     node->value = node->Left->value + node->Right->value;
27 }
28
29 //lazy propagation
30 void lazyPropagation(item* node, int lo, int hi){
31     if(node->lazy){
32         node->value += (hi-lo+1)*node->lazy;
33         if(lo != hi) {node->Left->lazy += node->lazy; node->Right->lazy += node->lazy;
34         ;}
35         node->lazy = 0;
36     }
37 }
38
39 void updateRange(item* node, int lo, int hi, int i, int j, ll val){
40     if(!node->Left) node->Left = new item();
41     if(!node->Right) node->Right = new item();
42     lazyPropagation(node, lo, hi);
43
44     if(lo > hi) return;
45     else if(lo > j || hi < i) return;
46     if(lo >= i && hi <= j) {node->lazy += val; lazyPropagation(node, lo, hi); return;}
47
48     updateRange(node->Left, lo, mid, i, j, val);
49     updateRange(node->Right, mid+1, hi, i, j, val);
50     node->value = node->Left->value + node->Right->value;
51 }
52
53 ll queryRange(item* node, int lo, int hi, int i, int j){
54     if(lo > hi) return 0;
55     else if(lo > j || hi < i) return 0;
56
57     if(!node->Left) node->Left = new item();
58     if(!node->Right) node->Right = new item();
59     lazyPropagation(node, lo, hi);
60
61     if(lo >= i && hi <= j) return node->value;
62     ll p1 = queryRange(node->Left, lo, mid, i, j);
63     ll p2 = queryRange(node->Right, mid+1, hi, i, j);
64     return p1 + p2;
65 }
66
67 //Free each node of the tree after each test case
68 void Free(item* node){
69     if(node->Left) Free(node->Left);
70     if(node->Right) Free(node->Right);
71     free(node);
72 }
73
74 int main() {root = new item();}
75

```

3.20 Wavelet Tree

```

1 //Tested : https://www.codechef.com/AUG19A/problems/CHGORAM
2
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct WaveletTree{
7     int lo, hi;
8     WaveletTree *l, *r;
9     vector<int> b;
10
11     //from and to are array pointers
12     //Build Wavelet Tree on A[from...to];
13     //x <= A[j] <= y, for each from <= j <= to
14     //Warning : Changes the original array A
15     WaveletTree(int *from, int *to, int x, int y){
16         lo = x, hi = y;
17         if(lo == hi || from >= to) return;
18         int mid = (lo+hi)/2;
19         auto f = [mid](int x) {return x <= mid;};
20
21         b.reserve(to - from + 1);
22         b.push_back(0);
23
24         for(auto it = from; it != to; it++)
25             b.push_back(b.back() + f(*it));
26
27         auto pivot = stable_partition(from, to, f);
28         l = new WaveletTree(from, pivot, lo, mid);
29         r = new WaveletTree(pivot, to, mid+1, hi);
30     }
31
32     //kth (1-indexed) smallest element in [l, r]
33     int kth(int l, int r, int k){
34         if(l > r) return 0;
35         if(lo == hi) return lo;
36         int inLeft = b[r] - b[l-1];
37         int lb = b[l-1]; //amt of nos in first (l-1) nos that go in left
38         int rb = b[r]; //amt of nos in first (r) nos that go in left
39         if(k <= inLeft) return this->l->kth(lb+1, rb, k);
40         return this->r->kth(l-lb, r-rb, k-inLeft);
41     }
42
43     //Count of numbers <= k in [l, r]
44     int LTE(int l, int r, int k){
45         if(l > r || k < lo) return 0;
46         if(hi <= k) return r - l + 1;
47         int lb = b[l-1], rb = b[r];
48         return this->l->LTE(lb+1, rb, k) + this->r->LTE(l-lb, r-rb, k);
49     }
50 };
51

```

```

52 const int MAXN = 100005;
53 const int MAXA = 1000005;
54 int A[MAXN];
55
56 int main(){
57     int n;
58     scanf("%d",&n);
59     for(int i=1; i<=n; i++) scanf("%d",&A[i]);
60     WaveletTree wt(A+1, A+n+1, 1, MAXA);
61 }

```

3.21 Sliding RMQ

```

1 //O(n) RMQ(Max) for all k-length subarrays of a n-length array using deque
2 deque<int>dq;
3 int A[MAX];
4 int n,k;
5
6 void solve(){
7     for(int i=1;i<k;i++){
8         while(!dq.empty() && A[dq.back()]<=A[i]) dq.pop_back();
9         dq.push_back(i);
10    }
11    for(int i=k;i<=n;i++){
12        while(!dq.empty() && A[dq.back()]<=A[i]) dq.pop_back();
13        dq.push_back(i);
14        while(!dq.empty() && dq.front()<=i-k) dq.pop_front();
15        printf("%d ",A[dq.front()]);
16    }
17 }

```

3.22 SparseTable (1D)

```

1 //1D SparseTable
2 int n;
3 int A[MAX];
4 int Log[MAX];
5 int M[MAX][MAXlogN];
6
7 void buildSparse(){
8     for(int i=1;i<=n;i++) M[i][0]=A[i];
9     for(int i=2;i<=n;i++) Log[i]=Log[i-1] + !(i&(i-1));
10
11     for(int j=1; (1<<j)<=n ;j++){
12         for(int i=1; (i+ (1<<j)-1) <=n; i++)
13             M[i][j]=max(M[i][j-1],M[i + (1<<(j-1))][j - 1]);
14     }
15 }
16
17 int Query(int i,int j){

```

```

18     int k=Log[j-i+1];
19     return max(M[i][k],M[j- (1<<k)+1][k]);
20 }

```

3.23 SparseTable (Rectangle Query)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 505;
5 const int LOGN = 9;
6
7 //O(n^2 (logn)^2)
8 //Supports Rectangular Query
9 int A[MAXN][MAXN];
10 int M[MAXN][MAXN][LOGN][LOGN];
11
12 void Build2DSparse(int N){
13     for(int i = 1; i <= N; i++){
14         for(int j = 1; j <= N; j++){
15             M[i][j][0][0] = A[i][j];
16         }
17         for(int q = 1; (1<<q) <= N; q++){
18             int add = 1<<(q-1);
19             for(int j=1; j+add <= N; j++){
20                 M[i][j][0][q] = max(M[i][j][0][q-1], M[i][j+add][0][q-1]);
21             }
22         }
23     }
24
25     for(int p=1; (1<<p)<=N; p++){
26         int add = 1<<(p-1);
27         for(int i=1; i+add <= N; i++){
28             for(int q=0; (1<<q) <= N; q++){
29                 for(int j=1; j<= N; j++){
30                     M[i][j][p][q] = max(M[i][j][p-1][q], M[i+add][j][p-1][q]);
31                 }
32             }
33         }
34     }
35 }
36
37 //returns max of all A[i][j], where x1<=i<=x2 and y1<=j<=y2
38 int Query(int x1,int y1,int x2,int y2){
39     int kX = log2(x2-x1+1); int kY = log2(y2-y1+1);
40     int addX = 1<<kX; int addY = 1<<kY;
41
42     int ret1 = max(M[x1][y1][kX][kY], M[x1][y2-addY+1][kX][kY]);
43     int ret2 = max(M[x2-addX+1][y1][kX][kY], M[x2-addX+1][y2-addY+1][kX][kY]);
44     return max(ret1, ret2);
45 }

```

3.24 SparseTable (Square Query)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 1005;
5 const int LOGN = 10;
6 int A[MAXN][MAXN];
7 int M[MAXN][MAXN][LOGN];
8
9 //O(n^2 logn)
10 //Supports Square Query
11 void Build2DSparse(int n){
12     for(int k=0; (1<<k) <= n; k++){
13         for(int i=1; i+(1<<k)-1 <= n; i++){
14             for(int j=1; j+(1<<k)-1 <= n; j++){
15                 if(k==0) M[i][j][k] = A[i][j];
16                 else{
17                     int add = 1<<(k-1);
18                     int ret1 = max(M[i][j][k-1], M[i+add][j][k-1]);
19                     int ret2 = max(M[i][j+add][k-1], M[i+add][j+add][k-1]);
20                     M[i][j][k] = max(ret1, ret2);
21                 }
22             }
23         }
24     }
25 }
26
27 //returns max of all A[i][j], where p<=i<=p+s-1 and q<=j<=q+s-1
28 int Query(int p, int q, int s){
29     int k = log2(s);
30     int add = 1<<k;
31     int ret1 = max(M[p][q][k], M[p+s-add][q][k]);
32     int ret2 = max(M[p][q+s-add][k], M[p+s-add][q+s-add][k]);
33     return max(ret1, ret2);
34 }

```

3.25 Treap (Implicit)

```

1 typedef long long ll;
2
3 struct node {
4     int prior, size;
5     ll val, sum, lazy, rev;
6     node *l, *r;
7     node(int v = 0) {
8         val = sum = v;
9         lazy = 0; rev = 0;
10        prior = rand();
11        size = 1;
12        l = r = NULL;

```

```

13    }
14 } *root;
15 ///If you initialize the root (root = new node()) then treap will be 1 based
16   indexing and root will be the 0th index
17 ///Otherwise it will be 0 based indexing, so decide carefully
18 typedef node* pnode;
19
20 int sz(pnode t) { return t ? t->size : 0; }
21 void upd_sz(pnode t) { if(t) t->size = sz(t->l) + 1 + sz(t->r); }
22
23 void push(pnode t) {
24     if(!t) return;
25
26     if(t->lazy){
27         t->val += t->lazy;
28         t->sum += t->lazy * sz(t);
29         if(t->l) t->l->lazy += t->lazy;
30         if(t->r) t->r->lazy += t->lazy;
31         t->lazy = 0;
32     }
33     //Reverse
34     if(t->rev){
35         swap(t->l, t->r);
36         if(t->l) t->l->rev ^= t->rev;
37         if(t->r) t->r->rev ^= t->rev;
38         t->rev = 0;
39     }
40 }
41
42 void combine(pnode t) {
43     // Note: This function should reset datas of t first (MUST!!)
44     // Then update datas from l and r.
45     if(!t) return;
46     push(t->l);
47     push(t->r);
48     t->sum = t->val; // Reset
49     if(t->l) t->sum += t->l->sum;
50     if(t->r) t->sum += t->r->sum;
51 }
52
53 void split(pnode t, pnode &l, pnode &r, int pos, int add = 0) {
54     if(!t) return void(l = r = NULL);
55     push(t);
56     int curr = sz(t->l) + add;
57     if(curr <= pos)
58         split(t->r, t->r, r, pos, curr + 1), l = t;
59     else
60         split(t->l, l, t->l, pos, add), r = t;
61     upd_sz(t);
62     combine(t);
63 }
64
65 void merge(pnode &t, pnode l, pnode r) {
66     push(l), push(r);

```

```

65     if(!l || !r)
66         t = l ? l : r;
67     else if(l -> prior > r -> prior)
68         merge(l -> r, l -> r, r), t = l;
69     else merge(r -> l, l, r -> l), t = r;
70     upd_sz(t);
71     combine(t);
72 }
73
74 ll query(pnode t, int l, int r) {
75     pnode L, mid, R;
76     split(t, L, mid, l - 1);
77     split(mid, t, R, r - 1);
78     ll ans = t -> sum;
79     merge(mid, L, t);
80     merge(t, mid, R);
81     return ans;
82 }
83
84 void update(pnode t, int l, int r, ll v) {
85     pnode L, mid, R;
86     split(t, L, mid, l - 1);
87     split(mid, t, R, r - 1);
88     t -> lazy += v;
89     merge(mid, L, t);
90     merge(t, mid, R);
91 }
92
93 void insert(pnode &t, int pos, int v) {
94     pnode L, R, tmp, y = new node(v);
95     split(t, L, R, pos - 1);
96     merge(tmp, L, y);
97     merge(t, tmp, R);
98 }
99
100 void Del(pnode &t, int pos) {
101     pnode L, R, mid;
102     split(t, L, mid, pos - 1);
103     split(mid, t, R, 0);
104     pnode tmp = t;
105     merge(t, L, R);
106     free(tmp);
107 }
108
109 void Reverse(pnode t, int l, int r) {
110     pnode L, mid, R;
111     split(t, L, mid, l - 1);
112     split(mid, t, R, r - 1);
113     t -> rev ^= 1;
114     merge(mid, L, t);
115     merge(t, mid, R);
116 }

```

3.26 Treap (Persistent)

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 #define MAX 1000005
5 typedef long long ll;
6
7 struct node{
8     int prior, size;
9     ll val, sum;
10    node *l, *r;
11    node(int v = 0) {
12        val = sum = v;
13        prior = rand();
14        size = 1;
15        l = r = NULL;
16    }
17 };
18
19 node* root[MAX];
20 typedef node* pnode;
21
22 int sz(pnode t) {return t ? t -> size : 0;}
23 void upd_sz(pnode t) {if(t) t -> size = sz(t -> l) + 1 + sz(t -> r);}
24
25 void combine(pnode t) {
26     // Note: This function should reset datas of t first (MUST!!)
27     // Then update datas from l and r.
28     if(!t) return;
29     t -> sum = t -> val; // Reset
30     if(t -> l) t -> sum += t -> l -> sum;
31     if(t -> r) t -> sum += t -> r -> sum;
32 }
33
34 void split(pnode t, pnode &l, pnode &r, int pos, int add = 0) {
35     if(!t) return void(l = r = NULL);
36     node* cn = new node();
37     *cn = *t;
38
39     int curr = sz(cn -> l) + add;
40     if(curr <= pos) split(cn -> r, cn -> r, r, pos, curr + 1), l = cn;
41     else split(cn -> l, l, cn -> l, pos, add), r = cn;
42
43     upd_sz(cn);
44     combine(cn);
45 }
46
47 void merge(pnode &t, pnode l, pnode r) {
48     if(!l || !r) {t = l ? l : r; return;}
49     node* cn = new node();
50
51     if(l -> prior > r -> prior) {*cn = *l; merge(cn -> r, cn -> r, r);}

```

```

52     else {*cn = *r; merge(cn -> l, l, cn -> l);}
53     t = cn;
54
55     upd_sz(t);
56     combine(t);
57 }
58
59 //Use pnode &t for non-persistent operation (will update original t)
60 //Use pnode t for persistent operation (will not update the original one)
61 //No need to merge back to retrieve the original node after split in persistent
   operation
62
63 void insert(pnode &t, int pos, int v){
64     pnode L, R, tmp, y = new node(v);
65     split(t, L, R, pos - 1);
66     merge(tmp, L, y);
67     merge(t, tmp, R);
68 }
69
70 void Replace(pnode &t, int x, int val){
71     pnode L, R, mid;
72     split(t, L, mid, x);
73     split(L, t, R, x-1);
74     R->val = R->sum = val;
75     merge(L, t, R);
76     merge(t, L, mid);
77 }
78
79 ll Query(pnode t, int pos1, int pos2){
80     pnode L, R, mid, tmp;
81     split(t, L, R, pos2);
82     split(L, mid, tmp, pos1 - 1);
83     return tmp->sum;
84 }
85
86 int main(){
87     int n;
88     scanf("%d", &n);
89     root[0] = new node();
90
91     int version = 0;
92     for(int i = 1; i <= n; i++){ //set A[version] = A[ID] and insert(pos, v)
93         int tp; scanf("%d", &tp);
94         if(tp == 1){
95             int ID, pos, v;
96             scanf("%d %d %d", &ID, &pos, &v);
97             root[++version] = root[ID];
98             insert(root[version], pos, v);
99         }
100         else if(tp == 2){ //set A[ID][pos] = v;
101             int ID, pos, v;
102             scanf("%d %d %d", &ID, &pos, &v);
103             Replace(root[ID], pos, v);

```

```

104     }
105     else if(tp == 3){ //print A[version]
106         int ID;
107         scanf("%d", &ID);
108         for(int i = 1; i < root[ID]->size; i++) printf("%d ", Query(root[ID], i, i));
109         printf("\n");
110     }
111 }
112 }

```

3.27 Trie (Basic)

```

1 //Space Complexity : O(Sum of |S_i|)
2
3 const int LOG = 20;
4 struct Trie{
5     struct Node{
6         Node* Child[2];
7         Node() {Child[0] = Child[1] = NULL;}
8     };
9     Node* root;
10
11     void Init() {root = new Node();}
12     bool Check(int x, int b) {return (x >> b) & 1;}
13
14     void Insert(int n){
15         Node *Cur = root;
16         for(int i = LOG-1; i >= 0; i--){
17             bool ID = Check(n, i);
18             if(!Cur->Child[ID]) Cur->Child[ID] = new Node();
19             Cur = Cur->Child[ID];
20         }
21     }
22
23     int MinQuery(int XOR){
24         int Ans = 0;
25         Node *Cur = root;
26         for(int i = LOG-1; i >= 0; i--){
27             bool ID = Check(XOR, i);
28             if(Cur->Child[ID]) Cur = Cur->Child[ID];
29             else Cur = Cur->Child[!ID], Ans += (1 << i);
30         }
31         return Ans;
32     }
33 };

```

3.28 Trie (Persistent)

```

1 // Tested : Codechef GPD - Gotham PD
2

```

```

3 const int LOG = 31;
4 struct PersistentTrie{
5     struct Node{
6         Node* Child[2];
7         Node() {Child[0]=Child[1]=NULL;}
8     };
9     Node* root;
10
11     void Init() {root = new Node();}
12     bool Check(int x,int b) {return (x>>b)&1;}
13
14     Node* Insert(int n){
15         Node *Old = root;
16         Node *Cur = new Node();
17         Node* newRoot = Cur;
18
19         for(int i = LOG-1; i>=0; i--){
20             bool ID = Check(n,i);
21             if(Old && Old->Child[!ID]) Cur->Child[!ID] = Old->Child[!ID];
22             Cur->Child[ID] = new Node();
23
24             Cur = Cur->Child[ID];
25             if(Old) Old = Old->Child[ID];
26         }
27         return newRoot;
28     }
29
30     int MinQuery(int XOR){
31         int Ans = 0;
32         Node *Cur = root;
33         for(int i = LOG-1; i>=0; i--){
34             bool ID = Check(XOR,i);
35
36             if(Cur->Child[ID]) Cur = Cur->Child[ID];
37             else Cur = Cur->Child[!ID], Ans += (1<<i);
38         }
39         return Ans;
40     }
41 };

```

4 Game Theory

4.1 Green Hackenbush

```

1 // Green Hackenbush
2 // Description:
3 //   Consider a two player game on a graph with a specified vertex (root).
4 //   In each turn, a player eliminates one edge.
5 //   Then, if a subgraph that is disconnected from the root, it is removed.
6 //   If a player cannot select an edge (i.e., the graph is singleton),
7 //   he will lose.

```

```

8 //
9 //   Compute the Grundy number of the given graph.
10 //
11 // Algorithm:
12 //   We use two principles:
13 //   1. Colon Principle: Grundy number of a tree is the xor of
14 //      Grundy number of child subtrees.
15 //      (Proof: easy).
16 //
17 //   2. Fusion Principle: Consider a pair of adjacent vertices u, v
18 //      that has another path (i.e., they are in a cycle). Then,
19 //      we can contract u and v without changing Grundy number.
20 //      (Proof: difficult)
21 //
22 //   We first decompose graph into two-edge connected components.
23 //   Then, by contracting each components by using Fusion Principle,
24 //   we obtain a tree (and many self loops) that has the same Grundy
25 //   number to the original graph. By using Colon Principle, we can
26 //   compute the Grundy number.
27 //
28 // Complexity:
29 //   O(m + n).
30 //
31 // Verified:
32 //   SPOJ 1477: Play with a Tree
33 //   IPSC 2003 G: Got Root?
34
35 #define fst first
36 #define snd second
37 #define all(c) ((c).begin()), ((c).end())
38 struct hackenbush {
39     int n;
40     vector<vector<int>> adj;
41     hackenbush(int n) : n(n), adj(n) { }
42     void add_edge(int u, int v) {
43         adj[u].push_back(v);
44         if (u != v) adj[v].push_back(u);
45     }
46     // r is the only root connecting to the ground
47     int grundy(int r) {
48         vector<int> num(n), low(n);
49         int t = 0;
50         function<int(int, int)> dfs = [&](int p, int u) {
51             num[u] = low[u] = ++t;
52             int ans = 0;
53             for (int v : adj[u]) {
54                 if (v == p) { p += 2 * n; continue; }
55                 if (num[v] == 0) {
56                     int res = dfs(u, v);
57                     low[u] = min(low[u], low[v]);
58                     if (low[v] > num[u]) ans ^= (1 + res) ^ 1; // bridge
59                     else ans ^= res; // non bridge
60                 } else low[u] = min(low[u], num[v]);

```

```

61     }
62     if (p > n) p -= 2 * n;
63     for (int v : adj[u])
64         if (v != p && num[u] <= num[v]) ans ^= 1;
65     return ans;
66 };
67 return dfs(-1, r);
68 }
69 };
70
71 int main() {
72     int cases; scanf("%d", &cases);
73     for (int icase = 0; icase < cases; ++icase) {
74         int n; scanf("%d", &n);
75         vector<int> ground(n);
76         int r;
77         for (int i = 0; i < n; ++i) {
78             scanf("%d", &ground[i]);
79             if (ground[i] == 1) r = i;
80         }
81         int ans = 0;
82         hackenbush g(n);
83         for (int i = 0; i < n - 1; ++i) {
84             int u, v;
85             scanf("%d %d", &u, &v);
86             --u; --v;
87             if (ground[u]) u = r;
88             if (ground[v]) v = r;
89             if (u == v) ans ^= 1;
90             else g.add_edge(u, v);
91         }
92         int res = ans ^ g.grundy(r);
93         printf("%d\n", res != 0);
94     }
95 }

```

4.2 Red-Blue Hackenbush

1 //Resources : <http://www.geometer.org/mathcircles/hackenbush.pdf>

2

3 Lets say you are given a rooted tree and edges are coloured red and blue.

4 Red player will cut red edges and blue player will cut blue edges.

5 When a edge is cut, then the subtree under it is cut. Who will win?

6

7 It can be solved by red-blue hacken bush.

8

9 If the tree is a chain, then it's simple.

10 Let cur = 1, and from root there are consecutive x blue/red edges.

11 Then we add cur x times to our grundy val if the first edges are blue,

12 otherwise minus cur x times if the first edges are red. And for every next edges

13 we make cur = cur / 2 and add it to our grundy value depending on the edge colour

14 .

```

14
15 For a tree follow this pseudo-code:
16
17 dfs(u):
18     if(u == LEAF_NODE) return 0
19     else:
20         double grundy = 0
21
22         for(all child v of u)
23             double x = dfs(v)
24             if(edge(u,v) is blue):
25                 y = smallest integer > 0 so that (x + y) > 1
26                 x = x + y
27                 y = 2 ^ (y - 1)
28                 grundy = grundy + (x / y) //Double Divison
29             else:
30                 y = smallest integer > 0 so that (x - y) < -1
31                 x = x - y
32                 y = 2 ^ (y - 1)
33                 grundy = grundy + (x / y) //Double Divison
34         return grundy
35
36 If the grundy is positive, then blue wins
37 if it's negative red wins
38 If it's 0, then the player who first moves lose.

```

5 Geometry

5.1 2D Geo

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double pi = 4 * atan(1);
5 const double eps = 1e-6;
6
7 inline int dcmp (double x) { if (fabs(x) < eps) return 0; else return x < 0 ? -1
8 : 1; }
9
10 double fix_acute(double th) {return th<-pi ? (th+2*pi): th>pi ? (th-2*pi) : th;}
11
12 inline double getDistance (double x, double y) { return sqrt(x * x + y * y); }
13 inline double torad(double deg) { return deg / 180 * pi; }
14
15 struct Point {
16     double x, y;
17     Point (double x = 0, double y = 0): x(x), y(y) {}
18     void read () { scanf("%lf%lf", &x, &y); }
19     void write () { printf("%lf %lf", x, y); }
20
21     bool operator == (const Point& u) const { return dcmp(x - u.x) == 0 && dcmp(y
22 - u.y) == 0; }

```

```

20 bool operator != (const Point& u) const { return !(*this == u); }
21 bool operator < (const Point& u) const { return dcmp(x - u.x) < 0 || (dcmp(x -
22 u.x)==0 && dcmp(y-u.y) < 0); }
23 bool operator > (const Point& u) const { return u < *this; }
24 bool operator <= (const Point& u) const { return *this < u || *this == u; }
25 bool operator >= (const Point& u) const { return *this > u || *this == u; }
26 Point operator + (const Point& u) { return Point(x + u.x, y + u.y); }
27 Point operator - (const Point& u) { return Point(x - u.x, y - u.y); }
28 Point operator * (const double u) { return Point(x * u, y * u); }
29 Point operator / (const double u) { return Point(x / u, y / u); }
30 double operator * (const Point& u) { return x*u.y - y*u.x; }
31 };
32 typedef Point Vector;
33 typedef vector<Point> Polygon;
34 struct Line {
35     double a, b, c;
36     Line (double a = 0, double b = 0, double c = 0): a(a), b(b), c(c) {}
37 };
38 struct Segment{
39     Point a;
40     Point b;
41     Segment(){}
42     Segment(Point aa,Point bb) {a=aa,b=bb;}
43 };
44 struct DirLine {
45     Point p;
46     Vector v;
47     double ang;
48     DirLine () {}
49     DirLine (Point p, Vector v): p(p), v(v) { ang = atan2(v.y, v.x); }
50     bool operator < (const DirLine& u) const { return ang < u.ang; }
51 };
52 namespace Punctual {
53     double getDistance (Point a, Point b) { double x=a.x-b.x, y=a.y-b.y; return
54     sqrt(x*x + y*y); }
55 };
56 namespace Vectorial {
57     double getDot (Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
58     double getCross (Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
59     double getLength (Vector a) { return sqrt(getDot(a, a)); }
60     double getPLength (Vector a) { return getDot(a, a); }
61     double getAngle (Vector u) { return atan2(u.y, u.x); }
62     double getSignedAngle (Vector a, Vector b) {return getAngle(b)-getAngle(a);}
63     Vector rotate (Vector a, double rad) { return Vector(a.x*cos(rad)-a.y*sin(rad)
64     ), a.x*sin(rad)+a.y*cos(rad)); }
65     Vector ccw(Vector a, double co, double si) {return Vector(a.x*co-a.y*si, a.y*
66     co+a.x*si);}
67     Vector cw (Vector a, double co, double si) {return Vector(a.x*co+a.y*si, a.y*
68     co-a.x*si);}
69     Vector scale(Vector a, double s = 1.0) {return a / getLength(a) * s;}
70     Vector getNormal (Vector a) { double l = getLength(a); return Vector(-a.y/l,
71     a.x/l); }
72 };
73 namespace ComplexVector {
74     typedef complex<double> Point;
75     typedef Point Vector;
76     double getDot(Vector a, Vector b) { return real(conj(a)*b); }
77     double getCross(Vector a, Vector b) { return imag(conj(a)*b); }
78     Vector rotate(Vector a, double rad) { return a*exp(Point(0, rad)); }
79 };
80 namespace Linear {
81     using namespace Vectorial;
82     Line getLine (double x1, double y1, double x2, double y2) { return Line(y2-y1
83     , x1-x2, y1*x2-x1*y2); }
84     Line getLine (double a, double b, Point u) { return Line(a, -b, u.y * b - u.x
85     * a); }
86     bool getIntersection (Line p, Line q, Point& o) {
87         if (fabs(p.a * q.b - q.a * p.b) < eps)
88             return false;
89         o.x = (q.c * p.b - p.c * q.b) / (p.a * q.b - q.a * p.b);
90         o.y = (q.c * p.a - p.c * q.a) / (p.b * q.a - q.b * p.a);
91         return true;
92     }
93     bool getIntersection (Point p, Vector v, Point q, Vector w, Point& o) {
94         if (dcmp(getCross(v, w)) == 0) return false;
95         Vector u = p - q;
96         double k = getCross(w, u) / getCross(v, w);
97         o = p + v * k;
98         return true;
99     }
100     double getDistanceToLine (Point p, Point a, Point b) { return fabs(getCross(b
101     -a, p-a) / getLength(b-a)); }
102     double getDistanceToSegment (Point p, Point a, Point b) {
103         if (a == b) return getLength(p-a);
104         Vector v1 = b - a, v2 = p - a, v3 = p - b;
105         if (dcmp(getDot(v1, v2)) < 0) return getLength(v2);
106         else if (dcmp(getDot(v1, v3)) > 0) return getLength(v3);
107         else return fabs(getCross(v1, v2) / getLength(v1));
108     }
109     double getDistanceSegToSeg (Point a,Point b,Point c,Point d){
110         double Ans=INT_MAX;
111         Ans=min(Ans,getDistanceToSegment(a,c,d));
112         Ans=min(Ans,getDistanceToSegment(b,c,d));
113     }
114 }

```

```

117     Ans=min(Ans,getDistanceToSegment(c,a,b));
118     Ans=min(Ans,getDistanceToSegment(d,a,b));
119     return Ans;
120 }
121
122 Point getPointToLine (Point p, Point a, Point b) { Vector v = b-a; return a+v
123 * (getDot(v, p-a) / getDot(v,v)); }
124
125 bool onSegment (Point p, Point a, Point b) { return dcmp(getCross(a-p, b-p))
126 == 0 && dcmp(getDot(a-p, b-p)) <= 0; }
127
128 bool haveIntersection (Point a1, Point a2, Point b1, Point b2) {
129     if(onSegment(a1,b1,b2)) return true;
130     if(onSegment(a2,b1,b2)) return true;
131     if(onSegment(b1,a1,a2)) return true;
132     if(onSegment(b2,a1,a2)) return true; //Case of touch
133
134     double c1=getCross(a2-a1, b1-a1), c2=getCross(a2-a1, b2-a1), c3=getCross
135     b2-b1, a1-b1), c4=getCross(b2-b1,a2-b1);
136     return dcmp(c1)*dcmp(c2) < 0 && dcmp(c3)*dcmp(c4) < 0;
137 }
138
139 bool onLeft (DirLine l, Point p) { return dcmp(l.v * (p-l.p)) >= 0; }
140 }
141
142 namespace Triangular {
143     using namespace Vectorial;
144
145     double getAngle (double a, double b, double c) { return acos((a*a+b*b-c*c) /
146     (2*a*b)); }
147     double getArea (double a, double b, double c) { double s =(a+b+c)/2; return
148     sqrt(s*(s-a)*(s-b)*(s-c)); }
149     double getArea (double a, double h) { return a * h / 2; }
150     double getArea (Point a, Point b, Point c) { return fabs(getCross(b - a, c -
151     a)) / 2; }
152     double getDirArea (Point a, Point b, Point c) { return getCross(b - a, c - a)
153     / 2;}
154
155     //ma/mb/mc = length of median from side a/b/c
156     double getArea_(double ma,double mb,double mc) {double s=(ma+mb+mc)/2; return
157     4/3.0 * sqrt(s*(s-ma)*(s-mb)*(s-mc));}
158
159     //ha/hb/hc = length of perpendicular from side a/b/c
160     double get_Area (double ha,double hb,double hc){
161         double H=(1/ha+1/hb+1/hc)/2; double _A_ = 4 * sqrt(H * (H-1/ha)*(H-1/hb)
162         *(H-1/hc)); return 1.0/_A_;
163     }
164
165     bool pointInTriangle(Point a, Point b, Point c, Point p){
166         double s1 = getArea(a,b,c);
167         double s2 = getArea(p,b,c) + getArea(p,a,b) + getArea(p,c,a);
168         return dcmp(s1 - s2) == 0;
169     }
170 };
171
172 namespace Polygonal {
173     using namespace Vectorial;
174     using namespace Linear;
175     using namespace Triangular;
176
177     double getSignedArea (Point* p, int n) {
178         double ret = 0;
179         for (int i = 0; i < n-1; i++)
180             ret += (p[i]-p[0]) * (p[i+1]-p[0]);
181         return ret/2;
182     }
183
184     int getConvexHull (Point* p, int n, Point* ch) {
185         sort(p, p + n);
186
187         // preparing lower hull
188         int m = 0;
189         for (int i = 0; i < n; i++){
190             while (m > 1 && dcmp(getCross(ch[m-1]-ch[m-2], p[i]-ch[m-1])) <= 0) m
191             --;
192             ch[m++] = p[i];
193         }
194
195         // preparing upper hull
196         int k = m;
197         for (int i = n-2; i >= 0; i--){
198             while (m > k && dcmp(getCross(ch[m-1]-ch[m-2], p[i]-ch[m-2])) <= 0) m
199             --;
200             ch[m++] = p[i];
201         }
202         if (n > 1) m--;
203         return m;
204     }
205
206     int isPointInPolygon (Point o, Point* p, int n) {
207         int wn = 0;
208         for (int i = 0; i < n; i++) {
209             int j = (i + 1) % n;
210             if (onSegment(o, p[i], p[j]) || o == p[i]) return 0;
211             int k = dcmp(getCross(p[j] - p[i], o-p[i]));
212             int d1 = dcmp(p[i].y - o.y);
213             int d2 = dcmp(p[j].y - o.y);
214             if (k > 0 && d1 <= 0 && d2 > 0) wn++;
215             if (k < 0 && d2 <= 0 && d1 > 0) wn--;
216         }
217         return wn ? -1 : 1;
218     }
219
220     void rotatingCalipers(Point *p, int n, vector<Segment>& sol) {
221         sol.clear();
222         int j = 1; p[n] = p[0];
223         for (int i = 0; i < n; i++) {
224             while (getCross(p[j+1]-p[i+1], p[i]-p[i+1]) > getCross(p[j]-p[i+1], p

```

```

[i]-p[i+1]))
    j = (j+1) % n;
    sol.push_back(Segment(p[i],p[j]));
    sol.push_back(Segment(p[i + 1],p[j + 1]));
}
}

void rotatingCalipersGetRectangle (Point *p, int n, double& area, double&
perimeter) {
    p[n] = p[0];
    int l = 1, r = 1, j = 1;
    area = perimeter = 1e20;

    for (int i = 0; i < n; i++) {
        Vector v = (p[i+1]-p[i]) / getLength(p[i+1]-p[i]);
        while (dcmp(getDot(v, p[r%n]-p[i]) - getDot(v, p[(r+1)%n]-p[i])) < 0)
            r++;
        while (j < r || dcmp(getCross(v, p[j%n]-p[i]) - getCross(v, p[(j+1)%n]-p[i])) < 0) j++;
        while (l < j || dcmp(getDot(v, p[l%n]-p[i]) - getDot(v, p[(l+1)%n]-p[i])) > 0) l++;
        double w = getDot(v, p[r%n]-p[i]) - getDot(v, p[l%n]-p[i]);
        double h = getDistanceToLine (p[j%n], p[i], p[i+1]);
        area = min(area, w * h);
        perimeter = min(perimeter, 2 * w + 2 * h);
    }
}

Polygon cutPolygon (Polygon u, Point a, Point b) {
    Polygon ret;
    int n = u.size();
    for (int i = 0; i < n; i++) {
        Point c = u[i], d = u[(i+1)%n];
        if (dcmp((b-a)*(c-a)) >= 0) ret.push_back(c);
        if (dcmp((b-a)*(d-c)) != 0) {
            Point t;
            getIntersection(a, b-a, c, d-c, t);
            if (onSegment(t, c, d))
                ret.push_back(t);
        }
    }
    return ret;
}

int halfPlaneIntersection(DirLine* li, int n, Point* poly) {
    sort(li, li + n);

    int first, last;
    Point* p = new Point[n];
    DirLine* q = new DirLine[n];
    q[first=last=0] = li[0];

    for (int i = 1; i < n; i++) {
        while (first < last && !onLeft(li[i], p[last-1])) last--;
        while (first < last && !onLeft(li[i], p[first])) first++;
        q[++last] = li[i];

        if (dcmp(q[last].v * q[last-1].v) == 0) {
            last--;
            if (onLeft(q[last], li[i].p)) q[last] = li[i];
        }

        if (first < last)
            getIntersection(q[last-1].p, q[last-1].v, q[last].p, q[last].v, p
[last-1]);
    }

    while (first < last && !onLeft(q[first], p[last-1])) last--;
    if (last - first <= 1) { delete [] p; delete [] q; return 0; }
    getIntersection(q[last].p, q[last].v, q[first].p, q[first].v, p[last]);

    int m = 0;
    for (int i = first; i <= last; i++) poly[m++] = p[i];
    delete [] p; delete [] q;
    return m;
}

Polygon simplify (const Polygon& poly) {
    Polygon ret;
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        Point a = poly[i];
        Point b = poly[(i+1)%n];
        Point c = poly[(i+2)%n];
        if (dcmp((b-a)*(c-b)) != 0 && (ret.size() == 0 || b != ret[ret.size()
-1]))
            ret.push_back(b);
    }
    return ret;
}

Point ComputeCentroid( Point* p, int n){
    Point c(0,0);
    double scale = 6.0 * getSignedArea(p,n);
    for (int i = 0; i < n; i++){
        int j = (i+1) % n;
        c = c + (p[i]+p[j])*(p[i].x*p[j].y - p[j].x*p[i].y);
    }
    return c / scale;
}

// Tested : https://www.spoj.com/problems/INOROUT
// pt must be in ccw order with no three collinear points
// returns inside = 1, on = 0, outside = -1
int pointInConvexPolygon(Point* pt, int n, Point p){
    assert(n >= 3);

```

```

311     int lo = 1 , hi = n - 1 ;
312     while(hi - lo > 1){
313         int mid = (lo + hi) / 2;
314         if(getCross(pt[mid] - pt[0], p - pt[0]) > 0) lo = mid;
315         else hi = mid;
316     }
317
318     bool in = pointInTriangle(pt[0], pt[lo], pt[hi], p);
319     if(!in) return -1;
320
321
322     if(getCross(pt[lo] - pt[lo-1], p - pt[lo-1]) == 0) return 0;
323     if(getCross(pt[hi] - pt[lo], p - pt[lo]) == 0) return 0;
324     if(getCross(pt[hi] - pt[(hi+1)%n], p - pt[(hi+1)%n]) == 0) return 0;
325
326
327     return 1;
328 }
329
330 // Tested : https://toph.co/p/cover-the-points
331 // Calculate [ACW, CW] tangent pair from an external point
332 #define CW    -1
333 #define ACW    1
334 int direction(Point st, Point ed, Point q) {return dcmp(getCross(ed -
335 st, q - ed));}
336 bool isGood(Point u, Point v, Point Q, int dir) {return direction(Q, u, v)
337 != -dir;}
338 Point better(Point u, Point v, Point Q, int dir) {return direction(Q, u, v)
339 == dir ? u : v;}
340
341 Point tangents(Point* pt, Point Q, int dir, int lo, int hi){
342     while(hi - lo > 1){
343         int mid = (lo + hi)/2;
344         bool pvs = isGood(pt[mid], pt[mid - 1], Q, dir);
345         bool nxt = isGood(pt[mid], pt[mid + 1], Q, dir);
346
347         if(pvs && nxt) return pt[mid];
348         if(!(pvs || nxt)){
349             Point p1 = tangents(pt, Q, dir, mid+1, hi);
350             Point p2 = tangents(pt, Q, dir, lo, mid - 1);
351             return better(p1, p2, Q, dir);
352         }
353         if(!pvs){
354             if(direction(Q, pt[mid], pt[lo]) == dir) hi = mid - 1;
355             else if(better(pt[lo], pt[hi], Q, dir) == pt[lo]) hi = mid - 1;
356             else lo = mid + 1;
357         }
358         if(!nxt){
359             if(direction(Q, pt[mid], pt[lo]) == dir) lo = mid + 1;
360             else if(better(pt[lo], pt[hi], Q, dir) == pt[lo]) hi = mid - 1;
361             else lo = mid + 1;
362         }
363     }

```

```

361     }
362
363     Point ret = pt[lo];
364     for(int i = lo + 1; i <= hi; i++) ret = better(ret, pt[i], Q, dir);
365     return ret;
366 }
367
368 // [ACW, CW] Tangent
369 pair<Point, Point> get_tangents(Point* pt, int n, Point Q){
370     Point acw_tan = tangents(pt, Q, ACW, 0, n - 1);
371     Point cw_tan = tangents(pt, Q, CW, 0, n - 1);
372     return make_pair(acw_tan, cw_tan);
373 }
374 };
375
376 struct Circle {
377     Point o;
378     double r;
379     Circle () {}
380     Circle (Point o, double r = 0): o(o), r(r) {}
381     void read () { o.read(), scanf("%lf", &r); }
382     Point point(double rad) { return Point(o.x + cos(rad)*r, o.y + sin(rad)*r); }
383     double getArea (double rad) { return rad * r * r / 2; }
384     //area of the circular sector cut by a chord with central angle alpha
385     double sector(double alpha) {return r * r * 0.5 * (alpha - sin(alpha));}
386 };
387
388 namespace Circular {
389     using namespace Linear;
390     using namespace Vectorial;
391     using namespace Triangular;
392
393     int getLineCircleIntersection (Point p, Point q, Circle O, double& t1, double
394 & t2, vector<Point>& sol) {
395         Vector v = q - p;
396         //sol.clear();
397         double a = v.x, b = p.x - O.o.x, c = v.y, d = p.y - O.o.y;
398         double e = a*a+c*c, f = 2*(a*b+c*d), g = b*b+d*d-O.r*O.r;
399         double delta = f*f - 4*e*g;
400         if (dcmp(delta) < 0) return 0;
401         if (dcmp(delta) == 0) {
402             t1 = t2 = -f / (2 * e);
403             sol.push_back(p + v * t1);
404             return 1;
405         }
406         t1 = (-f - sqrt(delta)) / (2 * e); sol.push_back(p + v * t1);
407         t2 = (-f + sqrt(delta)) / (2 * e); sol.push_back(p + v * t2);
408         return 2;
409     }
410
411     // signed area of intersection of circle(c.o, c.r) and
412     // triangle(c.o, s.a, s.b) [cross(a-o, b-o)/2]

```

```

413 double areaCircleTriIntersection(Circle c, Segment s){
414     using namespace Linear;
415     double OA = getLength(c.o - s.a);
416     double OB = getLength(c.o - s.b);
417
418     // sector
419     if (dcmp(getDistanceToSegment(c.o, s.a, s.b) - c.r) >= 0)
420         return fix_acute(getSignedAngle(s.a - c.o, s.b - c.o)) * (c.r*c.r) /
2.0;
421
422     // triangle
423     if (dcmp(OA - c.r) <= 0 && dcmp(OB - c.r) <= 0)
424         return getCross(c.o-s.b,s.a-s.b) / 2.0;
425
426     // three part: (A, a) (a, b) (b, B)
427     vector<Point>Sect; double t1,t2;
428     getLineCircleIntersection(s.a, s.b, c, t1, t2, Sect);
429     return areaCircleTriIntersection(c, Segment(s.a, Sect[0]))
430         + areaCircleTriIntersection(c, Segment(Sect[0], Sect[1]))
431         + areaCircleTriIntersection(c, Segment(Sect[1], s.b));
432 }
433
434 // area of intersection of circle(c.o, c.r) and simple polyson(p[])
435 // Tested : ZOJ 2675 - Little Mammoth
436 double areaCirclePolygon(Circle c, Polygon p){
437     double res = .0;
438     int n = p.size();
439     for (int i = 0; i < n; ++ i)
440         res += areaCircleTriIntersection(c, Segment(p[i], p[(i+1)%n]));
441     return fabs(res);
442 }
443
444 // interior          (d < R - r)          ----> -2
445 // interior tangents (d = R - r)          ----> -1
446 // concentric        (d = 0)
447 // secants           (R - r < d < R + r) ----> 0
448 // exterior tangents (d = R + r)          ----> 1
449 // exterior          (d > R + r)          ----> 2
450 int getPos(Circle o1, Circle o2) {
451     using namespace Vectorial;
452     double d = getLength(o1.o - o2.o);
453     int in = dcmp(d - fabs(o1.r - o2.r)), ex = dcmp(d - (o1.r + o2.r));
454     return in<0 ? -2 : in==0? -1 : ex==0 ? 1 : ex>0? 2 : 0;
455 }
456
457 int getCircleCircleIntersection (Circle o1, Circle o2, vector<Point>& sol) {
458     double d = getLength(o1.o - o2.o);
459     if (dcmp(d) == 0) {
460         if (dcmp(o1.r - o2.r) == 0) return -1;
461         return 0;
462     }
463     if (dcmp(o1.r + o2.r - d) < 0) return 0;
464     if (dcmp(fabs(o1.r-o2.r) - d) > 0) return 0;

```

```

465     Vector v = o2.o - o1.o;
466     double co = (o1.r*o1.r + getPLength(v) - o2.r*o2.r) / (2 * o1.r *
467     getLength(v));
468     double si = sqrt(fabs(1.0 - co*co));
469     Point p1 = scale(cw(v,co, si), o1.r) + o1.o;
470     Point p2 = scale(ccw(v,co, si), o1.r) + o1.o;
471
472     sol.push_back(p1);
473     if (p1 == p2) return 1;
474     sol.push_back(p2);
475     return 2;
476 }
477
478 double areaCircleCircle(Circle o1, Circle o2){
479     Vector AB = o2.o - o1.o;
480     double d = getLength(AB);
481     if(d >= o1.r + o2.r) return 0;
482     if(d + o1.r <= o2.r) return pi * o1.r * o1.r;
483     if(d + o2.r <= o1.r) return pi * o2.r * o2.r;
484
485     double alpha1 = acos((o1.r * o1.r + d * d - o2.r * o2.r) / (2.0 * o1.r *
d));
486     double alpha2 = acos((o2.r * o2.r + d * d - o1.r * o1.r) / (2.0 * o2.r *
d));
487     return o1.sector(2*alpha1) + o2.sector(2*alpha2);
488 }
489
490 int getTangents (Point p, Circle o, Vector* v) {
491     Vector u = o.o - p;
492     double d = getLength(u);
493     if (d < o.r) return 0;
494     else if (dcmp(d - o.r) == 0) {
495         v[0] = rotate(u, pi / 2);
496         return 1;
497     } else {
498         double ang = asin(o.r / d);
499         v[0] = rotate(u, -ang);
500         v[1] = rotate(u, ang);
501         return 2;
502     }
503 }
504
505 int getTangentPoints (Point p, Circle o, vector<Point>& v) {
506     Vector u = p - o.o ;
507     double d = getLength(u);
508     if (d < o.r) return 0;
509     else if (dcmp(d - o.r) == 0) {
510         v.push_back(o.o+u);
511         return 1;
512     } else {
513         double ang = acos(o.r / d);
514         u = u / getLength(u) * o.r;
515         v.push_back(o.o+rotate(u, -ang));

```

```

515         v.push_back(o.o+rotate(u, ang));
516         return 2;
517     }
518 }
519
520 int getTangents (Circle o1, Circle o2, Point* a, Point* b) {
521     int cnt = 0;
522     if (dcmp(o1.r-o2.r) < 0) { swap(o1, o2); swap(a, b); }
523     double d2 = getPLength(o1.o - o2.o);
524     double rdif = o1.r - o2.r, rsum = o1.r + o2.r;
525     if (dcmp(d2 - rdif * rdif) < 0) return 0;
526     if (dcmp(d2) == 0 && dcmp(o1.r - o2.r) == 0) return -1;
527
528     double base = getAngle(o2.o - o1.o);
529     if (dcmp(d2 - rdif * rdif) == 0) {
530         a[cnt] = o1.point(base); b[cnt] = o2.point(base); cnt++;
531         return cnt;
532     }
533
534     double ang = acos( (o1.r - o2.r) / sqrt(d2) );
535     a[cnt] = o1.point(base+ang); b[cnt] = o2.point(base+ang); cnt++;
536     a[cnt] = o1.point(base-ang); b[cnt] = o2.point(base-ang); cnt++;
537
538     if (dcmp(d2 - rsum * rsum) == 0) {
539         a[cnt] = o1.point(base); b[cnt] = o2.point(pi+base); cnt++;
540     }
541     else if (dcmp(d2 - rsum * rsum) > 0) {
542         double ang = acos( (o1.r + o2.r) / sqrt(d2) );
543         a[cnt] = o1.point(base+ang); b[cnt] = o2.point(pi+base+ang); cnt++;
544         a[cnt] = o1.point(base-ang); b[cnt] = o2.point(pi+base-ang); cnt++;
545     }
546     return cnt;
547 }
548
549 Circle CircumscribedCircle(Point p1, Point p2, Point p3) {
550     double Bx = p2.x - p1.x, By = p2.y - p1.y;
551     double Cx = p3.x - p1.x, Cy = p3.y - p1.y;
552     double D = 2 * (Bx * Cy - By * Cx);
553     double cx = (Cy * (Bx * Bx + By * By) - By * (Cx * Cx + Cy * Cy)) / D +
554     p1.x;
555     double cy = (Bx * (Cx * Cx + Cy * Cy) - Cx * (Bx * Bx + By * By)) / D +
556     p1.y;
557     Point p = Point(cx, cy);
558     return Circle(p, getLength(p1 - p));
559 }
560
561 Circle InscribedCircle(Point p1, Point p2, Point p3) {
562     double a = getLength(p2 - p3);
563     double b = getLength(p3 - p1);
564     double c = getLength(p1 - p2);
565     Point p = (p1 * a + p2 * b + p3 * c) / (a + b + c);
566     return Circle(p, getDistanceToLine(p, p1, p2));
567 }

```

```

566
567 //distance From P : distance from Q = rp : rq
568 Circle getApolloniusCircle(const Point& P, const Point& Q, double rp, double
569 rq) {
570     rq *= rq;
571     rp *= rp;
572     double a = rq - rp;
573     assert(dcmp(a));
574     double g = rq * P.x - rp * Q.x; g /= a;
575     double h = rq * P.y - rp * Q.y; h /= a;
576     double c = rq*P.x*P.x-rp*Q.x*Q.x+rq*P.y*P.y-rp*Q.y*Q.y;
577     c /= a;
578     Point o(g,h);
579     double R = g*g +h*h - c;
580     R = sqrt(R);
581     return Circle(o,R);
582 }
583
584 struct Star{
585     int n; // number of side of the star
586     double r; // radius of the circum-circle
587     Star(int n,double r) {this->n=n; this->r=r;}
588
589     double getArea(){
590         double theta=pi/n;
591         double s=2*r*sin(theta);
592         double R=0.5*s/tan(theta);
593         double a=0.5*n*s*R;
594         double a2=0.25*s*s/tan(1.5*theta);
595         return a-n*a2;
596     }
597 };

```

5.2 3D Geo

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 const double pi = 4 * atan(1);
5 const double eps = 1e-10;
6 inline int dcmp (double x) { if (fabs(x) < eps) return 0; else return x < 0 ? -1
7 : 1; }
8
9 inline double torad(double deg) { return deg / 180 * pi; }
10
11 struct Point{
12     double x, y;
13     Point (double x = 0, double y = 0): x(x), y(y) {}
14
15     Point operator + (const Point& u) { return Point(x + u.x, y + u.y); }
16     Point operator - (const Point& u) { return Point(x - u.x, y - u.y); }
17     Point operator * (const double u) { return Point(x * u, y * u); }

```

```

16 Point operator / (const double u) { return Point(x / u, y / u); }
17 double operator * (const Point& u) { return x*u.y - y*u.x; }
18 };
19
20 struct Point3D{
21     double x, y, z;
22     Point3D() {}
23     void read () {cin>>x>>y>>z;}
24     void write () {cout<<x<<" --- "<<y<<" --- "<<z<<"\n";}
25
26     Point3D(double x, double y, double z) : x(x), y(y), z(z) {}
27     Point3D(const Point3D &p) : x(p.x), y(p.y), z(p.z) {}
28     Point3D operator +(Point3D b) {return Point3D(x+b.x,y+b.y, z+b.z);}
29     Point3D operator -(Point3D b) {return Point3D(x-b.x,y-b.y, z-b.z);}
30     Point3D operator *(double b) {return Point3D(x*b,y*b, z*b);}
31     Point3D operator /(double b) {return Point3D(x/b,y/b, z/b);}
32     bool operator <(Point3D b) {return make_pair(make_pair(x,y),z) < make_pair(
33         make_pair(b.x,b.y),b.z);}
34     bool operator ==(Point3D b) {return dcmp(x-b.x)==0 && dcmp(y-b.y) == 0 &&
35         dcmp(z-b.z) == 0;}
36 };
37 typedef Point3D Vector3D;
38 typedef vector<Point> Polygon;
39 typedef vector<Point3D> Polyhedron;
40
41 namespace Vectorial{
42     double getDot (Vector3D a, Vector3D b) {return a.x*b.x+a.y*b.y+a.z*b.z;}
43     Vector3D getCross(Vector3D a, Vector3D b) {return Point3D(a.y*b.z-a.z*b.y, a.
44         z*b.x-a.x*b.z, a.x*b.y-a.y*b.x);}
45     double getLength (Vector3D a) {return sqrt(getDot(a, a)); }
46     double getPLength (Vector3D a) {return getDot(a, a); }
47     Vector3D unitVector(Vector3D v) {return v/getLength(v);}
48
49     double getUnsignedAngle(Vector3D u,Vector3D v){
50         double cosTheta = getDot(u,v)/getLength(u)/getLength(v);
51         cosTheta = max(-1.0,min(1.0,cosTheta));
52         return acos(cosTheta);
53     }
54
55     Vector3D rotate(Vector3D v,Vector3D a,double rad){
56         a = unitVector(a);
57         return v * cos(rad) + a * (1 - cos(rad)) * getDot(a,v) + getCross(a,v) *
58             sin(rad);
59     }
60 }
61
62 struct Line3D{
63     Vector3D v; Point3D o;
64     Line3D() {}
65     Line3D(Vector3D v,Point3D o):v(v),o(o){}
66     Point3D getPoint(double t) {return o + v*t;}
67 };
68
69 namespace Linear{
70     using namespace Vectorial;
71     double getDistSq(Line3D l, Point3D p) {return getPLength(getCross(l.
72         v,p-l.o))/getPLength(l.v);}
73     double getDistLinePoint (Line3D l, Point3D p) {return sqrt(getDistSq(l,p));}
74     bool cmp(Line3D l,Point3D p, Point3D q) {return getDot(l.v,p) <
75         getDot(l.v,q);}
76     Point3D projection(Line3D l,Point3D p) {return l.o + l.v * getDot(l.
77         .v,p-l.o)/getPLength(l.v);}
78     Point3D reflection(Line3D l,Point3D p) {return projection(l,p)+
79         projection(l,p)-p;}
80
81     double getAngle(Line3D l,Line3D m) {return getUnsignedAngle(l.v,m.
82         .v);}
83     bool isParallel(Line3D p,Line3D q) {return dcmp(getPLength(
84         getCross(p.v,q.v))) == 0;}
85     bool isPerpendicular(Line3D p,Line3D q) {return dcmp(getDot(p.v,q.v))
86         == 0;}
87
88     double getDist (Line3D l, Line3D m){
89         Vector3D n = getCross(l.v, m.v);
90         if(getPLength(n) == 0) return getDistLinePoint(l,m.o);
91         else return fabs(getDot(m.o-l.o , n)) / getLength(n);
92     }
93
94     Point3D getClosestPointOnLine1(Line3D l,Line3D m){
95         Vector3D n = getCross(l.v, m.v);
96         Vector3D n2 = getCross(m.v, n);
97         return l.o + l.v * getDot(m.o-l.o, n2) / getDot(l.v, n2);
98     }
99 }
100
101 struct Plane{
102     Vector3D n; //normal n
103     double d; //getDot(n,p) = d for any point p on the plane
104     Plane() {}
105     Plane(Vector3D n, double d) : n(n), d(d) {}
106     Plane(Vector3D n, Point3D p) : n(n), d(Vectorial :: getDot(n,p)) {}
107     Plane(const Plane &p) : n(p.n), d(p.d) {}
108 };
109
110 namespace Planar{
111     using namespace Vectorial;
112     Plane getPlane(Point3D a,Point3D b,Point3D c) {return Plane(getCross(b-a,c-a)
113         ,a);}
114     Plane translate(Plane p,Vector3D t) {return Plane(p.n, p.d+getDot(p.n,t)
115         );}
116     Plane shiftUp(Plane p,double dist) {return Plane(p.n, p.d+dist*getLength
117         (p.n));}
118     Plane shiftDown(Plane p,double dist) {return Plane(p.n, p.d-dist*getLength
119         (p.n));}
120
121     double getSide(Plane p,Point3D a) {return getDot(p.n,a)-p.d;}

```

```

107 double getDistance(Plane p,Point3D a) {return fabs(getSide(p,a))/getLength(p.151
n);} 152
108 Point3D projection(Plane p,Point3D a) {return a-p.n*getSide(p,a)/getPLength53
(p.n);} 154
109 Point3D reflection(Plane p,Point3D a) {return a-p.n*getSide(p,a)/getPLength55
(p.n)*2;} 156
110 157
111 bool intersect(Plane p, Line3D l, Point3D& a){ 158
112 if(dcmp(getDot(p.n,l.v)) == 0) return false; 159
113 a = l.o - l.v * getSide(p,l.o) / getDot(p.n,l.v); 160
114 return true; 161
115 } 162
116 163
117 bool intersect(Plane p,Plane q,Line3D& l){ 164
118 l.v = getCross(p.n,q.n); 165
119 if(dcmp(getPLength(l.v)) == 0) return false; 166
120 l.o = getCross(q.n*p.d - p.n*q.d , l.v) / getPLength(l.v); 167
121 return true; 168
122 } 169
123 170
124 double getAngle(Plane p,Plane q) {return getUnsignedAngle(p.n,q.n);} 171
125 bool isParallel(Plane p,Plane q) {return dcmp(getPLength(getCross(p.n,q.172
n))) == 0;} 173
126 bool isPerpendicular(Plane p,Plane q) {return dcmp(getDot(p.n,q.n)) == 0;} 174
127 175
128 bool getAngle(Plane p,Line3D l) {return pi/2.0 - getUnsignedAngle(p.n176
,l.v);} 177
129 bool isParallel(Plane p,Line3D l) {return dcmp(getDot(p.n,l.v)) == 0;} 178
130 bool isPerpendicular(Plane p,Line3D l) {return dcmp(getPLength(getCross(p.n,179
l.v))) == 0;} 180
131 181
132 Line3D perpThrough(Plane p,Point3D a) {return Line3D(p.n,a);} 182
133 Plane perpThrough(Line3D l,Point3D a) {return Plane(l.v,a);} 183
134 184
135 //Modify p.n if necessary with respect to the reference point 185
136 Vector3D rotateCCW90(Plane p,Vector3D d) {return getCross(p.n,d);} 186
137 Vector3D rotateCW90(Plane p,Vector3D d) {return getCross(d,p.n);} 187
138 188
139 pair<Point3D, Point3D> TwoPointsOnPlane(Plane p){ 189
140 Vector3D N = p.n; double D = p.d; 190
141 191
142 assert(dcmp(N.x) != 0 || dcmp(N.y) != 0 || dcmp(N.z) != 0); 192
143 if(dcmp(N.x) == 0 && dcmp(N.y) == 0) return {Point3D(1,0,D/N.z), Point3D 193
(0,1,D/N.z)}; 194
144 if(dcmp(N.y) == 0 && dcmp(N.z) == 0) return {Point3D(D/N.x,1,0), Point3D 195
(D/N.x,0,1)}; 196
145 if(dcmp(N.z) == 0 && dcmp(N.x) == 0) return {Point3D(1,D/N.y,0), Point3D 197
(0,D/N.y,1)}; 198
146 if(dcmp(N.x) == 0) return {Point3D(1,D/N.y,0), Point3D(0,0,D/N.z)}; 199
147 if(dcmp(N.y) == 0) return {Point3D(0,1,D/N.z), Point3D(D/N.x,0,0)}; 200
148 if(dcmp(N.z) == 0) return {Point3D(D/N.x,0,1), Point3D(0,D/N.y,0)}; 201
149 if(dcmp(D)!=0) return {Point3D(D/N.x,0,0), Point3D(0,D/N.y,0)}; 202
150 return {Point3D(N.y,-N.x,0), Point3D(-N.y,N.x,0)};

```

```

}
Point From3Dto2D(Plane p, Point3D a){
    assert( dcmp(getSide(p,a)) == 0 );
    auto Pair = TwoPointsOnPlane(p);
    Point3D A = Pair.first;
    Point3D B = Pair.second;

    Vector3D Z = p.n;                Z = Z / getLength(Z);
    Vector3D X = B - A;                X = X / getLength(X);
    Vector3D Y = getCross(Z,X);

    Vector3D v = a - A;
    assert( dcmp(getDot(v,Z)) == 0);
    return Point(getDot(v,X),getDot(v,Y));
}

Point3D From2Dto3D(Plane p, Point a){
    auto Pair = TwoPointsOnPlane(p);
    Point3D A = Pair.first;
    Point3D B = Pair.second;

    Vector3D Z = p.n;                Z = Z / getLength(Z);
    Vector3D X = B - A;                X = X / getLength(X);
    Vector3D Y = getCross(Z,X);

    return A + X * a.x + Y * a.y;
}

struct Sphere{
    Point3D c;
    double r;
    Sphere() {}
    Sphere(Point3D c, double r) : c(c), r(r) {}

    //Spherical cap with polar angle theta
    double Height(double alpha) {return r*(1-cos(alpha));}
    double BaseRadius(double alpha) {return r*sin(alpha);}
    double Volume(double alpha) {double h = Height(alpha); return pi*h*h*(3*
r-h)/3.0;}
    double SurfaceArea(double alpha) {double h = Height(alpha); return 2*pi*r*h;}
};

namespace Spherical{
    using namespace Vectorial;
    using namespace Planar;
    using namespace Linear;

    Sphere CircumscribedSphere(Point3D a,Point3D b,Point3D c,Point3D d){
        assert( dcmp(getSide(getPlane(a,b,c), d)) != 0);

        Plane U = Plane(a-b, (a+b)/2);

```

```

203 Plane V = Plane(b-c, (b+c)/2);
204 Plane W = Plane(c-d, (c+d)/2);
205
206 Line3D l1,l2;
207 bool ret1 = intersect(U,V,l1);
208 bool ret2 = intersect(V,W,l2);
209 assert(ret1 == true && ret2 == true);
210 assert( dcmp(getDist(l1,l2)) == 0);
211
212 Point3D C = getClosestPointOnLine1(l1,l2);
213 return Sphere(C, getLength(C-a));
214 }
215
216 pair<double,double> SphereSphereIntersection(Sphere s1,Sphere s2){
217     double d = getLength(s1.c-s2.c);
218     if(dcmp(d - s1.r -s2.r) >= 0) return {0,0};
219
220     double R1 = max(s1.r,s2.r); double R2 = min(s1.r,s2.r);
221     double y = R1 + R2 - d;
222     double x = (R1*R1 - R2*R2 + d*d) / (2*d);
223     double h1 = R1 - x;
224     double h2 = y - h1;
225
226     double Volume      = pi*h1*h1*(3*R1-h1)/3.0 + pi*h2*h2*(3*R2-h2)/3.0;
227     double SurfaceArea = 2*pi*R1*h1 + 2*pi*R2*h2;
228     return make_pair(SurfaceArea,Volume);
229 }
230
231 Point3D getPointOnSurface(double r,double Lat,double Lon){
232     Lat = torad(Lat); //North-South
233     Lon = torad(Lon); //East-West
234     return Point3D(r*cos(Lat)*cos(Lon), r*cos(Lat)*sin(Lon), r*sin(Lat));
235 }
236
237 int intersect(Sphere s,Line3D l, vector<Point3D>& ret){
238     double h2 = s.r*s.r - getDistSq(l,s.c);
239     if(dcmp(h2)<0) return 0;
240
241     Point3D p = projection(l,s.c);
242     if(dcmp(h2) == 0) {ret.push_back(p); return 1;}
243
244     Vector3D h = l.v * sqrt(h2) / getLength(l.v);
245     ret.push_back(p-h); ret.push_back(p+h); return 2;
246 }
247
248 double GreatCircleDistance(Sphere s,Point3D a,Point3D b){
249     return s.r * getUnsignedAngle(a-s.c, b-s.c);
250 }
251 }
252
253 namespace Poly{
254     using namespace Vectorial;
255     Sphere SmallestEnclosingSphere(Polyhedron p){

```

```

256     int n = p.size();
257     Point3D C(0,0,0);
258     for(int i=0; i<n; i++) C = C + p[i];
259     C = C / n;
260
261     double P = 0.1;
262     int pos = 0;
263     int Accuracy = 70000;
264     for (int i = 0; i < Accuracy; i++) {
265         pos = 0;
266         for (int j = 1; j < n; j++){
267             if(getPLength(C - p[j]) > getPLength(C - p[pos])) pos = j;
268         }
269         C = C + (p[pos] - C)*P;
270         P *= 0.998;
271     }
272     return Sphere(C, getPLength(C - p[pos]));
273 }
274 }
275
276 struct Pyramid{
277     int n; //number of side of the pyramid
278     double l; //length of each side
279     double ang;
280
281     Pyramid(int n, double l) {this->n=n; this->l=l; ang=pi/n;}
282     double getBaseArea() {return l * l * n / (4* tan(ang));}
283     double getHeight() {return l * sqrt(1 - 1 / (4*sin(ang)*sin(ang))) ;}
284     double getVolume() {return getBaseArea() * getHeight() / 3;}
285 };

```

5.3 Area of Union Of Circles

```

1 // Problem ID : UVa 12056
2 struct Range {
3     double t; // -pi <= t <= pi
4     int evt;
5     Point p;
6     Range() {}
7     Range(double t, int evt, Point p) : t(t), evt(evt), p(p) {}
8
9     bool operator <(const Range &s) const {
10         return dcmp(t - s.t) < 0 || (dcmp(t - s.t) == 0 && evt > s.evt);
11     }
12 };
13
14 const int MAX = 1005;
15 Circle C[MAX];
16 vector<Range>R;
17
18 bool cmp_r(const Circle &a, const Circle &b) {return a.r > b.r;}
19 double AreaUnionCircle(Circle C[], int &n){

```

```

20 using namespace Vectorial;
21 using namespace Circular;
22
23 sort(C, C + n, cmp_r);
24 int k = 0;
25 for (int i = 0; i < n; i++){
26     if (dcmp(C[i].r) == 0) break;
27     int j = 0;
28     for (j = 0; j < k; j++) //interior or concentric
29         if (getPos(C[i], C[j]) < 0 || !dcmp(getLength(C[i].o - C[j].o))) break;
30     if (j == k) C[k++] = C[i];
31 }
32 n = k;
33
34 double ans = 0;
35 for (int i = 0; i < n; ++ i){
36     int nc = 0;
37     R.clear();
38     Point mpi = Point(- C[i].r, 0) + C[i].o;
39     R.push_back(Range(-pi, 1, mpi));
40     R.push_back(Range(pi, -1, mpi));
41
42     for (int j = 0; j < n; ++ j){
43         if (j == i) continue;
44
45         vector<Point>sol;
46         int ret = getCircleCircleIntersection(C[i], C[j], sol);
47         if (ret < 2) continue;
48
49         Point a = sol[0]; Point b = sol[1];
50         double jR = getAngle(a - C[i].o), jL = getAngle(b - C[i].o);
51         if (dcmp(jR - jL) > 0) ++ nc;
52         R.push_back(Range(jR, 1, a));
53         R.push_back(Range(jL, -1, b));
54     }
55     sort(R.begin(), R.end());
56
57     double pj = - pi;
58     Point pp = mpi;
59     for (Range rng : R){
60         nc += rng.evt;
61         if ((nc == 2 && rng.evt > 0) || nc == 0)
62             ans += C[i].sector(rng.t - pj) + getCross(pp, rng.p) / 2;
63         pj = rng.t; pp = rng.p;
64     }
65 }
66 return ans;
67 }

```

5.4 Integer Tricks

```

1 // 1. Checks the relation between a/b and c/d, a, c >= 0, b, d > 0

```

```

2 bool comp (ll a, ll b, ll c, ll d){
3     if (a / b != c / d) return a / b < c / d;
4     a %= b, c %= d;
5     if (c == 0) return 0;
6     if (a == 0) return 1;
7     return comp(d, c, b, a);
8 }
9
10 // 2. Sorting points based on polar angle
11 bool sgn (Point p) {return p.y < 0 || (p.y == 0 && p.x < 0);}
12
13 bool operator < (Point p, Point q) const{
14     if (sgn(p) != sgn(q)) return sgn(p) < sgn(q);
15     return getCross(p, q) > 0;
16 }
17
18 // 3. Check whether Angle(a1, b1) < Angle(a2, b2)
19 bool angleLess (Vector a1, Vector b1, Vector a2, Vector b2) {
20     pt p1 ( getDot(a1, b1), abs(getCross(a1, b1)) );
21     pt p2 ( getDot(a2, b2), abs(getCross(a2, b2)) );
22     return getCross(p1, p2) > 0;
23 }

```

5.5 IntegerGeo

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define ll long long int
4 inline ll dcmp(ll x) {return x;}
5
6 struct Fraction{
7     ll m,n;
8     Fraction(){}
9     Fraction(ll x,ll y) {ll g=__gcd(x,y); m=x/g; n=y/g; if(n<0) {m=-m,n=-n;} }
10     Fraction operator + (const Fraction& u) const {return Fraction(m*u.n+u.m*n,n*
11     u.n); }
12     Fraction operator - (const Fraction& u) const {return Fraction(m*u.n-u.m*n,n*
13     u.n); }
14     Fraction operator * (const Fraction& u) const {return Fraction(m*u.m,n*u.n);
15     }
16     Fraction operator / (const Fraction& u) const {return Fraction(m*u.n,n*u.m);
17     }
18 };
19
20 struct Point{
21     ll x, y;
22     Point (ll x = 0, ll y = 0): x(x), y(y) {}
23
24     bool operator == (const Point& u) const { return dcmp(x - u.x) == 0 && dcmp(y
25     - u.y) == 0; }
26     bool operator != (const Point& u) const { return !(*this == u); }

```

```

22 bool operator < (const Point& u) const { return dcmp(x - u.x) < 0 || (dcmp(x-
u.x)==0 && dcmp(y-u.y) < 0); }
23 bool operator > (const Point& u) const { return u < *this; }
24 bool operator <= (const Point& u) const { return *this < u || *this == u; }
25 bool operator >= (const Point& u) const { return *this > u || *this == u; }
26 Point operator + (const Point& u) { return Point(x + u.x, y + u.y); }
27 Point operator - (const Point& u) { return Point(x - u.x, y - u.y); }
28 Point operator * (const ll u) { return Point(x * u, y * u); }
29 ll operator * (const Point& u) { return x*u.y - y*u.x; }
30 };
31 typedef Point Vector;
32
33 namespace Vectorial {
34     ll getDot (Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
35     ll getCross (Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
36     ll getPLength (Vector a) { return getDot(a, a); }
37 };
38
39 struct Line {
40     ll a,b,c;
41     Line(){ }
42     Line(ll x=0,ll y=0,ll z=0){
43         ll G=__gcd(x,y); //will equal gcd(x,y,z)
44         a=x/G; b=y/G; c=z/G;
45         if(a<0) {a=-a; b=-b; c=-c;} //a=0 case auto-handled
46     }
47 };
48
49 namespace Linear {
50     using namespace Vectorial;
51
52     bool getIntersection (Point p, Vector v, Point q, Vector w, Point& o) {
53         if (dcmp(getCross(v, w)) == 0) return false; //parallel
54
55         Vector u = p - q;
56         ll cross1 = getCross(w, u); ll cross2 = getCross(v, w);
57         ll g = __gcd(v.x,v.y); v.x/=g; v.y/=g; cross1 *= g;
58         if(cross1 % cross2) return false; //intersection point non-integer
59
60         ll k = cross1 / cross2;
61         o = p + v * k;
62         return true;
63     }
64
65     ll sign(ll x) {return x == 0 ? x : x/abs(x);}
66     bool haveIntersection (Point a1, Point a2, Point b1, Point b2) {
67         ll c1=getCross(a2-a1, b1-a1), c2=getCross(a2-a1, b2-a1), c3=getCross(b2-
b1, a1-b1), c4=getCross(b2-b1,a2-b1);
68         c1=sign(c1); c2=sign(c2); c3=sign(c3); c4=sign(c4);
69         return dcmp(c1)*dcmp(c2) <= 0 && dcmp(c3)*dcmp(c4) <= 0;
70     }
71 }

```

5.6 Radial Sweep

```

1 #define ll long long int
2
3 struct Point{
4     ll x; ll y; ll val;
5     bool operator < (const Point& other) const {
6         if(x * other.x > 0) return y*other.x < x*other.y;
7         else return y*other.x > x*other.y;
8     }
9 };
10 Point p[MAX];
11
12 int main(){
13     int n;
14     scanf("%d",&n);
15
16     ll Plus=0, Minus=0;
17     for(int i=1;i<=n;i++) cin>>p[i].x>>p[i].y>>p[i].val;
18
19     //Precalculate w.r.t. Y-axis
20     for(int i=1;i<=n;i++){
21         if(p[i].x>=0) Plus+=p[i].val;
22         else Minus+=p[i].val;
23     }
24     sort(p+1,p+n+1);
25
26     //Calculate w.r.t. all slopes
27     for(int i=1;i<=n;i++){
28         if(p[i].x>=0) Plus-=p[i].val, Minus+=p[i].val;
29         else Minus-=p[i].val, Plus+=p[i].val;
30     }
31 }

```

5.7 RotationMatrix

```

1 Translation Matrix :
2 | 1  0  x |
3 | 0  1  y |
4 | 0  0  1 |
5
6 Rotation Matrix :
7 | θcos  -θsin  1 |
8 | θsin   θcos  1 |
9 |  0      0    1 |
10
11 Rotate a Point Q by θ degrees counter-clockwise around the point P(x,y).
12 | θcos  -θsin  x(1-θcos) + θysin |
13 | θsin   θcos  y(1-θcos) - θxsin |
14 |  0      0      1              |

```

6 Graph

6.1 ArticulationBridge

```

1 #define MAX 100005
2 #define f first
3 #define s second
4 int cnt;
5 vector<vector<int>> > graph(MAX);
6 int d[MAX],md[MAX],vis[MAX];
7 map<pair<int,int>,int> bridges;
8 int all_bridges(int now,int from) {
9     d[now]=md[now]=cnt++;
10    vis[now]=1;
11    int i;
12    for(i=0; i<graph[now].size(); i++) {
13        if(graph[now][i]==from) continue;
14        if(vis[graph[now][i]]) {
15            md[now]=min(md[now],d[graph[now][i]]);
16            continue;
17        }
18        md[now]=min(md[now],all_bridges(graph[now][i],now));
19        if(md[graph[now][i]]>d[now])
20            bridges[{min(now,graph[now][i]),max(now,graph[now][i])}]=1;
21    }
22    return md[now];
23 }
```

6.2 ArticulationPoint

```

1 #define MAX 100005
2 #define f first
3 #define s second
4 int cnt, d[MAX],md[MAX],vis[MAX];
5 bool is_cutpoint[MAX];
6 vector<vector<int>> > graph(MAX);
7 int all_cutpoints(int now=0,int from=-1) {
8     d[now]=md[now]=cnt++;
9     vis[now]=1;
10    int i,total_child=0,is_cut=0;
11    for(i=0; i<graph[now].size(); i++) {
12        if(graph[now][i]==from) continue;
13        if(vis[graph[now][i]]) {
14            md[now]=min(md[now],d[graph[now][i]]);
15            continue;
16        }
17        total_child++;
18        md[now]=min(md[now],all_cutpoints(graph[now][i],now));
19        if(md[graph[now][i]]>=d[now]) is_cut=1;
20    }
21    if(from==-1 && total_child>=2)is_cutpoint[now]=true;
```

```

22     else if(from!=-1 && is_cut) is_cutpoint[now]=true;
23     else is_cutpoint[now]=false;
24     return md[now];
25 }
```

6.3 Block-CutVertexTree

```

1 struct graph {
2     int n;
3     vector<vector<int>> adj;
4     graph(int n) : n(n), adj(n) {}
5     void add_edge(int u, int v) {
6         adj[u].push_back(v);
7         adj[v].push_back(u);
8     }
9     int add_node() {
10        adj.push_back({});
11        return n++;
12    }
13    vector<int>& operator[](int u) {
14        return adj[u];
15    }
16 };
17
18 int node_id[500005];
19 int comp_id[500005];
20 int art[500005];
21 // tree = block cut tree
22 // return BCCs
23 vector<vector<int>> biconnected_components(graph &adj,graph&tree) {
24     int n = adj.n;
25     vector<int> num(n), low(n), stk;
26     vector<vector<int>> comps;
27     function<void(int, int, int&)> dfs = [&](int u, int p, int &t) {
28         num[u] = low[u] = ++t;
29         stk.push_back(u);
30         for (int v : adj[u]) if (v != p) {
31             if (!num[v]) {
32                 dfs(v, u, t);
33                 low[u] = min(low[u], low[v]);
34                 if (low[v] >= num[u]) {
35                     art[u] = (num[u] > 1 || num[v] > 2);
36                     comps.push_back({u});
37                     while (comps.back().back() != v)
38                         comps.back().push_back(stk.back()),stk.pop_back();
39                 }
40             } else low[u] = min(low[u], num[v]);
41         }
42     };
43     for (int u = 0, t; u < n; ++u)if (!num[u]) dfs(u, -1, t = 0);
44     for (int u = 0; u < n; ++u)if (art[u]) node_id[u] = tree.add_node();
45     int cmpi = 0;
```

```

46     for (auto &comp : comps) {
47         int node = tree.add_node();
48         comp_id[cmpi++] = node;
49         for (int u : comp)
50             if (!art[u]) node_id[u] = node;
51             else tree.add_edge(node, node_id[u]);
52     }
53     return comps;
54 }

```

6.4 LinkCutTree(rooted)

```

1 struct Node{
2     ll sz, label, w ;
3     Node *p, *pp, *l, *r;
4     Node() { p = pp = l = r = 0; }
5 };
6 void update(Node *x){
7     x->sz = x->w;
8     if(x->l) x->sz += x->l->sz;
9     if(x->r) x->sz += x->r->sz;
10 }
11 void rotr(Node *x){
12     Node *y, *z;
13     y = x->p, z = y->p;
14     if((y->l == x->r)) y->l->p = y;
15     x->r = y, y->p = x;
16     if((x->p == z)){
17         if(y == z->l) z->l = x;
18         else z->r = x;
19     }
20     x->pp = y->pp;
21     y->pp = 0;
22     update(y);
23 }
24
25 void rotl(Node *x){
26     Node *y, *z;
27     y = x->p, z = y->p;
28     if((y->r == x->l)) y->r->p = y;
29     x->l = y, y->p = x;
30     if((x->p == z)){
31         if(y == z->l) z->l = x;
32         else z->r = x;
33     }
34     x->pp = y->pp;
35     y->pp = 0;
36     update(y);
37 }
38
39 void splay(Node *x){
40     Node *y, *z;

```

```

41 while(x->p){
42     y = x->p;
43     if(y->p == 0){
44         if(x == y->l) rotr(x);
45         else rotl(x);
46     }
47     else{
48         z = y->p;
49         if(y == z->l){
50             if(x == y->l) rotr(y), rotr(x);
51             else rotl(x), rotr(x);
52         }
53         else
54             {
55                 if(x == y->r) rotr(y), rotr(x);
56                 else rotr(x), rotl(x);
57             }
58     }
59 }
60 update(x);
61 }
62
63 Node *access(Node *x){
64     splay(x);
65     if(x->r){
66         x->r->pp = x;
67         x->r->p = 0;
68         x->r = 0;
69         update(x);
70     }
71
72     Node *last = x;
73     while(x->pp) {
74         Node *y = x->pp;
75         last = y;
76         splay(y);
77         if(y->r){
78             y->r->pp = y;
79             y->r->p = 0;
80         }
81         y->r = x;
82         x->p = y;
83         x->pp = 0;
84         update(y);
85         splay(x);
86     }
87     return last;
88 }
89
90 Node *root(Node *x){
91     access(x);
92     while(x->l) x = x->l;
93     splay(x);

```

```

94     return x;
95 }
96
97 void cut(Node *x){
98     access(x);
99     x->l->p = 0;
100    x->l = 0;
101    update(x);
102 }
103
104 void link(Node *x, Node *y){
105     access(x);
106     access(y);
107     x->l = y;
108     y->p = x;
109     update(x);
110 }
111
112 Node *lca(Node *x, Node *y){
113     access(x);
114     return access(y);
115 }
116
117 ll depth(Node *x){
118     access(x);
119     return x->sz;
120 }
121
122 class LinkCut{
123     Node *x;
124
125 public:
126     void initLinkCut(ll n, ll *vat){
127         x = new Node[n+1];
128         for(ll i = 1; i <= n; i++){
129             x[i].label = i;
130             x[i].w = vat[i];
131             update(&x[i]);
132         }
133     }
134     virtual ~LinkCut(){ delete[] x; }
135     //parent[u]=v.
136     void link(ll u, ll v){ ::link(&x[u], &x[v]); }
137
138     void cut(ll u){ ::cut(&x[u]); }
139
140     ll root(ll u){ return ::root(&x[u])->label; }
141
142     ll depth(ll u){ return ::depth(&x[u]); }
143
144     ll lca(ll u, ll v){ return ::lca(&x[u], &x[v])->label; }
145 };

```

6.5 LinkCutTree

```

1 // can be used as online minimum spanning tree
2 // dynamic add edge & cut edge
3 // check the size of connected component
4 // check whether two node is connected
5 const int MAXN = 100005;
6 const int MAXM = 300005;
7
8 struct node {
9     node *l, *r, *p, *m; int f, v, w, s, id;
10    node(int v, int w, int id) : l(), r(), p(),
11    m(this), f(), v(v), w(w), s(w), id(id) {}
12 };
13
14 inline bool is_root(node *n){return n->p==NULL||n->p->l!=n&&n->p->r!=n;}
15 inline bool left(node *n){return n == n -> p -> l;}
16 inline int sum(node *n){return n ? n -> s : 0;}
17 inline node* max(node *n){return n ? n -> m : NULL;}
18 inline node* max(node *a, node *b) {
19     if(a == NULL || b == NULL) return a ? a : b;
20     return a -> v > b -> v ? a : b;
21 }
22 inline void push(node *n) {
23     if(!n -> f) return; n -> f = 0; swap(n -> l, n -> r);
24     if(n -> l) n -> l -> f ^= 1; if(n -> r) n -> r -> f ^= 1;
25 }
26 inline void pull(node *n) {
27     n -> m = max(max(max(n -> l), max(n -> r)), n);
28     n -> s = sum(n -> l) + sum(n -> r) + n -> w;
29 }
30 inline void connect(node *n, node *p, bool l) {
31     (l ? p -> l : p -> r) = n; if(n) n -> p = p;
32 }
33 inline void rotate(node *n) {
34     node *p = n -> p, *g = p -> p; bool l = left(n);
35     connect(l ? n -> r : n -> l, p, l);
36     if(!is_root(p)) connect(n, g, left(p)); else n -> p = g;
37     connect(p, n, !l); pull(p), pull(n);
38 }
39 inline void splay(node *n) {
40     while(!is_root(n)) {
41         node *p = n -> p;
42         if(!is_root(p)) push(p -> p);
43         push(p), push(n);
44         if(!is_root(p)) rotate(left(n) ^ left(p) ? n : p);
45         rotate(n);
46     }
47     push(n);
48 }
49 inline void expose(node *n) {
50     node *last = NULL;
51     for(node *m = n; m; m = m -> p) {

```

```

52     splay(m);
53     m -> w -= sum(last); m -> w += sum(m -> r);
54     m -> r = last; last = m; pull(m);
55 }
56 splay(n);
57 }
58 inline void evert(node *n){expose(n);n -> f ^= 1;}
59 inline void link(node *m,node *n){evert(m);expose(n);m->p=n;n->w+=sum(m);}
60 inline void cut(node *m,node *n){evert(m);expose(n);n->l=n->l->p=NULL;}
61 inline node* path_max(node *m,node *n){evert(m);expose(n);return n->m;}
62 inline int size(node *n){evert(n);return sum(n);}
63 bool connected(node *m,node *n){expose(m);expose(n);return m->p!=NULL;}
64 struct edge {
65     int a, b, w, id;
66     bool operator<(const edge e) const {return w != e.w ? w > e.w : id > e.id;}
67 };
68 int n, m, o;
69 node *v[MAXN], *ev[MAXM];
70 vector<edge> ed;
71 set<edge> s;
72 void delete_edge(edge e){cut(v[e.a],ev[e.id]);cut(v[e.b],ev[e.id]);}
73 void add_edge(edge e) {link(v[e.a],ev[e.id]);link(v[e.b],ev[e.id]);}
74 void new_edge(int a, int b, int w) {
75     int id = ed.size();
76     edge e = {a, b, w, id};
77     ed.push_back(e);
78     ev[id] = new node(w, 0, id);
79     if(connected(v[a], v[b])) {
80         node* m = path_max(v[a], v[b]);
81         if(m -> v <= w) return;
82         delete_edge(ed[m -> id]);
83         s.erase(ed[m -> id]);
84     }
85     add_edge(e);
86     s.insert(e);
87 }
88 void init(){for(int i=1;i<=n;i++){v[i]=new node(0,1,-i);}}

```

6.6 Dinic

```

1 const int MAXN = 5000+5 ;
2 typedef long long T;
3 const T INF = 1e15 ;
4 const T eps = 0;
5 struct edge {
6     int a, b;
7     T cap,flow;
8     int yo, x, y;
9 };
10 struct Dinic {
11     int s,t,d[MAXN], ptr[MAXN] ;
12     vector<edge>e;

```

```

13     vector<int>g[MAXN];
14     void init() {
15         e.clear();
16         memset(d,0,sizeof(d));
17         for(int i = 0; i < MAXN ; i++)g[i].clear();
18     }
19     void addEdge(int a,int b,T cap, int x = -1, int y = -1) {
20         edge e1 = { a, b, cap, 0, 1, x, y } ;
21         edge e2 = { b, a, 0, 0, 0, x, y } ;
22         g[a].push_back((int)e.size());
23         e.push_back(e1);
24         g[b].push_back((int)e.size());
25         e.push_back(e2);
26     }
27     bool bfs() {
28         queue < int > Q ;
29         Q.push(s);
30         memset(d,-1,sizeof(d));
31         d[s]=0 ;
32         while (!Q.empty()) {
33             int u=Q.front() ;
34             Q.pop() ;
35             for(int i=0;i<g[u].size(); i++) {
36                 int id=g[u][i];
37                 int v=e[id].b;
38                 // printf("%d %d %0.3lf %0.3lf\n",u,v,e[id].cap,e[id].flow);
39                 if(d[v]==-1&&e[id].flow<e[id].cap) {
40                     Q.push(v) ;
41                     d[v]=d[u]+1 ;
42                 }
43             }
44         }
45         return d[t]!=-1 ;
46     }
47     T dfs(int u,T flow) {
48         if (flow<=eps) return 0 ;
49         if ( u==t ) return flow ;
50         for(int& i = ptr[u] ; i<g[u].size(); i++) {
51             int id = g[u][i];
52             int v = e[id].b ;
53             if ( d[v] != d[u]+1 ) continue ;
54             T pushed = dfs (v,min (flow,e[id].cap-e[id].flow)) ;
55             //cout << "pushed " << pushed << endl;
56             if (pushed>eps) {
57                 e [id].flow+=pushed ;
58                 e [id^1].flow-=pushed ;
59                 return pushed ;
60             }
61         }
62         return 0 ;
63     }
64     T dinic() {
65         T flow = 0 ;

```

```

66     while(true) {
67         if(!bfs()) break ;
68         memset(ptr, 0, sizeof(ptr)) ;
69         while (true){
70             T pushed = dfs(s,INF );
71             if(pushed<=eps)break;
72             flow += pushed ;
73         }
74     }
75     return flow ;
76 }
77 };

```

6.7 FasterMaxflowButWeird

```

1  //
2  .....
3  only works in planer graph!!
4  //
5  .....
6  #include <bits/stdc++.h>
7
8  using namespace std;
9
10 const int Nmax = 1200 + 7;
11 const int Mmax = 100 * Nmax;
12 const int NIL = -1;
13
14 struct Edge
15 {
16     int v;
17     int capacity;
18     int flow;
19     int urm;
20 };
21 Edge Graph[Mmax];
22 int head[Nmax];
23
24 long long excess[Nmax];
25 int height[Nmax];
26 bool active[Nmax];
27 int pointer[Nmax];
28
29 int countHeight[2 * Nmax];
30
31 queue<int> Q;
32
33 int N, M, countEdges;

```

```

34 void addEdge(int u, int v, int cap1, int cap2)
35 {
36     Graph[countEdges] = {v, cap1, 0, head[u]};
37     head[u] = countEdges++;
38
39     Graph[countEdges] = {u, cap2, 0, head[v]};
40     head[v] = countEdges++;
41 }
42
43 void enqueue(int v)
44 {
45     if (!active[v] && excess[v] > 0)
46     {
47         active[v] = true;
48         Q.push(v);
49     }
50 }
51
52 void gap(int k)
53 {
54     for (int i = 1; i <= N; ++i)
55     {
56         if (height[i] >= k)
57         {
58             countHeight[ height[i] ]--;
59             height[i] = max(height[i], N + 1);
60             countHeight[ height[i] ]++;
61
62             enqueue(i);
63         }
64     }
65 }
66
67 void push(int u, int p)
68 {
69     int v = Graph[p].v;
70     long long delta = min(excess[u], (long long) (Graph[p].capacity - Graph[p].
71     flow));
72
73     if (height[u] <= height[v] || delta == 0) return;
74
75     Graph[p].flow += delta;
76     Graph[p ^ 1].flow -= delta;
77
78     excess[u] -= delta;
79     excess[v] += delta;
80
81     enqueue(v);
82 }
83
84 void relabel(int v)
85 {

```

```

86     countHeight[ height[v] ]--;
87
88     height[v] = 2 * N;
89
90     for (int p = head[v]; p != NIL; p = Graph[p].urm)
91     {
92         if (Graph[p].capacity - Graph[p].flow > 0)
93             height[v] = min(height[v], height[ Graph[p].v ] + 1);
94     }
95
96     countHeight[ height[v] ]++;
97
98     enqueue(v);
99 }
100
101 void discharge(int u)
102 {
103     bool doneGap = 0;
104
105     while (excess[u] > 0)
106     {
107         if (countHeight[ height[u] ] == 1 && (doneGap == false))
108         {
109             gap(height[u]);
110
111             doneGap = true;
112             pointer[u] = NIL;
113         }
114
115         if (pointer[u] == NIL)
116         {
117             relabel(u);
118             pointer[u] = head[u];
119         }
120         else
121         {
122             int p = pointer[u];
123             int v = Graph[p].v;
124
125             if (height[u] >= height[v] + 1 && Graph[p].capacity > Graph[p].flow)
126                 push(u, p);
127             else
128                 pointer[u] = Graph[ pointer[u] ].urm;
129         }
130     }
131 }
132
133 void initPreflow(int S, int T)
134 {
135     for (int i = 1; i <= N; ++i)
136     {
137         excess[i] = 0;
138         height[i] = 0;

```

```

139         pointer[i] = head[i];
140     }
141
142     height[S] = N;
143     active[S] = active[T] = true;
144
145     countHeight[N] = 1;
146     countHeight[0] = N - 1;
147
148     for (int p = head[S]; p != NIL; p = Graph[p].urm)
149     {
150         excess[S] += Graph[p].capacity;
151         push(S, p);
152     }
153 }
154
155 long long push_relabel(int S, int T)
156 {
157     initPreflow(S, T);
158
159     while (Q.empty() == false)
160     {
161         int v = Q.front();
162         Q.pop();
163         active[v] = false;
164         discharge(v);
165     }
166
167     long long flow = 0;
168
169     for (int p = head[S]; p != NIL; p = Graph[p].urm)
170         flow += Graph[p].flow;
171
172     return flow;
173 }
174
175 int main()
176 {
177     //freopen("data.in", "r", stdin);
178
179     scanf("%d ", &N);
180     countEdges = 0;
181
182     scanf("%d ", &M);
183     int s, t;
184     scanf("%d %d", &s, &t);
185     for (int i = 1; i <= N; ++i)
186         head[i] = NIL;
187
188     int x, y, c;
189
190     while (M--)
191     {

```

```

192     scanf("%d %d %d", &x, &y, &c);
193     addEdge(x, y, c, 0);
194 }
195
196 cout << push_relabel(s, t) << "\n";
197 return 0;
198 }
199
200 //
201 // .....
202 // One limitation of the HLPP implementation is that you can't recover the
203 // weights for the full flow - use Dinic's for this.
204 // .....
205
206 template <int MAXN, class T = int> struct HLPP {
207     const T INF = numeric_limits<T>::max();
208     struct edge {
209         int to, rev;
210         T f;
211     };
212     int s = MAXN - 1, t = MAXN - 2;
213     vector<edge> adj[MAXN];
214     vector<int> lst[MAXN], gap[MAXN];
215     T excess[MAXN];
216     int highest, height[MAXN], cnt[MAXN], work;
217     void addEdge(int from, int to, int f, bool isDirected = true) {
218         adj[from].push_back({to, adj[to].size(), f});
219         adj[to].push_back({from, adj[from].size() - 1, isDirected ? 0 : f});
220     }
221     void updHeight(int v, int nh) {
222         work++;
223         if (height[v] != MAXN)
224             cnt[height[v]]--;
225         height[v] = nh;
226         if (nh == MAXN)
227             return;
228         cnt[nh]++, highest = nh;
229         gap[nh].push_back(v);
230         if (excess[v] > 0)
231             lst[nh].push_back(v);
232     }
233     void globalRelabel() {
234         work = 0;
235         fill(all(height), MAXN);
236         fill(all(cnt), 0);
237         for (int i = 0; i < highest; i++)
238             lst[i].clear(), gap[i].clear();
239         height[t] = 0;
240         queue<int> q({t});
241         while (!q.empty()) {
242             int v = q.front();

```

```

243             q.pop();
244             for (auto &e : adj[v])
245                 if (height[e.to] == MAXN && adj[e.to][e.rev].f > 0)
246                     q.push(e.to), updHeight(e.to, height[v] + 1);
247             highest = height[v];
248         }
249     }
250     void push(int v, edge &e) {
251         if (excess[e.to] == 0)
252             lst[height[e.to]].push_back(e.to);
253         T df = min(excess[v], e.f);
254         e.f -= df, adj[e.to][e.rev].f += df;
255         excess[v] -= df, excess[e.to] += df;
256     }
257     void discharge(int v) {
258         int nh = MAXN;
259         for (auto &e : adj[v]) {
260             if (e.f > 0) {
261                 if (height[v] == height[e.to] + 1) {
262                     push(v, e);
263                     if (excess[v] <= 0)
264                         return;
265                 } else
266                     nh = min(nh, height[e.to] + 1);
267             }
268         }
269         if (cnt[height[v]] > 1)
270             updHeight(v, nh);
271         else {
272             for (int i = height[v]; i <= highest; i++) {
273                 for (auto j : gap[i])
274                     updHeight(j, MAXN);
275                 gap[i].clear();
276             }
277         }
278     }
279     T calc(int heur_n = MAXN) {
280         fill(all(excess), 0);
281         excess[s] = INF, excess[t] = -INF;
282         globalRelabel();
283         for (auto &e : adj[s])
284             push(s, e);
285         for (; highest >= 0; highest--) {
286             while (!lst[highest].empty()) {
287                 int v = lst[highest].back();
288                 lst[highest].pop_back();
289                 discharge(v);
290                 if (work > 4 * heur_n)
291                     globalRelabel();
292             }
293         }
294         return excess[t] + INF;
295     }

```

293

};

6.8 GlobalMinCut

```

1 //Global Mincut ( stoer Wagner N^3)
2 typedef long long ll;
3 const ll maxN = 305 ;
4 const ll inf = 0x3f3f3f3f ;
5 // Stoer-wagner algorithm, complexity: O(n^3)
6 // compute the global minimum cut with self-loop ignored.
7 ll n,m, g[maxN][maxN]; //1-indexed, initialised with 0
8 bool vis[maxN];
9 ll dis[maxN], v[maxN];
10 ll stoer_wagner() {
11     ll i, j, now, ans=inf;
12     for(i=0; i<n; i++) v[i]=i+1;
13     while(n>1) {
14         for(now=0, i=1; i<n; i++) dis[v[i]]=0;
15         for(i=1; i<n; i++) {
16             swap(v[now], v[i-1]);
17             for(now=j=i; j<n; j++) {
18                 dis[v[j]]+=g[v[i-1]][v[j]];
19                 if(dis[v[now]]<dis[v[j]]) now=j;
20             }
21         }
22         ans=min(ans, dis[v[now]]);
23         for(j=0; j<n; j++)
24             g[v[j]][v[now-1]]=g[v[now-1]][v[j]]+=g[v[j]][v[now]] ;
25         v[now]=v[--n];
26     }
27     return ans ;
28 }

```

6.9 Gomori-Hu Tree

```

1 // ret = gomori-hu tree
2 // edge e : i to ret[i].second with cost=ret[i].first
3 Dinic dinic ;
4 struct edg{int s,e,x;};
5 vector<edg>edgs; // original graph
6
7 void clear(){edgs.clear();}
8 void add_edge(int s,int e,int x) {
9     edgs.push_back({s,e,x});
10 }
11 #define maxn 1000
12 bool vis[maxn];
13 void dfs(int x) {
14     if(vis[x]) return;
15     vis[x]=1;

```

```

16     for(auto&i:dinic.g[x]) {
17         if(dinic.e[i].cap-dinic.e[i].flow>0)
18             dfs(dinic.e[i].b);
19     }
20 }
21 vector<pair<int,int> > solve(int n) {
22     vector<pair<int,int> > ret(n);
23     for(int i=1; i<n; i++) {
24         dinic.init();
25         for(auto&j : edgs) {
26             dinic.addEdge(j.s,j.e,j.x);
27             dinic.addEdge(j.e,j.s,j.x);
28         }
29         dinic.s=i;
30         dinic.t=ret[i].second;
31         ret[i].first=dinic.dinic();
32         memset(vis,0,sizeof vis);
33         dfs(i);
34         for(int j=i+1; j<n; j++) {
35             if(ret[j].second==ret[i].second&&vis[j]) {
36                 ret[j].second=i;
37             }
38         }
39     }
40     return ret;
41 }
42 }

```

6.10 Kth Minimum Cut

```

1 //finding weight of the Kth Minimum Cut
2 //courtesy : antiquality
3 #include<bits/stdc++.h>
4 using namespace std;
5 typedef long long ll;
6 const int maxn = 103;
7 const int maxm = 5035;
8 const int INF = 1e9;
9 struct Edge
10 {
11     int u,v,f,c;
12     Edge(int a=0, int b=0, int c=0, int d=0):u(a),v(b),f(c),c(d) {}
13 }edges[maxn],e[maxn];
14 struct node
15 {
16     int len,val;
17     bool a[maxn];
18     bool operator < (node a) const
19     {
20         return val > a.val;
21     }
22     void init(){memset(a, 0, sizeof a);}

```

```

23 }tmp,trans;
24 bool vis[maxn];
25 int n,m,k,S,T,src,snk;
26 int edgeTot,head[maxn],nxt[maxn],lv[maxn];
27 priority_queue<node> q;
28
29 void addedge(int u, int v, int c)
30 {
31     edges[edgeTot] = Edge(u, v, 0, c), nxt[edgeTot] = head[u], head[u] = edgeTot,
32     ++edgeTot;
33     edges[edgeTot] = Edge(v, u, 0, 0), nxt[edgeTot] = head[v], head[v] = edgeTot,
34     ++edgeTot;
35 }
36 bool buildLevel()
37 {
38     queue<int> q;
39     memset(lv, 0, sizeof lv);
40     q.push(S), lv[S] = 1;
41     for (int tmp; q.size(); )
42     {
43         tmp = q.front(), q.pop();
44         for (int i=head[tmp]; i!=-1; i=nxt[i])
45         {
46             int v = edges[i].v;
47             if (edges[i].f < edges[i].c&&!lv[v]){
48                 lv[v] = lv[tmp]+1, q.push(v);
49                 if (v==T) return true;
50             }
51         }
52     }
53     return false;
54 }
55 int fndPath(int x, int lim)
56 {
57     if (!lim||x==T) return lim;
58     int sum = 0;
59     for (int i=head[x]; i!=-1&&sum < lim; i=nxt[i])
60     {
61         int v = edges[i].v, val = 0;
62         if (lv[v]==lv[x]+1&&edges[i].f < edges[i].c){
63             if ((val = fndPath(v, min(edges[i].c-edges[i].f, lim-sum)))){
64                 sum += val, edges[i].f += val, edges[i^1].f -= val;
65             }else lv[v] = -1;
66         }
67     }
68     return sum;
69 }
70 int dinic()
71 {
72     int ret = 0, val = 0;
73     while (buildLevel())
74         while ((val = fndPath(S, INF))) ret += val;
75     return ret;

```

```

74 }
75 void calc(node &x)
76 {
77     memset(head, -1, sizeof head), edgeTot = 0;
78     for (int i=1; i<=m; i++) addedge(e[i].u, e[i].v, e[i].c);
79     addedge(S, src, INF), addedge(snk, T, INF);
80     for (int i=1; i<=x.len; i++)
81         if (x.a[i]) addedge(S, i, INF);
82         else addedge(i, T, INF);
83     queue<int> q;
84     x.val = dinic(), q.push(S);
85     memset(vis, 0, sizeof vis);
86     for (int i=1; i<=n; i++) x.a[i] = 0;
87     for (int tmp; q.size(); )
88     {
89         tmp = q.front(), q.pop();
90         for (int i=head[tmp]; i!=-1; i=nxt[i])
91             if (edges[i].f < edges[i].c){
92                 int v = edges[i].v;
93                 if (!vis[v]) vis[v] = true, x.a[v] = 1, q.push(v);
94             }
95     }
96 }
97 void solve()
98 {
99     scanf("%d%d%d%d", &n, &m, &k, &src, &snk), S = 0, T = n+1;
100     for (int i=1; i<=m; i++) scanf("%d%d", &e[i].u, &e[i].v, &e[i].c);
101     tmp.len = 0, tmp.a[src] = true, calc(tmp), q.push(tmp);
102     while (--k)
103     {
104         tmp = q.top(), q.pop();
105         for (int i=tmp.len+1; i<=n; i++)
106         {
107             trans.init(), trans.len = i, trans.a[i] = !tmp.a[i];
108             for (int j=1; j<i; j++) trans.a[j] = tmp.a[j];
109             calc(trans), q.push(trans);
110         }
111     }
112     printf("%d\n", q.top().val);
113 }

```

6.11 MinCostMaxFlow

```

1 typedef long long T1;//for cost
2 typedef long long T2;//for flow
3 const int maxn = 20100;
4 const T1 INF = 1e12;
5 const T2 inf = 1e12;
6 const T1 eps = 0;
7 struct Edge {
8     int from, to;
9     T2 cap, flow;

```

```

10     T1 cost;
11 };
12 struct MCMF { //0-indexed
13     int n, m, s, t;
14     vector<Edge> edges;
15     vector<int> G[maxn];
16     int p[maxn], inq[maxn];
17     T1 d[maxn];
18     T2 a[maxn];
19     void init(int n) {
20         this->n = n;
21         for(int i = 0; i < n; i++) G[i].clear();
22         edges.clear();
23     }
24     void AddEdge(int from, int to, T2 cap, T1 cost) {
25         edges.push_back((Edge){from, to, cap, 0, cost});
26         edges.push_back((Edge){to, from, 0, 0, -cost});
27         m = edges.size();
28         G[from].push_back(m-2);
29         G[to].push_back(m-1);
30     }
31     pair<T1, T2> Mincost(int s, int t) { //bellmanFord
32         T1 tot_cost = 0;
33         T2 tot_flow = 0;
34         while(true) {
35             for(int i = 0; i < n; i++) d[i] = INF;
36             d[s] = 0;
37             p[s] = 0;
38             a[s] = inf;
39             bool up=true;
40             while(up) {
41                 up=false;
42                 for(int u = 0; u < n; u++) {
43                     if(d[u]-INF>=-eps) continue;
44                     for(int j:G[u]) {
45                         Edge &e=edges[j];
46                         if(e.cap > e.flow && d[e.to] > d[u] + e.cost+eps) {
47                             d[e.to] = d[u] + e.cost;
48                             p[e.to] = j;
49                             a[e.to] = min(a[u], e.cap - e.flow);
50                             up=true;
51                         }
52                     }
53                 }
54             }
55             if(abs(d[t]-INF)<=eps) break;
56             tot_cost += (T1)d[t] * a[t];
57             tot_flow += (T2)a[t];
58             int u = t;
59             while(u != s) {
60                 edges[p[u]].flow += a[t];
61                 edges[p[u]^1].flow -= a[t];
62                 u = edges[p[u]].from;

```

```

63     }
64 }
65     return {tot_cost, tot_flow};
66 }
67 pair<T1, T2> Mincost2(int s, int t) { //SPFA
68     T1 tot_cost = 0;
69     T2 tot_flow = 0;
70     while(true) {
71         for(int i = 0; i < n; i++) d[i] = INF;
72         memset(inq, 0, sizeof(inq));
73         d[s] = 0;
74         inq[s] = 1;
75         p[s] = 0;
76         a[s] = inf;
77         queue<int> Q;
78         srand(time(NULL));
79         Q.push(s);
80         while(!Q.empty()) {
81             int u = Q.front();
82             Q.pop();
83             inq[u] = 0;
84             for(int i = 0; i < G[u].size(); i++) {
85                 Edge& e = edges[G[u][i]];
86                 if(e.cap > e.flow && d[e.to] > d[u] + e.cost+eps) {
87                     d[e.to] = d[u] + e.cost;
88                     p[e.to] = G[u][i];
89                     a[e.to] = min(a[u], e.cap - e.flow);
90                     if(!inq[e.to]) {
91                         Q.push(e.to);
92                         inq[e.to] = 1;
93                     }
94                 }
95             }
96         }
97         if(abs(d[t]-INF)<=eps) break;
98         tot_cost += (T1)d[t] * a[t];
99         tot_flow += a[t];
100         int u = t;
101         while(u != s) {
102             edges[p[u]].flow += a[t];
103             edges[p[u]^1].flow -= a[t];
104             u = edges[p[u]].from;
105         }
106     }
107     return {tot_cost, tot_flow};
108 }
109 }
110 } mcmf;

```

6.12 MinimumCostCirculation

```
1 #define MAXN 105
```

```

2 typedef double T;
3 struct cost_circ {
4     struct edg {
5         int pos, cap, rev;
6         T cost;
7     };
8     vector<edg> gph[MAXN];
9     void clear() {
10         for(int i=0; i<MAXN; i++) gph[i].clear();
11     }
12     void add_edge(int s, int e, int x, T c) {
13         //cout << s << " " << e << " " << x << " " << c << endl;
14         gph[s].push_back({e, x, (int)gph[e].size(), c});
15         gph[e].push_back({s, 0, (int)gph[s].size()-1, -c});
16     }
17
18     int par[MAXN], pae[MAXN];
19     T dist[MAXN];
20     int negative_cycle(int n) {
21         bool mark[MAXN] = {};
22         memset(dist, 0, sizeof(dist));
23         int upd = -1;
24         for(int i=0; i<=n; i++) {
25             for(int j=0; j<n; j++) {
26                 int idx = 0;
27                 for(auto &k : gph[j]) {
28                     if(k.cap > 0 && dist[k.pos] > dist[j] + k.cost) {
29                         dist[k.pos] = dist[j] + k.cost;
30                         par[k.pos] = j;
31                         pae[k.pos] = idx;
32                         if(i == n) {
33                             upd = j;
34                             while(!mark[upd]) {
35                                 mark[upd] = 1;
36                                 upd = par[upd];
37                             }
38                             return upd;
39                         }
40                     }
41                     idx++;
42                 }
43             }
44         }
45
46         return -1;
47     }
48     typedef pair<int,int> pi ;
49     T match(int n) {
50         int rt = -1;
51         T ans = 0;
52         while(~(rt = negative_cycle(n))) {
53             bool mark[MAXN] = {};
54             vector<pi > cyc;

```

```

55         while(!mark[rt]) {
56             cyc.push_back(pi(par[rt], pae[rt]));
57             mark[rt] = 1;
58             rt = par[rt];
59         }
60         reverse(cyc.begin(), cyc.end());
61         int capv = 1e9;
62         for(auto &i : cyc) {
63             auto e = &gph[i.first][i.second];
64             capv = min(capv, e->cap);
65         }
66         for(auto &i : cyc) {
67             auto e = &gph[i.first][i.second];
68             e->cap -= capv;
69             gph[e->pos][e->rev].cap += capv;
70             ans += e->cost * capv;
71         }
72     }
73     return ans;
74 }
75 } circ;
76 //call circ.match(n+1)

```

6.13 ____FlowNotes

```

1  /*
2      *****
3  Partial Ordered Set
4
5  Let S be a set of elements and  $\leq$  be a partial ordering on the set. That is,
6  for some elements x and y in S we may have  $\leq xy$ . The only properties that  $\leq$ 
7  must satisfy are reflexivity ( $\leq xx$ ), antisymmetry (if  $\leq xy$  and  $\leq yx$ , then  $x = y$ ),
8  and transitivity (if  $\leq xy$  and  $\leq yz$ , then  $\leq xz$ ). Note that because it is a
9  partial ordering,
10 not all x and y are comparable.
11 An example of a partially ordered set (poset) is the set of points (x ,y) in the
12 Cartesian plane with the
13 operator  $(x1 ,y1) \leq (x2 ,y2)$  iff  $\leq x1x2$  and  $\leq y1y2$  (e.g. NCPC 2007 problem).
14
15 We define a chain C to be a subset of S such that the elements of C can be
16 labelled  $x1 ,x2 ,\dots,xn$ 
17 such that  $\leq \leq x1x2 \leq \dots,xn$ . A partition P is a set of chains where each element
18 occurs in exactly one chain.
19
20 We define an antichain A to be a subset of S such that for any x and y in A we
21 have neither  $\leq xy$  nor  $\leq yx$ .
22 That is, no two elements of an antichain are comparable.
23
24

```

19	Dilworth's Theorem	20	We can define the width of a poset in two ways, which is the result of Dilworth's Theorem.
21	One definition is the size of the maximum antichain; the other is the size of the minimum partition.	22	It is easy to see that any partition must have at least as many chains as the size of the maximum antichain because every element of the antichain must be in a different chain.
23	Dilworth's Theorem	24	tells us that there exists a partition of exactly that size and can be proved inductively.
25	Calculating the Width	26	So how does one calculate the width of a poset? To solve this problem in general, we can use maximum matching
27	on a bipartite graph. Duplicate each element x in S as ux and vx and if xy (for $x \neq y$) then add the edge .	28	If you compute the maximum matching on this graph, this is equivalent to partitioning the elements into chains,
29	where if the edge is chosen then x and y are in the same chain. In particular, if the size of the matching is m	30	and $ S = n$, then the number of partitions is $n - m$. Notice that this gives a bijection between partitions and
31	matchings, so the maximum matching gives the minimum partition, which we know is equal to the width as given	32	by Dilworth's Theorem.
33	*****	34	*****
35	*****	36	*****
37	MAX/MIN Closure :	38	Closure is a directed subgraph with no outgoing edges.
39	MAX/Min Closure means a closure with max/min sum of weighted nodes outside the graph.	40	For MAX, join source with positive nodes, sink with negative nodes and capacities are
41	absolute value, infinite capacity between existing edges.	42	For min, source & sink is reversed.
43	Ans = sum of positive nodes - min cut (For max)	44	Ans = sum of negative nodes + min cut (For min)
45	*****	46	*****
47	*****	48	*****
49	Demand Capacity :	50	We add a new source 's and a new sink 't, a new edge from the source 's to every other vertex,
51	a new edge for every vertex to the sink 't, and one edge from t to s.	52	*****
53	*****	54	*****
55	*****	56	*****
57	*****	58	*****
59	*****	60	*****
61	*****	62	*****
63	*****	64	*****
65	*****	66	*****
67	*****	68	*****
69	König-Egeváry Theorem	70	The König-Egeváry theorem (König's theorem) , asserts that
71	the maximum matching is equal to the vertex cover number for a bipartite graph.	72	*****
73	*****	74	*****
75	Minimum Vertex Cover in bipartite graph	76	Let's say the maximum matching is M.
77	And orientation of edges will be :	78	Those edges that belong to M will go from right to left,
79	all other edges will go from left to right.	80	Now run DFS starting at all left vertices that are not incident to edges in M.
81	Some vertices of the graph will become visited during this DFS and some not-visited.	82	To get minimum vertex cover you take all visited right vertices of M, and all not-visited left vertices of M.
83	*****	84	*****
85	*****	86	*****
87	*****	88	*****
89	Minimum Path cover in a Directed Acyclic Graph :	90	find the minimum number of vertex-disjoint paths to cover each vertex in V.
91	make a bipartite graph $G' = \{V_{out} \times V_{in}, E\}$	92	V_{out} = nodes having positive out-degree
93	*****	94	*****

94	Vin = nodes have positive in-degree	34	*****
95	G' has a matching of size m iff there exists n-m vertex-disjoint paths that cover each vertex in G.		
96		135	Finding an euler circuit in a graph G with both directed and undirected edges
97	*****		where undirected edges can be used once.
		136	
98	*****	137	Euler circuit exists iff every vettex's indegree = outdegree
		138	
99		139	make a bipartite graph.One partition contains all edges and other contains all
100	In a DAG delete some edges so that each 'nodes indegree=outdegree=k and maximise the sum of the deleted 'edges cost :		nodes.
101		140	For each directed edge , connect this edge to its head endpoint .
102	From source to each node i (1<=i<=n), add edge of capacity k, cost 0	141	For each undirected edge , connect this edge to its both endpoints .
103	From each node j (n<j<=n+n) to Sink, add edge of capacity k, cost 0	142	Now from Source node , add edges to each G's edge.
104	For all i,j (1<=i,j<=n) If in the main graph from i to j , there is an edge , then add from each node i to n+j of capacity 1	143	all of these edges' capacity will be 1.
105	and cost = original 'edges cost. Then run the min cost max flow algorithm.	144	Now from each G's node , add edges to sink.
106	if max flow = n*k , then minimum cost will be the answer.	145	Edge i's capacity will be total degree/2 of node i.
107	Otherwise it will not be possible.	146	
108		147	Then check the flow and so on.
109		148	
110	*****	149	
		150	*****
		151	*****
111	*****	152	
		153	Find a subgraph where edge / node] is maximum.
112		154	
113	Given a graph where each node has value and each 'edges cost = value[y] xor value[x].	155	make a bipartite graph.One partition contains all edges and other contains all
114	Initially some 'nodes values are given and we have to find other 'nodes value such that sum of all edge is minimum.		nodes.
115		156	For each G's edge , connect this edge to its both endpoints .
116	We make LOG(MAX_VAL) graphs for each bit.	157	Now from Source node , add edges to each G's edge.
117	For ith bit,for each known node u,	158	all of these edges' capacity will be 1.
118	add edge with capacity INF from source to u whose i bit is set.	159	Now from each G's node , add edges to sink.
119	otherwise add edge with capacity INF from node to sink.	160	Edge i's capacity will be X.
120	add the regular edge with capacity 1 .	161	
121	Then after finding the MinCut , let U are those node which are in source's component.	162	we do binary search on X's value.
122	Then their bit will be set.	163	if(Flow < Number of Edges) left = mid
123		164	otherwise right = mid.
124	*****	165	
		166	*****
125	*****	167	*****
		168	*****
			*/
126			
127	find a subgraph such that cost(edge)-cost(node) is maximum		
128	Consider each EDGE & NODE as node.		
129	Add edge from Source to each EDGE of capacity w & from each EDGE to both nodes of capacity INF & from each node to sink of capacity p.		
130	Ans = sum of w - maxflow		
131			
132			
133	*****	169	*****

6.14 Bipartite Matching (Hopcroft Karp)

```

1 #define mset0(x) memset(x,0,sizeof(x))
2 const int maxN = 50000 ;
3 const int maxM = 50000 ;
4 struct HopcroftKarp {
5     int vis[maxN], level[maxN], ml[maxN], mr[maxM];

```

```

6 vector<int> edge[maxN]; // constructing edges for left part only
7 void init(int n) {
8     for (int i = 1; i <= n; ++i) edge[i].clear();
9 }
10 void add(int u, int v) {
11     edge[u].push_back(v);
12 }
13 bool dfs(int u) {
14     vis[u] = true;
15     for (vector<int>::iterator it = edge[u].begin(); it != edge[u].end(); ++
16 it) {
17         int v = mr[*it];
18         if (v == -1 || (!vis[v] && level[u] < level[v] && dfs(v))) {
19             ml[u] = *it;
20             mr[*it] = u;
21             return true;
22         }
23     }
24     return false;
25 }
26 int matching(int n) { // n for left
27     mset0(vis);
28     mset0(level);
29     memset(ml, -1, sizeof(ml));
30     memset(mr, -1, sizeof(mr));
31     for (int match = 0;;) {
32         queue<int> que;
33         for (int i = 1; i <= n; ++i) {
34             if (ml[i] == -1) {
35                 level[i] = 0;
36                 que.push(i);
37             } else level[i] = -1;
38         }
39         while (!que.empty()) {
40             int u = que.front();
41             que.pop();
42             for (vector<int>::iterator it = edge[u].begin(); it != edge[u].
43 end(); ++it) {
44                 int v = mr[*it];
45                 if (v != -1 && level[v] < 0) {
46                     level[v] = level[u] + 1;
47                     que.push(v);
48                 }
49             }
50         }
51         for (int i = 1; i <= n; ++i) vis[i] = false;
52         int d = 0;
53         for (int i = 1; i <= n; ++i) if (ml[i] == -1 && dfs(i)) ++d;
54         if (d == 0) return match;
55         match += d;
56 };

```

6.15 Bipartite Matching(Kuhn)

```

1 class KuhnBPM{
2 public:
3     int n, k;
4     vector < vector<int> > g;
5     vector<int> pairs_of_right, pairs_of_left;
6     vector<bool> used;
7     bool kuhn (int v) {
8         if (used[v]) return false;
9         used[v] = true;
10        for (int i = 0; i < g[v].size(); ++i) {
11            int to = g[v][i];
12            if (pairs_of_right[to] == -1 || kuhn (pairs_of_right[to])) {
13                pairs_of_right[to] = v;
14                pairs_of_left[v] = to;
15                return true;
16            }
17        }
18        return false;
19    }
20    int find_max_matching(vector<vector<int> > &_g, int _n, int _k) {
21        g = _g;
22        n = _n; //n is number of vertices in left part of graph
23        k = _k; //k is number of vertices in right part of graph
24        pairs_of_right = vector<int> (k, -1);
25        pairs_of_left = vector<int> (n, -1);
26        //pairs_of_right[i] is a neighbor of vertex i from right part, on marked
27        edge
28        //pairs_of_left[i] is a neighbor of vertex i from left part, on marked
29        edge
30        used = vector<bool> (n, false);
31        bool path_found;
32        do {
33            fill(used.begin(), used.end(), false);
34            path_found = false;
35            //remember to start only from free vertices which are not visited yet
36            for (int i = 0; i < n; ++i)
37                if (pairs_of_left[i] < 0 && !used[i]) path_found |= kuhn (i);
38        } while (path_found);
39        int res = 0 ;
40        for(int i = 0; i < k; i++) if(pairs_of_right[i] != -1) res++ ;
41        return res;
42    }
43 };

```

6.16 General Matching (Edmonds Blossom)

```

1 //complexity EV^2,0-indexed
2 const int maxn=250;
3 struct Edge{int to,next;}edge[maxn*maxn];

```

```

4 int Adj[maxn], Size;
5 void init() {memset(Adj, -1, sizeof(Adj)); Size=0;}
6 void add_edge(int u, int v) {
7     edge[Size].to=v; edge[Size].next=Adj[u]; Adj[u]=Size++;
8 }
9 int n, Match[maxn];
10 bool G[maxn][maxn];
11 int Start, Finish, NewBase, Father[maxn], Base[maxn];
12 bool InQueue[maxn], InPath[maxn], InBlossom[maxn];
13 int Count;
14 queue<int> q;
15 int FindCommonAncestor(int u, int v) {
16     memset(InPath, false, sizeof(InPath));
17     while(true) {
18         u=Base[u]; InPath[u]=true;
19         if(u==Start) break;
20         u=Father[Match[u]];
21     }
22     while(true) {
23         v=Base[v];
24         if(InPath[v]) break;
25         v=Father[Match[v]];
26     }
27     return v;
28 }
29 void ResetTrace(int u) {
30     int v;
31     while(Base[u] != NewBase) {
32         v=Match[u];
33         InBlossom[Base[u]] = InBlossom[Base[v]] = true;
34         u=Father[v];
35         if(Base[u] != NewBase) Father[u]=v;
36     }
37 }
38 void BlossomContract(int u, int v) {
39     NewBase=FindCommonAncestor(u, v);
40     memset(InBlossom, false, sizeof(InBlossom));
41     ResetTrace(u); ResetTrace(v);
42     if(Base[u] != NewBase) Father[u]=v;
43     if(Base[v] != NewBase) Father[v]=u;
44     for(int tu=1; tu<=n; tu++) {
45         if(InBlossom[Base[tu]]) {
46             Base[tu]=NewBase;
47             if(!InQueue[tu]) {
48                 q.push(tu);
49                 InQueue[tu]=true;
50             }
51         }
52     }
53 }
54 void FindAugmentingPath() {
55     memset(InQueue, false, sizeof(InQueue));
56     memset(Father, 0, sizeof(Father));

```

```

57     for(int i=1; i<=n; i++) Base[i]=i;
58     while(!q.empty()) q.pop();
59     q.push(Start); InQueue[Start]=true;
60     Finish=0;
61     while(!q.empty()) {
62         int u=q.front(); //InQueue[u]=false;
63         q.pop();
64         for(int i=Adj[u]; ~i; i=edge[i].next) {
65             int v=edge[i].to;
66             if((Base[u] != Base[v]) && Match[u] != v) {
67                 if(v==Start || (Match[v]>0 && Father[Match[v]]>0))
68                     BlossomContract(u, v);
69                 else if(Father[v]==0) {
70                     Father[v]=u;
71                     if(Match[v]>0) {
72                         q.push(Match[v]);
73                         InQueue[Match[v]]=true;
74                     }
75                     else {
76                         Finish=v;
77                         return;
78                     }
79                 }
80             }
81         }
82     }
83 }
84 void AugmentPath() {
85     int u, v, w; u=Finish;
86     while(u>0) {
87         v=Father[u]; w=Match[v];
88         Match[v]=u; Match[u]=v; u=w;
89     }
90 }
91 void Edmonds() {
92     memset(Match, 0, sizeof(Match));
93     for(int u=1; u<=n; u++) {
94         if(Match[u]==0) {
95             Start=u;
96             FindAugmentingPath();
97             if(Finish>0) AugmentPath();
98         }
99     }
100 }
101 void PrintMatch() {
102     Count=0;
103     for(int i=1; i<=n; i++) if(Match[i]) Count++;
104     printf("%d\n", Count);
105     for(int u=1; u<=n; u++) if(u<Match[u]) printf("%d %d\n", u, Match[u]);
106 }

```

6.17 StableMarriage

```

1 // man-preference's priority is high
2 const int maxn=1050;
3 queue<int> q;
4 int future_wife[maxn],future_husband[maxn],
5 order[maxn][maxn],perfer[maxn][maxn],nextt[maxn];
6 //order[i][j]=indexOfMan i in j-th women'sListOfPreference
7 //prefer[i]=listOfWomen inOrderOf decreasingPreference
8 void enage(int man,int woman) {
9     int m1=future_husband[woman];
10    if(m1==0) {
11        future_husband[woman]=man;
12        future_wife[man]=woman;
13    } else {
14        future_wife[m1]=0;
15        future_husband[woman]=man;
16        future_wife[man]=woman;
17        q.push(m1);
18    }
19 }
20 void stableMarriage() {
21     while(!q.empty())q.pop();
22     int n,x;scanf("%d",&n);
23     for(int i=1; i<=n; i++) {
24         for(int j=1; j<=n; j++)
25             scanf("%d",&perfer[i][j]);
26         nextt[i]=1;q.push(i);future_wife[i]=0;
27     }
28     for(int i=1; i<=n; i++) {
29         for(int j=1; j<=n; j++)
30             scanf("%d",&x),order[i][x]=j;
31         future_husband[i]=0;
32     }
33     while(!q.empty()) {
34         int man=q.front();q.pop();
35         int woman=perfer[man][nextt[man]++];
36         if(future_husband[woman]==0) enage(man,woman);
37         else if(order[woman][man]<order[woman][future_husband[woman]]) enage(man,
38             woman);
39         else q.push(man);
40     }
41 }

```

6.18 Weighted Bipartite Matching(Hungarian)

```

1 #define MAX 666
2 #define MAXIMIZE +1
3 #define MINIMIZE -1
4 #define inf (~0U >> 1)
5 #define clr(ar) memset(ar, 0, sizeof(ar))

```

```

6 #define ran(a, b) (((rand() << 15) ^ rand()) % ((b) - (a) + 1)) + (a)
7 namespace wm{ /// hash = 581023
8     bool visited[MAX];
9     int U[MAX], V[MAX], P[MAX], way[MAX], minv[MAX], match[MAX], ar[MAX][MAX];
10
11     /// n = number of row and m = number of columns in 1 based, flag = MAXIMIZE
12     or MINIMIZE
13     /// match[i] contains the column to which row i is matched
14     int hungarian(int n, int m, int mat[MAX][MAX], int flag){
15         clr(U), clr(V), clr(P), clr(ar), clr(way);
16
17         for (int i = 1; i <= n; i++){
18             for (int j = 1; j <= m; j++){
19                 ar[i][j] = mat[i][j];
20                 if (flag == MAXIMIZE) ar[i][j] = -ar[i][j];
21             }
22         }
23         if (n > m) m = n;
24
25         int i, j, a, b, c, d, r, w;
26         for (i = 1; i <= n; i++){
27             P[0] = i, b = 0;
28             for (j = 0; j <= m; j++) minv[j] = inf, visited[j] = false;
29
30             do{
31                 visited[b] = true;
32                 a = P[b], d = 0, w = inf;
33
34                 for (j = 1; j <= m; j++){
35                     if (!visited[j]){
36                         r = ar[a][j] - U[a] - V[j];
37                         if (r < minv[j]) minv[j] = r, way[j] = b;
38                         if (minv[j] < w) w = minv[j], d = j;
39                     }
40                 }
41
42                 for (j = 0; j <= m; j++){
43                     if (visited[j]) U[P[j]] += w, V[j] -= w;
44                     else minv[j] -= w;
45                 }
46                 b = d;
47             } while (P[b] != 0);
48
49             do{
50                 d = way[b];
51                 P[b] = P[d], b = d;
52             } while (b != 0);
53         }
54         for (j = 1; j <= m; j++) match[P[j]] = j;
55
56         return (flag == MINIMIZE) ? -V[0] : V[0];
57     }
58 }

```

6.19 Weighted Bipartite Matching2(Hungarian)

```

1 typedef long long ll;
2 #define MAX 105
3 #define maxM 107
4 #define maxN 107
5 const ll inf = 1e12+7;
6 template<typename T>
7 struct KuhnMunkras { // n for left, m for right
8     int n, m, match[maxM];
9     T g[maxN][maxM], lx[maxN], ly[maxM], slack[maxM];
10    bool vx[maxN], vy[maxM];
11    void init(int n_, int m_) {
12        n = n_, m = m_;
13        for(int i=1; i<=n; i++)
14            for(int j=1; j<=m; j++)
15                g[i][j] = -inf;
16    }
17    void add(int u, int v, T w) {
18        g[u][v] = w;
19    }
20    bool find(int x) {
21        vx[x] = true;
22        for (int y = 1; y <= m; ++y) {
23            if (!vy[y]) {
24                T delta = lx[x] + ly[y] - g[x][y];
25                if (delta == T(0)) {
26                    vy[y] = true;
27                    if (match[y] == 0 || find(match[y])) {
28                        match[y] = x;
29                        return true;
30                    }
31                } else slack[y] = min(slack[y], delta);
32            }
33        }
34        return false;
35    }
36    pair<T, T> matching() { // maximum weight matching
37        fill(lx + 1, lx + 1 + n, 0);
38        memset(ly, 0, sizeof(ly)); memset(match, 0, sizeof(match));
39        for (int i = 1; i <= n; ++i) {
40            for (int j = 1; j <= m; ++j) lx[i] = max(lx[i], g[i][j]);
41        }
42        for (int k = 1; k <= n; ++k) {
43            fill(slack + 1, slack + 1 + m, numeric_limits<T>::max());
44            while (true) {
45                memset(vx, 0, sizeof(vx)); memset(vy, 0, sizeof(vy));
46                if (find(k)) break;
47            } else {
48                T delta = numeric_limits<T>::max();
49                for (int i = 1; i <= m; ++i) {
50                    if (!vy[i]) delta = min(delta, slack[i]);
51                }

```

```

52         for (int i = 1; i <= n; ++i) {
53             if (vx[i]) lx[i] -= delta;
54         }
55         for (int i = 1; i <= m; ++i) {
56             if (vy[i]) ly[i] += delta;
57             if (!vy[i]) slack[i] -= delta;
58         }
59     }
60 }
61
62 T resultCost = 0;
63 T resultMatch = 0;
64 for (int i = 1; i <= n; ++i) resultCost += lx[i];
65 for (int i = 1; i <= m; ++i) resultCost += ly[i];
66 for (int i = 1; i <= m; i++) {
67     if (g[match[i]][i] > -inf)
68         resultMatch++;
69     else resultCost += inf;
70 }
71 return {resultCost, resultMatch};
72 }
73 };

```

6.20 Weighted General Matching

```

1 /**for only perfect matching
2 struct MinimumWeightedMatching {
3     static const int MN = 105 ;
4     int n, edge[MN][MN];
5     int match[MN], dis[MN], onstk[MN] ;
6     vector<int> stk;
7     void init(int _n) {
8         n = _n;
9         for(int i=0; i<n; i++)
10             for(int j=0; j<n; j++)
11                 edge[i][j] = 0;
12    }
13    void add_edge(int u, int v, int w) {
14        edge[u][v] = edge[v][u] = w;
15    }
16    bool SPFA(int u) {
17        if(onstk[u]) return true;
18        stk.push_back(u);
19        onstk[u] = 1;
20        for(int v=0; v<n; v++) {
21            if(v != u && match[u] != v && !onstk[v]) {
22                int m = match[v];
23                if(dis[m] > dis[u] - edge[v][m] + edge[u][v]) {
24                    dis[m] = dis[u] - edge[v][m] + edge[u][v];
25                    onstk[v] = 1;
26                    stk.push_back(v);
27                    if(SPFA(m)) return true;

```

```

28         stk.pop_back();
29         onstk[v]=0;
30     }
31 }
32 }
33 onstk[u]=0;
34 stk.pop_back();
35 return false;
36 }
37 int solve() {
38     for(int i=0; i<n; i+=2) {
39         match[i]=i+1;
40         match[i+1]=i;
41     }
42     while(true) {
43         int found=0;
44         for(int i=0; i<n; i++) dis[i]=onstk[i]=0;
45         for(int i=0; i<n; i++) {
46             stk.clear();
47             if(!onstk[i]&&SPFA(i)) {
48                 found=1;
49                 while((stk.size())>=2) {
50                     int u = stk.back();
51                     stk.pop_back();
52                     int v = stk.back();
53                     stk.pop_back();
54                     match[v]=u;
55                     match[u]=v;
56                 }
57             }
58         }
59         if(!found)break;
60     }
61     int ret = 0;
62     for(int i=0; i<n; i++)ret+=edge[i][match[i]];
63     ret /= 2;
64     return ret ;
65 }
66 };

```

6.21 DMST

```

1 const int inf = 1e9 ;
2 struct edge {
3     int u, v, w;
4     edge() {}
5     edge(int a,int b,int c) : u(a), v(b), w(c) {}
6     bool operator < (const edge& o) const {
7         if (u == o.u)
8             if (v == o.v)return w < o.w;
9             else return v < o.v;
10        return u < o.u;

```

```

11    }
12    };
13    int dmst(vector<edge> &edges, int root, int n) {
14        int ans = 0;
15        int cur_nodes = n;
16        while (true) {
17            vector<int> lo(cur_nodes, inf), pi(cur_nodes, inf);
18            for (int i = 0; i < edges.size(); ++i) {
19                int u = edges[i].u, v = edges[i].v, w = edges[i].w;
20                if (w < lo[v] and u != v) {
21                    lo[v] = w;
22                    pi[v] = u;
23                }
24            }
25            lo[root] = 0;
26            for (int i = 0; i < lo.size(); ++i) {
27                if (i == root) continue;
28                if (lo[i] == inf) return -1;
29            }
30            int cur_id = 0;
31            vector<int> id(cur_nodes, -1), mark(cur_nodes, -1);
32            for (int i = 0; i < cur_nodes; ++i) {
33                ans += lo[i];
34                int u = i;
35                while (u != root and id[u] < 0 and mark[u] != i) {
36                    mark[u] = i;
37                    u = pi[u];
38                }
39                if (u != root and id[u] < 0) { // Cycle
40                    for (int v = pi[u]; v != u; v = pi[v]) id[v] = cur_id;
41                    id[u] = cur_id++;
42                }
43            }
44            if (cur_id == 0) break;
45            for (int i = 0; i < cur_nodes; ++i)
46                if (id[i] < 0) id[i] = cur_id++;
47            for (int i = 0; i < edges.size(); ++i) {
48                int u = edges[i].u, v = edges[i].v, w = edges[i].w;
49                edges[i].u = id[u];
50                edges[i].v = id[v];
51                if (id[u] != id[v]) edges[i].w -= lo[v];
52            }
53            cur_nodes = cur_id;
54            root = id[root];
55        }
56        return ans;
57    }

```

6.22 DMSTwithSolution

```

1 const int INF = 1029384756;
2 #define MAXN 1000

```

```

3 #define FOR(i,x) for(auto i :x )
4 struct edge_t {
5     int u,v,w;
6     set< pair<int,int> > add, sub;
7     edge_t() : u(-1), v(-1), w(0) {}
8     edge_t(int _u, int _v, int _w) {
9         u = _u;
10        v = _v;
11        w = _w;
12        add.insert({u, v});
13    }
14    edge_t& operator += (const edge_t& obj) {
15        w += obj.w;
16        for (auto it : obj.add) {
17            if (!sub.count(it)) add.insert(it);
18            else sub.erase(it);
19        }
20        for (auto it : obj.sub) {
21            if (!add.count(it)) sub.insert(it);
22            else add.erase(it);
23        }
24        return *this;
25    }
26    edge_t& operator -= (const edge_t& obj) {
27        w -= obj.w;
28        for (auto it : obj.sub) {
29            if (!sub.count(it)) add.insert(it);
30            else sub.erase(it);
31        }
32        for (auto it : obj.add) {
33            if (!add.count(it)) sub.insert(it);
34            else add.erase(it);
35        }
36        return *this;
37    }
38 } eg[MAXN*MAXN], prv[MAXN], EDGE_INF(-1,-1, INF);
39 int N,M;
40 int cid, incyc[MAXN], contracted[MAXN];
41 vector<int> E[MAXN];
42
43 edge_t dmst(int rt) {
44     edge_t cost;
45     for (int i=0; i<N; i++) {
46         contracted[i] = incyc[i] = 0;
47         prv[i] = EDGE_INF;
48     }
49     cid = 0;
50     int u,v;
51     while (true) {
52         for (v=0; v<N; v++) {
53             if (v != rt && !contracted[v] && prv[v].w == INF)
54                 break;
55         }

```

```

56         if (v >= N) break; // end
57         for (int i=0; i<M; i++) {
58             if (eg[i].v == v && eg[i].w < prv[v].w)
59                 prv[v] = eg[i];
60         }
61         if (prv[v].w == INF) // not connected
62             return EDGE_INF;
63         cost += prv[v];
64         for (u=prv[v].u; u!=v && u!=-1; u=prv[u].u);
65         if (u == -1) continue;
66         incyc[v] = ++cid;
67         for (u=prv[v].u; u!=v; u=prv[u].u) {
68             contracted[u] = 1;
69             incyc[u] = cid;
70         }
71         for (int i=0; i<M; i++) {
72             if (incyc[eg[i].u] != cid && incyc[eg[i].v] ==
73                 cid) {
74                 eg[i] -= prv[eg[i].v];
75             }
76         }
77         for (int i=0; i<M; i++) {
78             if (incyc[eg[i].u] == cid) eg[i].u = v;
79             if (incyc[eg[i].v] == cid) eg[i].v = v;
80             if (eg[i].u == eg[i].v) eg[i--] = eg[--M];
81         }
82         for (int i=0; i<N; i++) {
83             if (contracted[i]) continue;
84             if (prv[i].u>=0 && incyc[prv[i].u] == cid)
85                 prv[i].u = v;
86         }
87         prv[v] = EDGE_INF;
88     }
89     return cost;
90 }
91 #define F first
92 #define S second
93 void solve() {
94     edge_t cost = dmst(0);
95     for (auto it : cost.add) { // find a solution
96         E[it.F].push_back(it.S);
97         prv[it.S] = edge_t(it.F, it.S, 0);
98     }
99 }

```

6.23 ManhattanMST

```

1 struct point {
2     int x, y, index;
3     bool operator<(const point &p) const { return x == p.x ? y < p.y : x < p.x; }
4 } p[maxN];
5 struct node {

```

```

6   int value, p;
7 } T[maxN];
8
9 int query(int x) {
10   int r = maxint, p = -1;
11   for (; x <= n; x += (x & -x)) if (T[x].value < r) r = T[x].value, p = T[x].p;
12   return p;
13 }
14
15 void modify(int x, int w, int p) {
16   for (; x > 0; x -= (x & -x)) if (T[x].value > w) T[x].value = w, T[x].p = p;
17 }
18
19 // impl add & kruskal
20 int manhattan() {
21   for (int i = 1; i <= n; ++i) p[i].index = i;
22   for (int dir = 1; dir <= 4; ++dir) {
23     if (dir == 2 || dir == 4) {
24       for (int i = 1; i <= n; ++i) swap(p[i].x, p[i].y);
25     } else if (dir == 3) {
26       for (int i = 1; i <= n; ++i) p[i].x = -p[i].x;
27     }
28     sort(p + 1, p + 1 + n);
29     vector<int> v; static int a[maxN];
30     for (int i = 1; i <= n; ++i) a[i] = p[i].y - p[i].x, v.push_back(a[i]);
31     sort(v.begin(), v.end());
32     v.erase(unique(v.begin(), v.end()), v.end());
33     for (int i = 1; i <= n; ++i) a[i] = lower_bound(v.begin(), v.end(), a[i]) - v
34       .begin() + 1;
35     for (int i = 1; i <= n; ++i) T[i].value = maxint, T[i].p = -1;
36     for (int i = n; i >= 1; --i) {
37       int pos = query(a[i]);
38       if (pos != -1) add(p[i].index, p[pos].index, dist(p[i], p[pos]));
39       modify(a[i], p[i].x + p[i].y, i);
40     }
41   }
42   return kruskal();

```

6.24 PrimsBorukuvaMST

```

1  /*
2  MST : 'Prims Algorithm
3  initial distance of all nodes are INFINITE except the starting node , which
4  distance is 0.
5  and all nodes are marked as non-visited.
6  Now until all nodes are visited, pick the non-visited node u
7  which distance is lowest than all other non-visited nodes.
8  After picking any nodes, update its all adjacent node 'vs distance
9  if dis[u]+edgeCost(u,v)<dis[v] .
10 MST : 'Borukuvas Algorithm

```

```

11 *all edge cost must be distinct.
12 initially all nodes are individual components.
13 While there is more than one components,
14   for each components
15     find the smallest edge which connects this component
16     to any other component. And then add this edge.
17 These edges are the edges of minimum spanning tree. */

```

6.25 ChromaticNumber

```

1 //complexity n*2^n
2 #define fst first
3 #define snd second
4 #define all(c) ((c).begin()), ((c).end())
5 #define TEST(s) if (!(s)) { cout << __LINE__ << " " << #s << endl; exit(-1); }
6
7 struct Graph {
8   int n;
9   vector<vector<int>> adj;
10   Graph(int n) : n(n), adj(n) { }
11   void addEdge(int u, int v) {
12     adj[u].push_back(v);
13     adj[v].push_back(u);
14   }
15 };
16
17 int chromaticNumber(Graph g) {
18   const int N = 1 << g.n;
19   vector<int> nbh(g.n);
20   for (int u = 0; u < g.n; ++u)
21     for (int v: g.adj[u])
22       nbh[u] |= (1 << v);
23
24   int ans = g.n;
25   for (int d: {7}) { // ,11,21,33,87,93} {
26     long long mod = 1e9 + d;
27     vector<long long> ind(N), aux(N, 1);
28     ind[0] = 1;
29     for (int S = 1; S < N; ++S) {
30       int u = __builtin_ctz(S);
31       ind[S] = ind[S^(1<<u)] + ind[(S^(1<<u))&~nbh[u]];
32     }
33     for (int k = 1; k < ans; ++k) {
34       long long chi = 0;
35       for (int i = 0; i < N; ++i) {
36         int S = i ^ (i >> 1); // gray-code
37         aux[S] = (aux[S] * ind[S]) % mod;
38         chi += (i & 1) ? aux[S] : -aux[S];
39       }
40       if (chi % mod) ans = k;
41     }
42   }

```

```

43     return ans;
44 }

```

6.26 CycleEnumeration

```

1
2 // For a directed graph, it enumerates all cycles
3 // Hawick and James's implementation of Johnson algorithm.
4 // Complexity O(n+m) average time for each cycles,O(n+m) space.
5 #define fst first
6 #define snd second
7 #define all(c) ((c).begin()), ((c).end())
8 struct graph {
9     int n;
10    vector<vector<int>>> adj;
11    graph(int n) : n(n), adj(n) { }
12    void add_edge(int u, int v) {
13        adj[u].push_back(v);
14    }
15    void all_cycles() {
16        vector<int> S;
17        for (int s = 0; s < n; ++s) {
18            vector<bool> blocked(n);
19            vector<unordered_set<int>>> B(n);
20            function<void(int)> unblock = [&](int u) {
21                blocked[u] = false;
22                for (int v: B[u])
23                    if (blocked[v]) unblock(v);
24                B[u].clear();
25            };
26            function<bool(int)> rec = [&](int u) {
27                bool is_cycle = false;
28                blocked[u] = true;
29                S.push_back(u);
30                for (int v: adj[u]) {
31                    if (v < s) continue;
32                    if (v == s) {
33                        is_cycle = true; // S forms a cycle
34                        cout << "found cycle" << endl;
35                        for (auto w: S) cout << " " << w;
36                        cout << endl;
37                    } else if (!blocked[v] && rec(v)) is_cycle = true;
38                    if (is_cycle) unblock(u);
39                    else {
40                        for (int v: adj[u]) {
41                            if (v < s) continue;
42                            if (!B[v].count(u)) B[v].insert(u);
43                        }
44                    }
45                }
46                S.pop_back();
47                return is_cycle;

```

```

48     }; rec(s);
49 }
50 }
51 };
52 //graph.all_cycles()

```

6.27 EdgeColoring

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 /* Problem: Given a bipartite graph, find out minimum number of colors
4 to color all the edges such that no two adjacent edges have a same color
5 assigned. Number of minimum colors equals the max degree of a vertex in
6 the bipartite graph. We also need to assign colors to each edge.
7 */
8 /* Comment by 300iq:
9 Minimum answer is max degree.
10 If max degree ≤ 1, we can just color all edges in one color.
11 Else, let's split edges of the graph into two sets, such that max degree
12 will be ≤ (max degree + 1) / 2 in each of these two sets. You can do it
13 with euler circuit, add some dummy vertex to the left and right part, and
14 connect with them vertices with odd degree (and maybe you need to connect
15 them if they have odd degree). And then color the edges into two colors by
16 order of euler circuit. Then separate all initial edges into two groups by the
17 color. Then let's solve recursively for these two sets, and then just merge the
18 answers.
19 */
20 struct edge{int a, b;};
21 const int N = 1e6 + 7;
22 vector <int> g[N];
23 int col[N],pp;
24 vector <edge> glob_edges;
25 bool vis[N],us[N];
26 void dfs(int v){
27     us[v] = true;
28     while (g[v].size() > 0){
29         int ind = g[v].back();g[v].pop_back();
30         if (vis[ind])continue;
31         vis[ind] = true;col[ind] = (pp ^= 1);
32         dfs(glob_edges[ind].a == v ? glob_edges[ind].b : glob_edges[ind].a);
33     }
34 }
35 vector <int> solve(vector <edge> e){
36     if (e.empty())return {};
37     vector <int> l, r;
38     for (auto c : e){
39         l.push_back(c.a);r.push_back(c.b);
40     }
41     sort(l.begin(), l.end());sort(r.begin(), r.end());
42     l.resize(unique(l.begin(), l.end()) - l.begin());
43     r.resize(unique(r.begin(), r.end()) - r.begin());
44     glob_edges.clear();

```

```

44 int x = (int) l.size() + 1, y = (int) r.size() + 1;
45 for (int i = 0; i < x + y; i++) g[i].clear();
46 int ind = 0;
47 for (auto &c : e){
48     c.a = lower_bound(l.begin(), l.end(), c.a) - l.begin();
49     c.b = lower_bound(r.begin(), r.end(), c.b) - r.begin();
50     auto ret = c; ret.b += x;
51     glob_edges.push_back(ret);
52     g[ret.a].push_back(ind); g[ret.b].push_back(ind); ind++;
53 }
54 vector<int> ids;
55 int mx = 0;
56 for (int i = 0; i < x + y; i++){
57     us[i] = 0; mx = max(mx, (int) g[i].size());
58     if (g[i].size() % 2) ids.push_back(i);
59 }
60 bool bad = false;
61 for (int i = 0; i < x + y; i++){
62     if (g[i].size() > 1) bad = true;
63 }
64 if (!bad) {vector<int> res(ind); return res;}
65 else{
66     vector<int> deg(x + y);
67     for (int v : ids){
68         if (v < x){
69             glob_edges.push_back({v, x + y - 1});
70             g[v].push_back(ind);
71             g[x + y - 1].push_back(ind);
72             ind++;
73         }
74         else{
75             glob_edges.push_back({v, x - 1});
76             g[v].push_back(ind);
77             g[x - 1].push_back(ind);
78             ind++;
79         }
80     }
81     if (g[x - 1].size() % 2){
82         glob_edges.push_back({x - 1, x + y - 1});
83         g[x - 1].push_back(ind); g[x + y - 1].push_back(ind);
84         ind++;
85     }
86     for (int i = 0; i < ind; i++) col[i] = -1;
87     for (int i = 0; i < ind; i++) vis[i] = 0;
88     for (int i = 0; i < x + y; i++) if (!g[i].empty()) dfs(i);
89     vector<edge> to_l, to_r;
90     vector<int> cols;
91     for (int i = 0; i < (int) e.size(); i++){
92         cols.push_back(col[i]);
93         if (col[i] == 0) to_l.push_back(e[i]);
94         else to_r.push_back(e[i]);
95     }
96     auto x = solve(to_l), y = solve(to_r);

```

```

97     int mx = *max_element(x.begin(), x.end()) + 1;
98     int p_x = 0, p_y = 0;
99     vector<int> ans;
100     for (int i = 0; i < (int) e.size(); i++){
101         if (cols[i] == 0) ans.push_back(x[p_x++]);
102         else ans.push_back(mx + y[p_y++]);
103     }
104     return ans;
105 }
106 }
107 int main(){
108     vector<edge> e;
109     int m; cin >> m;
110     for (int i = 0; i < m; i++){
111         int a, b; cin >> a >> b;
112         a--, b--; e.push_back({a, b});
113     }
114     auto ret = solve(e);
115     cout<<*max_element(ret.begin(), ret.end())+1<<'\n';
116     for (int c : ret) cout << c + 1;
117 }

```

6.28 EulerWalk

```

1 /*
2  Eulerian undirected/directed path/cycle algorithm. Returns a list of nodes
3  in the Eulerian path/cycle with src at both start and end, or
4  empty list if no cycle/path exists. To get edge indices back, also put it->
5  second in s (and then ret).
6  Time: O(E) where E is the number of edges.
7  */
8 typedef pair<int,int> pii;
9 typedef vector<int> vi;
10 typedef vector<pii> vpi;
11 #define sz(x) (int)x.size()
12 struct V {
13     vector<pii> outs; // (dest, edge index)
14     int nins = 0;
15     void clr() {nins=0; outs.clear();}
16 };
17 #define ff first
18 #define ss second
19 vpi euler_walk(vector<V>& nodes, int nedges, int src=0) {
20     int c = 0;
21     for (auto &n: nodes) c += abs(n.nins - sz(n.outs));
22     if (c > 2) return {};
23     vector<vector<pii>::iterator> its;
24     for (auto &n: nodes)
25         its.push_back(n.outs.begin());
26     vector<bool> eu(nedges+7);
27     vpi ret, s = {{src, 0}};
28     while (!s.empty()) {

```

```

28     auto x = s.back();
29     auto& it = its[x.ff], end = nodes[x.ff].outs.end();
30     while(it != end && eu[it->second]) ++it;
31     if(it == end) { ret.push_back(x); s.pop_back(); }
32     else { s.push_back(*it); eu[it->second] = true; }
33 }
34 if(sz(ret) != nedges+1)
35     ret.clear(); // No Eulerian cycles/paths.
36 // else, non-cycle if ret.front() != ret.back()
37 reverse(ret.begin(), ret.end());
38 return ret;
39 }
40 vector<V>nodes(maxn);

```

6.29 MatrixTreeTheorem

```

1 /**
2  * To count the number of spanning trees in an undirected graph G
3  * create an N*N matrix mat, and for each edge (a,b) in G, do
4  *   mat[a][a]++, mat[b][b]++, mat[a][b]--, mat[b][a]--.
5  * Remove the last row and column, and take the determinant.
6  */

```

6.30 MaxClique

```

1 class MaxClique {
2 public:
3     static const int MV = 210;
4     int V;
5     int el[MV][MV/30+1];
6     int dp[MV];
7     int ans;
8     int s[MV][MV/30+1];
9     vector<int> sol;
10    void init(int v) {
11        V = v; ans = 0;
12        FZ(el); FZ(dp);
13    }
14    /* Zero Base */
15    void addEdge(int u, int v) {
16        if(u > v) swap(u, v);
17        if(u == v) return;
18        el[u][v/32] |= (1<<(v%32));
19    }
20    bool dfs(int v, int k) {
21        int c = 0, d = 0;
22        for(int i=0; i<(V+31)/32; i++) {
23            s[k][i] = el[v][i];
24            if(k != 1) s[k][i] &= s[k-1][i];
25            c += __builtin_popcount(s[k][i]);

```

```

26        }
27        if(c == 0) {
28            if(k > ans) {
29                ans = k;
30                sol.clear();
31                sol.push_back(v);
32                return 1;
33            }
34            return 0;
35        }
36        for(int i=0; i<(V+31)/32; i++) {
37            for(int a = s[k][i]; a ; d++) {
38                if(k + (c-d) <= ans) return 0;
39                int lb = a&(-a), lg = 0;
40                a ^= lb;
41                while(lb!=1) {
42                    lb = (unsigned int)(lb) >> 1;
43                    lg++;
44                }
45                int u = i*32 + lg;
46                if(k + dp[u] <= ans) return 0;
47                if(dfs(u, k+1)) {
48                    sol.push_back(v);
49                    return 1;
50                }
51            }
52        }
53        return 0;
54    }
55
56    int solve() {
57        for(int i=V-1; i>=0; i--) {
58            dfs(i, 1);
59            dp[i] = ans;
60        }
61        return ans;
62    }
63 };

```

6.31 MinimumMeanCycle

```

1 #define maxN 10
2 #define maxM 10
3 int dp[maxN][maxN]; // minimum mean cycle(allow negative weight)
4 double mmc(int n) {
5     for (int i = 0; i < n; ++i) {
6         memset(dp[i + 1], 0x7f, sizeof(dp[i + 1]));
7         for (int j = 1; j <= ec; ++j) {
8             int u = edge[j].u, v = edge[j].v, w = edge[j].w;
9             if (dp[i][u] != maxint)
10                dp[i + 1][v] = min(dp[i + 1][v], dp[i][u] + w);
11        }

```

```

12     }
13     double res = maxdbl;
14     for (int i = 1; i <= n; ++i) {
15         if (dp[n][i] == maxint) continue;
16         double value = -maxdbl;
17         for (int j = 0; j < n; ++j) {
18             value = max(value, double(dp[n][i] - dp[j][i]) / (n - j));
19         }
20         res = min(res, value);
21     }
22     return res;
23 }

```

6.32 TwoSat

```

1 typedef long long ll;
2 const int maxn = 2 * 100005;
3 struct TwoSat {
4     int n;
5     vector<int> G[maxn * 2];
6     // i = x', i+n = x , x = variable , i = index
7     bool mark[maxn * 2]; // store actual value
8     int S[maxn * 2], c;
9     bool dfs(int x) {
10         if (mark[x + n]) return false;
11         if (mark[x]) return true;
12         mark[x] = true;
13         S[c++] = x;
14         for (int i = 0; i < G[x].size(); i++) {
15             if (!dfs(G[x][i])) return false;
16         }
17         return true;
18     }
19     void init(int n) {
20         this->n = n;
21         for (int i = 0; i < n * 2; i++) G[i].clear();
22         memset(mark, 0, sizeof(mark));
23     }
24     void add_clause(int x, int y) {
25         G[x].push_back(y);
26         // printf("%d ----- %d\n", x + 1, y + 1);
27     }
28     bool solve() {
29         for (int i = 0; i < n; i++) {
30             if (!mark[i] && !mark[i + n]) {
31                 c = 0;
32                 if (!dfs(i)) {
33                     while (c > 0) mark[S[--c]] = false;
34                     if (!dfs(i + n)) return false;
35                 }
36             }
37         }

```

```

38         return true;
39     }
40 } sat;

```

6.33 Centroid Decomposition on Edge

```

1 vector<pair<int, int>> adj[maxn];
2 vector<pair<int, int>> edge[maxn];
3 int __cnt;
4 #define vv first
5 #define ww second
6 inline void addEdge(int u, int v, int w) {
7     edge[u].push_back({v, w});
8     edge[v].push_back({u, w});
9 }
10 namespace Compress {
11     // make all node's degree at most 3
12     void dfs(int u, int p = 0) {
13         int cu = u;
14         for (int i = 0; i < adj[u].size(); i++) {
15             int v = adj[u][i].first;
16             int w = adj[u][i].second;
17             if (v == p) continue;
18             dfs(v, u);
19             if (i < 2) addEdge(u, v, w);
20             else {
21                 addEdge(cu, ++__cnt, 0);
22                 cu = __cnt;
23                 addEdge(cu, v, w);
24             }
25         }
26     }
27 }
28 namespace CD {
29     int lvl[maxn], sub[maxn], p[maxn], vis[maxn];
30     int total = 0;
31     void calc(int u, int par) {
32         p[u] = par;
33         sub[u] = 1;
34         for (auto e : edge[u]) if (e.vv - par && !vis[e.vv])
35             calc(e.vv, u), sub[u] += sub[e.vv];
36     }
37     int centroid(int u, int par, int r) {
38         for (auto e : edge[u]) if (e.vv - par && !vis[e.vv])
39             if (sub[e.vv] >= r) return centroid(e.vv, u, r);
40         return u;
41     }
42     void calc2(int u) {
43         // cout << " yo yo " << u << endl;
44     }
45     void decompose(int u, int par) {
46         calc(u, 0);

```

```

47     total += sub[u];
48     if(sub[u]==1) return ;
49     int c = centroid(u,0,(sub[u]+2)/3);
50     int pp = p[c] ;
51     vis[pp] = 1 ; decompose(c,0) ; vis[pp]= 0;
52     vis[c]= 1; decompose(pp,0); vis[c] = 0 ;
53     //calc2(pp);
54     //calc2(c);
55 }
56 }
57 //Compress::dfs(1);
58 //CD::decompose(1,0);

```

6.34 Centroid Decomposition

```

1 // p[u] = parent of u in centroid tree
2 // d[x][u] = distance from u to a parent of u at level x of centroid tree
3 //           if u is in subtree of centroid c, then d[lvl[c]][u] = dist(c, 1)
4 // Taken from Rezwan Arefin
5 // If (x, y) edge exist, then x must be in adj[y] and y must be in adj[x]
6 const int maxn = 1e5 + 10;
7 vector<int> adj[maxn];
8 int lvl[maxn], sub[maxn], p[maxn], vis[maxn], d[18][maxn], ans[maxn];
9
10 void calc(int u, int par) { sub[u] = 1;
11     for(int v : adj[u]) if(v - par && !vis[v])
12         calc(v, u), sub[u] += sub[v];
13 }
14 int centroid(int u, int par, int r) {
15     for(int v : adj[u]) if(v - par && !vis[v])
16         if(sub[v] > r) return centroid(v, u, r);
17     return u;
18 }
19 void dfs(int l, int u, int par) {
20     if(par + 1) d[l][u] = d[l][par] + 1;
21     for(int v : adj[u]) if(v - par && !vis[v])
22         dfs(l, v, u);
23 }
24 void decompose(int u, int par) {
25     calc(u, -1);
26     int c = centroid(u, -1, sub[u] >> 1);
27     vis[c] = 1, p[c] = par, lvl[c] = 0;
28     if(par + 1) lvl[c] = lvl[par] + 1;
29     dfs(lvl[c], c, -1);
30     for(int v : adj[c]) if(v - par && !vis[v])
31         decompose(v, c);
32 }
33
34 void update(int u) {
35     for(int v = u; v + 1; v = p[v])
36         ans[v] = min(ans[v], d[lvl[v]][u]);
37 }

```

```

38 int query(int u) {
39     int ret = 1e9;
40     for(int v = u; v + 1; v = p[v])
41         ret = min(ret, ans[v] + d[lvl[v]][u]);
42     return ret;
43 }

```

6.35 CompressTree

```

1 typedef vector<int> vi;
2 typedef vector<pair<int,int> > vpi;
3 vpi compressTree(const vector<int>& subset) {
4     vi rev(n+1);
5     vi li = subset;
6     auto cmp = [&](int a, int b) {
7         return st[a] < st[b];
8     };
9     sort(li.begin(),li.end(), cmp);
10    int m = li.size()-1;
11    for(int i=0; i<m; i++) {
12        int a = li[i], b = li[i+1];
13        li.push_back(lca(a, b));
14    }
15    sort(li.begin(),li.end(), cmp);
16    li.erase(unique(li.begin(),li.end()), li.end());
17    for(int i=0; i<li.size(); i++) rev[li[i]] = i;
18    vpi ret = {pair<int,int>(0, li[0])};
19    for(int i=0; i<li.size()-1; i++) {
20        int a = li[i], b = li[i+1];
21        ret.emplace_back(rev[lca(a, b)],b);
22    }
23    return ret;
24 }

```

6.36 DSU

```

1 typedef int ll;
2 map<int,int> ck[1000006];
3 int id[1000006];
4 set<pair<int,int> > s[1000006];
5 int x, y, n, ans[1000006], offset[1000006];
6 vector<int> adj[1000006];
7 void go(int u,int p) {
8     ck[id[u]][0]=1;
9     s[id[u]].insert({-1,0});
10    for(int v : adj[u]) {
11        if(v==p) continue;
12        go(v,u);
13        if(s[id[v]].size()>s[id[u]].size()) {
14            for(auto p : s[id[u]]) {

```

```

15         int ki = p.second-1+offset[id[u]]-offset[id[v]];
16         int koyta = ck[id[v]][ki];
17         if(koyta!=0)s[id[v]].erase(s[id[v]].find({-koyta,ki}));
18         koyta -= p.first ;
19         s[id[v]].insert({-koyta,ki});
20         ck[id[v]][ki] = koyta ;
21     }
22     offset[id[v]]++;
23     swap(id[u],id[v]);
24 } else {
25     for(auto p : s[id[v]]) {
26         int ki = p.second+1+offset[id[v]]-offset[id[u]];
27         int koyta = ck[id[u]][ki];
28         if(koyta!=0)s[id[u]].erase(s[id[u]].find({-koyta,ki}));
29         koyta -= p.first ;
30         s[id[u]].insert({-koyta,ki});
31         ck[id[u]][ki] = koyta ;
32     }
33 }
34 }
35 auto it = *(s[id[u]].begin());
36 ans[u] = it.second + offset[id[u]] ;
37 }

```

6.37 DominatorTree (for Any Directed Graph)

```

1 #define Max 1000000
2 #define sz(u) u.size()
3 #define pb push_back
4 struct DominatorTree {
5     vector<int> g[Max], tree[Max], bucket[Max], rg[Max];
6     int sdом[Max], dom[Max], label[Max], arr[Max], rev[Max], T;
7     int papa[Max]; //store u's dominator in papa[u]
8     int n,m,par[Max], dsu[Max];
9     int Find(int u, int x=0) {
10         if(u == dsu[u])return x ? -1 : u;
11         int v = Find(dsu[u], x+1);
12         if(v<0)return u;
13         if(sdom[label[dsu[u]]]<sdom[label[u]])
14             label[u] = label[ dsu[u] ];
15         dsu[u] = v;
16         return x ? v : label[u];
17     }
18     void Union(int u, int v) {dsu[v] = u;}
19     void dfs0(int u) {
20         ++T;
21         rev[T]=u;
22         arr[u]=sdom[T]=label[T]=dsu[T]=T;
23         for(int i = 0; i < sz(g[u]); i++) {
24             int w = g[u][i];
25             if(!arr[w])dfs0(w), par[arr[w]]=arr[u];
26             rg[arr[w]].pb(arr[u]);

```

```

27     }
28 }
29 void dominator_tree_init() {
30     dfs0(1);
31     for(int i = T; i >= 1; i--) {
32         for(int j = 0; j<sz(rg[i]); j++)
33             sdom[i] = min(sdom[i], sdom[ Find(rg[i][j]) ]);
34         if(i > 1)bucket[ sdom[i] ].pb(i);
35         for(int j = 0; j < sz(bucket[i]); j++) {
36             int w = bucket[i][j];
37             int v = Find(w);
38             if( sdom[w] == sdom[v] )dom[w] = sdom[w];
39             else dom[w] = v;
40         }
41         if(i > 1)Union(par[i], i);
42     }
43     for(int i = 2; i <= T; i++) {
44         if(dom[i] != sdom[i])dom[i] = dom[ dom[i] ];
45         tree[ rev[ dom[i] ] ].pb(rev[i]);
46         papa[rev[i]]=rev[dom[i]];
47         //here rev[dom[i]] dominates rev[i]
48     }
49 }
50 void Clear() {
51     for(int i = 1; i <= n; i++) {
52         arr[i] = sdom[i] = dom[i] = par[i] = dsu[i] = label[i] = rev[i] = 0;
53         tree[i].clear();
54         g[i].clear();
55         rg[i].clear();
56         bucket[i].clear();
57     }
58     T=0;
59 }
60 } dtree;
61 //dree.g[u].pb(v)&then call dominator_tree_init()

```

6.38 DominatorTree

```

1 //construct DAG by dijkstra and then create dominator tree
2 #define uu second
3 typedef long long ll;
4 const int maxN = 100007;
5 const int inf=1e9;
6 struct DominatorTree {
7     int ans[maxN], par[maxN][25], level[maxN], dis[maxN];
8     vector<int>adj[maxN], cost[maxN], mama[maxN] ;
9     vector<pair<int, int> >nodes ;
10     void clr() {
11         nodes.clear();
12         for(int i = 0 ; i < maxN ; i++)adj[i].clear(), cost[i].clear(), mama[i].clear();
13         memset(par, 0, sizeof(par));

```

```

14     memset(level,0,sizeof(level));
15     memset(dis,0,sizeof(dis));
16     memset(ans,0,sizeof(ans));
17 }
18 int lca(int u,int v) {
19     if(level[u]< level[v])swap(u,v);
20     int tmp=1;
21     for(; (1<<tmp)<=level[u]; ++tmp);
22     --tmp;
23     for(int i=tmp; i>=0; --i) {
24         if(level[u]-(1<<i)>=level[v])u=par[u][i];
25     }
26     if(u==v) return u ;
27     for(int i = tmp ; i >= 0 ; --i) if(par[u][i]!=par[v][i])u=par[u][i],v=par
[v][i] ;
28     return par[u][0];
29 }
30 void dijkstra( int n, int src ) {
31     priority_queue< pair < int, int > > pq ;
32     pq.push( make_pair( 0, src ) ) ;
33     for( int i = 0 ; i <= n ; i++ ) dis[i] = inf ;
34     dis[src] = 0LL ;
35     while( !pq.empty() ) {
36         pair<int,int>node = pq.top() ;
37         pq.pop();
38         for( int i = 0 ; i < adj[node.uu].size() ; i++ ) {
39             int v = adj[node.uu][i] ;
40             int w = cost[node.uu][i] ;
41             if(dis[node.uu]+w<dis[v]) {
42                 dis[v]=dis[node.uu]+w;
43                 pq.push(make_pair(-dis[v], v) );
44             }
45         }
46     }
47 }
48 void constructDag(int n, int src) {
49     for(int i=1 ; i <=n; i++)
50         nodes.push_back(make_pair(dis[i],i));
51     sort(nodes.begin(),nodes.end());
52     for(int i = 0 ; i < n ; i++) {
53         int u = nodes[i].uu;
54         for(int j=0; j < adj[u].size() ; j++ ) {
55             int v = adj[u][j];
56             int w = cost[u][j];
57             if(dis[u]+w==dis[v])mama[v].push_back(u);
58         }
59     }
60 }
61 void dominatorTree(int n,int src) {
62     dijkstra(n,src);
63     constructDag(n,src);
64     for(int i = 0; i<20; i++)par[1][i]=1;
65     level[1]=1;

```

```

66     n=nodes.size();
67     for(int i=1; i < n ; i++) {
68         int u = nodes[i].second ;
69         if(mama[u].size()==0) {
70             continue;
71         }
72         int sz = mama[u].size();
73         int jaime = mama[u][0] ;
74         int cur = 1 ;
75         while(cur < sz ) {
76             jaime = lca( jaime, mama[u][cur]) ;
77             cur++;
78         }
79         par[u][0]=jaime;
80         level[u]=1+level[jaime];
81         for( int i = 1 ; i <20 ; i++ )par[u][i] = par[par[u][i-1]][i-1] ;
82     }
83 }
84 };

```

6.39 F_x_times_x_le_N_Truck

```

1 void calc(int l,int r,int L,int R){
2     if(l>r)return;
3     if(L==R){
4         for(int i=l;i<=r;i++)ans[i]=L;
5         return;
6     }
7     int mid=(l+r)/2;
8     int ans=F(mid);
9     calc(l,mid-1,ans,R);
10    calc(mid+1,r,L,ans);
11 }
12 //call calc(1,n,0,n);

```

6.40 HLD

```

1 HeavyLightDecomposition
2
3 LazySegmentTree Tree ;
4 int sz[MAX];
5 int in[MAX];
6 int rin[MAX];
7 int out[MAX];
8 int head[MAX];
9 int par[MAX];
10 vector<int>g[MAX];
11 void dfs_sz(int u,int p) {
12     sz[u] = 1;
13     par[u] = p;

```

```

14     for(auto &v: g[u]) {
15         if(v==p) continue;
16         dfs_sz(v,u);
17         sz[u] += sz[v];
18         if(sz[v] > sz[g[u][0]])
19             swap(v,g[u][0]);
20     }
21 }
22 int t;
23 void dfs_hld(int u,int p) {
24     in[u] = ++t;
25     rin[in[u]] = u;
26     for(auto v: g[u]) {
27         if(v==p) continue;
28         head[v] = (v == g[u][0] ? head[u] : v);
29         dfs_hld(v,u);
30     }
31     out[u] = t;
32 }
33 bool isParent(int p,int u){
34     return in[p]<=in[u]&&out[u]<=out[p];
35 }
36 int n ;
37 int pathQuery(int u,int v){
38     int ret = -inf;
39     while(true){
40         if(isParent(head[u],v)) break;
41         ret=max(ret,Tree.queryRange(1,1,n,in[head[u]],in[u]));
42         u=par[head[u]];
43     }
44     swap(u,v);
45     while(true){
46         if(isParent(head[u],v)) break;
47         ret=max(ret,Tree.queryRange(1,1,n,in[head[u]],in[u]));
48         u=par[head[u]];
49     }
50     if(in[v]<in[u]) swap(u,v);
51     ret = max(ret,Tree.queryRange(1,1,n,in[u],in[v]));
52     return ret;
53 }
54 void updateSubTree(int u,int val){
55     Tree.updateRange(1,1,n,in[u],out[u],val);
56 }
57 void buildHLD(int root){
58     dfs_sz(root,root);
59     head[root]=root;
60     dfs_hld(root,root);
61 }
62 // call buildHLD

```

6.41 LCA (using SparseTable)

```

1 int Par[MAX], D[MAX];
2 void BuildSparse(int n){
3     //Using 0 for uninitialized parent (Remember and use carefully)
4     for(int i=1; i<=n; i++) for(int j=0; j<LOG; j++) P[i][j] = 0;
5
6     for(int i=1; i<=n; i++) P[i][0] = Par[i];
7     for(int j=1; j<LOG; j++){
8         for(int i=1; i<=n; i++){
9             if(P[i][j-1] != 0){
10                 int x = P[i][j-1];
11                 P[i][j] = P[x][j-1];
12             }
13         }
14     }
15 }
16
17 int LCA(int p,int q){
18     if(D[p]<D[q]) swap(p,q);
19
20     int Log = log2(D[p])+1;
21     for(int i=Log;i>=0;i--) if(D[p]-D[q] >= (1<<i)) p = P[p][i];
22     if(p==q) return p;
23
24     for(int i=Log;i>=0;i--) if(P[p][i]!=0 && P[p][i] != P[q][i]) {p = P[p][i]; q = P[q][i];}
25     int LCA = Par[p];
26     return LCA;
27 }
28
29 //Note 1 : Reversing v while merging (u,v) might be necessary in some case.
30 //Note 2 : We assign cost of an edge as the cost of the child node.
31 //Note 3 : In case values/costs for each node given (instead of each edge), merge LCA too along with merging u and v.
32
33 int kth_par(int u,int d){
34     int Log = log2(d)+1;
35     for(int i=Log; i>=0; i--) if(d >= (1<<i)) {u = P[u][i]; d -= (1<<i);}
36     return u;
37 }

```

6.42 LCA,O(VlogV+Q)

```

1 //0-indexed , root = 0
2 typedef vector<pair<int,int> > vpi;
3 typedef vector<vpi> graph;
4 typedef long long ll ;
5 int sz(graph& C) {
6     return C.size();
7 }
8 template<class T>
9 struct RMQ {
10     vector<vector<T>> jmp;

```

```

11 RMQ(const vector<T>& V) {
12     int N = V.size(), on = 1, depth = 1;
13     while (on < V.size()) on *= 2, depth++;
14     jmp.assign(depth, V);
15     for(int i=0; i<depth-1; i++) for(int j=0; j<N; j++)
16         jmp[i+1][j]=min(jmp[i][j], jmp[i][min(N-1, j+(1<<i))]);
17 }
18 T query(int a, int b) {
19     assert(a < b); // or return inf if a == b
20     int dep = 31-__builtin_clz(b-a);
21     return min(jmp[dep][a], jmp[dep][b-(1<<dep)]);
22 }
23 };
24 struct LCA {
25     vector<int> time;
26     vector<long long> dist;
27     RMQ<pair<int, int>> rmq;
28     LCA(graph& C) : time(sz(C), -99), dist(sz(C)), rmq(dfs(C)) {}
29     vpi dfs(graph& C) {
30         vector<tuple<int, int, int, ll>> q(1);
31         vector<pair<int, int>> ret;
32         int T = 0, v, p, d;
33         ll di;
34         while (!q.empty()) {
35             tie(v, p, d, di) = q.back();
36             q.pop_back();
37             if (d) ret.emplace_back(d, p);
38             time[v] = T++;
39             dist[v] = di;
40             for(auto &e: C[v]) if (e.first != p)
41                 q.emplace_back(e.first, v, d+1, di + e.second);
42         }
43         return ret;
44     }
45     int query(int a, int b) {
46         if (a == b) return a;
47         a = time[a], b = time[b];
48         return rmq.query(min(a, b), max(a, b)).second;
49     }
50     long long distance(int a, int b) {
51         int lca = query(a, b);
52         return dist[a] + dist[b] - 2 * dist[lca];
53     }
54 };

```

6.43 MO on Tree

```

1 typedef pair< pair<int, int> , pair<int, int>> que;
2 vector< que >query;
3
4 int S = sqrt(100005);
5 #define L first.first

```

```

6 #define R first.second
7 #define lca second.first
8 #define id second.second
9
10 bool cmp(const que &P, const que &Q){
11     if(P.L/S!=Q.L/S) return P.L/S<Q.L/S;
12     if((P.L/S)&1) return P.R>Q.R;
13     return P.R<Q.R;
14 }
15
16 void insert(int u){}
17 void erase(int u){}
18
19 void up(int idx) {
20     int u = node[idx];
21     vis[u]?erase(u):insert(u);
22     vis[u] ^= 1;
23 }
24
25 void addQuery(int u, int v) {
26     if(st[u]>st[v]) swap(u, v); //st=dfs start time
27     int lca = LCA(u, v);
28     if(lca!=u) query.push_back({{en[u], st[v]}, {st[lca], i}});
29     else query.push_back({{st[u], st[v]}, {-1, i}});
30 }
31
32 void solve() {
33     sort(query.begin(), query.end(), cmp);
34     l=1;
35     r=0;
36
37     for (auto q : query){
38         while(l>q.L) up(--l);
39         while(r<q.R) up(++r);
40         while(l<q.L) up(l++);
41         while(r>q.R) up(r--);
42     }
43
44     if(q.lca!=-1) up(lca);
45     calc(ans);
46     if(q.lca!=-1) up(lca);
47 }

```

6.44 Path Path Intersection

```

1 vector<int>cmp;
2 int calc(int a, int b, int c, int d) {
3     int ac = lca(a, c);
4     if(level[ac]<level[b])
5         return 0 ;
6     if(level[ac]<level[d])
7         return 0 ;

```

```

8   if(level[b]<level[d])
9       swap(b,d);
10  int ret = level[ac]-level[b]+1;
11  cmp.push_back(ac);
12  cmp.push_back(b);
13  return ret;
14 }
15 pair<int,int> solve(int a,int b,int c,int d) {
16     cmp.clear();
17     int ab= lca(a,b);
18     int cd= lca(c,d);
19     int ans = 0;
20     ans += calc(a,ab,c,cd);
21     ans += calc(a,ab,d,cd);
22     ans += calc(b,ab,c,cd);
23     ans += calc(b,ab,d,cd);
24     int f = 0;
25     f += calc(ab,ab,c,cd);
26     f += calc(ab,ab,d,cd);
27     f += calc(a,ab,cd,cd);
28     f += calc(b,ab,cd,cd);
29     ans -= f ;
30     if(ab==cd)
31         ans++;
32     sort(cmp.begin(),cmp.end());
33     cmp.erase(unique(cmp.begin(),cmp.end()),cmp.end());
34     pair<int,int>ret={-1,-1};
35     assert(cmp.size()<=3);
36     int mxdis=0;
37     for(int i=0;i<cmp.size();i++){
38         for(int j=i;j<cmp.size();j++){
39             int u = cmp[i];
40             int v = cmp[j];
41             int lc = lca(u,v);
42             int dis = level[u]+level[v]-2*level[lc]+1;
43             if(dis>mxdis){
44                 ret={u,v};
45                 mxdis=dis;
46             }
47         }
48     }
49     return ret ;
50 }

```

6.45 0-1BFS

```

1  /*
2  if edge weight is 1, insert at the end
3  otherwise insert at the beginning
4  */

```

6.46 BellmanFord

```

1  typedef double ll;
2  const int maxn = 105;
3  const int maxm = 10005;
4  const ll inf = 1e9;
5  ll d[maxn],w[maxm];
6  int u[maxm],v[maxm],n,m;
7  bool BellmanFord() { //1-indexed
8      for(int i=1;i<=n;i++)d[i]=inf;
9      d[1]=0;
10     for(int i=1; i<=n; i++)
11         for(int j=0; j<m; j++)
12             if(d[u[j]]+w[j] < d[v[j]])
13                 d[v[j]]=d[u[j]]+w[j];
14     bool negCycle = false;
15     for(int j=0; j<m; j++)
16         if(d[u[j]]+w[j] < d[v[j]]) {
17             negCycle=true;
18             break;
19         }
20     return negCycle;
21 }

```

6.47 Dijkstra(V^2)

```

1  #define maxN 1000
2  int n;
3  int dis[maxN],cost[maxN][maxN],par[maxN];
4  void Dijkstra (int start) { // v^2
5      vector<int>found(n+1);
6      dis[start] = 0;
7      while (start != -1) {
8          found[start] = true;
9          int best = -1;
10         for (int k = 1; k <= n; k++)
11             if (!found[k]) {
12                 if (dis[k] > dis[start] + cost[start][k]) {
13                     dis[k] = dis[start] + cost[start][k];
14                     par[k] = start;
15                 }
16                 if (best== -1 || dis[k]<dis[best])best=k;
17             }
18         start = best;
19     }
20 }

```

6.48 Dijkstra(VlogV+E)

```

1 const int inf = 1e9;
2 const int N = 1e5;
3 int dis[N],n,m,k,p,g,t,a[N],par[N];
4 vector<int>adj[N],cost[N];
5 #define uu second
6 #define ww first
7 void dijkstra(int src ) { //VlogV+E
8     priority_queue< pair < int, int > > pq ;
9     pq.push( make_pair( 0, src ) ) ;
10    for( int i = 0; i <= n ; i++ ) dis[i] = inf ;
11    dis[src] = 0 ;
12    while( !pq.empty() ) {
13        pair<int,int>node = pq.top() ;
14        int u = node.uu;
15        pq.pop();
16        if(dis[u]<=-node.ww)continue;
17        for( int i =0; i <adj[u].size() ; i++ ) {
18            int v = adj[u][i] ;
19            int w = cost[u][i] ;
20            if(dis[u]+w<dis[v]) {
21                dis[v]=dis[u]+w;
22                par[v]=u;
23                pq.push( make_pair( -dis[v], v ) ) ;
24            }
25        }
26    }
27 }

```

6.49 FloydWarshall

```

1 void floydWarshall(){
2     for(int k=1; k<=n; k++)
3         for(int i=1; i<=n; i++)
4             for(int j=1; j<=n; j++)
5                 dis[i][j]=min(dis[i][j],dis[i][k]+dis[k][j]);
6 }

```

6.50 JohnsonAlgorithm

```

1 #define MAX 90
2 #define clr(ar) memset(ar, 0, sizeof(ar))
3 typedef long long ll;
4 /// Johnson's algorithm for all pair shortest paths in sparse graphs
5 /// Complexity: O(N * M) + O(N * M * log(N))
6 const ll INF = (1LL << 60) - 666;
7 struct Edge{ // u to v edge
8     int u, v;
9     ll w;
10    Edge(){}
11    Edge(int u, int v, ll w) : u(u), v(v), w(w){}

```

```

12 };
13 bool bellman_ford(int n, int src, vector <Edge> E, vector <ll>& dis){
14     dis[src] = 0;
15     for (int i = 0; i <= n; i++){
16         int flag = 0;
17         for (auto e: E){
18             if ((dis[e.u] + e.w) < dis[e.v]){
19                 flag = 1;
20                 dis[e.v] = dis[e.u] + e.w;
21             }
22         }
23         if (flag == 0) return true;
24     }
25     return false;
26 }
27 vector <ll> dijkstra(int n, int src, vector <Edge> E, vector <ll> potential){
28     set<pair<ll, int> > S;
29     vector <ll> dis(n + 1, INF);
30     vector <ll> temp(n + 1, INF);
31     vector <pair<int, ll> > adj[n + 1];
32     dis[src] = temp[src] = 0;
33     S.insert(make_pair(temp[src], src));
34     for (auto e: E){
35         adj[e.u].push_back(make_pair(e.v, e.w));
36     }
37     int __sigh = 0;
38     while (!S.empty()){
39         pair<ll, int> cur = *(S.begin());
40         S.erase(cur);
41         int u = cur.second;
42         for (int i = 0; i < adj[u].size(); i++){
43             int v = adj[u][i].first;
44             ll w = adj[u][i].second;
45             if ((temp[u] + w) < temp[v]){
46                 S.erase(make_pair(temp[v], v));
47                 temp[v] = temp[u] + w;
48                 dis[v] = dis[u] + w;
49                 S.insert(make_pair(temp[v], v));
50             }
51         }
52     }
53     return dis;
54 }
55 void johnson(int n, ll ar[MAX][MAX], vector <Edge> E){
56     vector <ll> potential(n + 1, INF);
57     for (int i = 1; i <= n; i++) E.push_back(Edge(0, i, 0));
58
59     assert(bellman_ford(n, 0, E, potential));
60     for (int i = 1; i <= n; i++) E.pop_back();
61
62     for (int i = 1; i <= n; i++){
63         vector <ll> dis = dijkstra(n, i, E, potential);
64         for (int j = 1; j <= n; j++){

```

```

65         ar[i][j] = dis[j];
66     }
67 }
68 }
69 ll ar[MAX][MAX]; // output all pair shortest distance
70 vector <Edge> E; // input graph

```

7 Math

7.1 Fibonacci Combination Sums

```

1 //Fibonacci Combination Sums (Binet's Formula)
2 #include <bits/stdc++.h>
3 using namespace std;
4 #define ll long long int
5 #define MOD 1000000007
6 #define MAXK 205
7
8 ll bigMOD(ll n, ll r) {
9     if(r==0) return 1LL;
10    ll ret = bigMOD(n, r/2);
11    ret = (ret*ret)%MOD;
12    if(r%2==1) ret = (ret*n)%MOD;
13    return ret;
14 }
15
16 ll inverse(ll x) {return bigMOD(x%MOD, MOD - 2);}
17 ll cMOD(ll x) {x%=MOD; return (x>=0) ? x : x+MOD;}
18
19 struct cong {
20     ll a, b;
21     cong (ll x, ll y) {a = cMOD(x); b = cMOD(y);}
22     cong operator + (const cong &x) const {
23         return cong(a + x.a, b + x.b);
24     }
25     cong operator - (const cong &x) const {
26         return cong(a - x.a, b - x.b);
27     }
28     cong operator * (const cong &x) const {
29         ll c = x.a;
30         ll d = x.b;
31         return cong((a * c) + (5 * b * d), (a * d) + (b * c));
32     }
33     cong operator / (ll x) const {
34         return cong(a * inverse(x), b * inverse(x));
35     }
36     cong operator / (const cong g) const {
37         ll c = g.a;
38         ll d = g.b;
39         ll x = cMOD(a*c - 5*b*d) * inverse(c*c - 5*d*d);
40         ll y = cMOD(a*d - b*c) * inverse(5*d*d - c*c);

```

```

41         return cong(x, y);
42     }
43 };
44
45 cong bigMOD(cong n, ll r) {
46     if(r==0) return cong(1, 0);
47     cong ret = bigMOD(n, r/2);
48     ret = ret * ret;
49     if(r%2 == 1) ret = ret * n;
50     return ret;
51 }
52
53 cong geosum(cong x, ll n) {
54     if(x.a == 1 && x.b == 0) return cong(n+1, 0);
55     return (bigMOD(x, n+1) - cong(1, 0)) / (x - cong(1, 0));
56 }
57
58 cong alpha = cong(1, 1) / 2;
59 cong beta = cong(1, -1) / 2;
60
61 ll c[MAXK][MAXK];
62 ll a[MAXK][MAXK];
63
64 ll fibPower(ll n, int m) {
65     cong ans = cong(0, 0);
66     for(int j = 0; j <= m; j++) {
67         cong t = bigMOD(alpha, j) * bigMOD(beta, m - j);
68         cong x = cong(c[m][j], 0) * geosum(t, n);
69         if((m - j) & 1) ans = ans - x;
70         else ans = ans + x;
71     }
72     ans = ans * bigMOD((cong(0, 1) / 5), m);
73     return ans.a;
74 }
75
76 void PreCalc() {
77     for(int i = 0; i < MAXK; i++) {
78         c[i][0] = 1;
79         for(int j = 1; j <= i; j++) {
80             c[i][j] = c[i - 1][j] + c[i - 1][j - 1];
81             c[i][j] %= MOD;
82         }
83     }
84
85     a[0][0] = 1;
86     for(int i = 1; i < MAXK; i++) {
87         a[i][0] = 1;
88         for(int j = 1; j <= i+1; j++) {
89             a[i][j] = a[i - 1][j] + i * a[i - 1][j - 1];
90             a[i][j] %= MOD;
91         }
92     }
93 }

```

```

94 // query(n,k) = sum of C(Fib(i),k) in range 0 <= i <=n
95 ll query(ll n, int k) {
96     if(n < 0) return 0;
97     ll ans = 0;
98     for(int i = 0; i <= k; i++) {
99         ll x = a[k - 1][i] * (fibPower(n + 2, k - i) - 1);
100         if(i & 1) ans = cMOD(ans - x);
101         else ans = cMOD(ans + x);
102     }
103     for(int i = 1; i <= k; i++) {ans *= inverse(i); ans %= MOD;}
104     return ans;
105 }
106
107 int main(){
108     PreCalc();
109
110     ll k, l, r;
111     scanf("%lld %lld %lld",&k,&l,&r);
112     ll ans = query(r, k) - query(l-1, k);
113     cout << cMOD(ans) << endl;
114 }
115

```

7.2 Partition Function

```

1 // Calculates Partition Function(x) for all x <= n
2 // p(n) = number of distinct ways of representing n
3 // as sum of natural numbers (with order irrelevant).
4 // Complexity : O(n sqrt(n))
5
6 #define MAX 60005
7 #define MOD 1000000007
8 int p[MAX];
9 inline int Add(int a,int b){return (a+b)%MOD;}
10
11 int PartitionFunction(){
12     p[0] = 1;
13     for(int i = 1; i < MAX; i++){
14         int j = 1, r = 1;
15         while(i - (3*j*j - j) / 2 >= 0){
16             p[i] = Add(p[i], p[i - (3*j*j - j) / 2] * r);
17
18             if(i - (3*j*j + j) / 2 >= 0)
19                 p[i] = Add(p[i], p[i - (3*j*j + j) / 2] * r);
20
21             j += 1;
22             r *= -1;
23         }
24         if(p[i]<0) p[i]+=MOD;
25     }
26 }

```

7.3 Stirling Number (via NTT)

```

1 NTT ntt(mod);
2 vector<ll>v[MAX];
3
4 //Stirling1 (n,k) = co-eff of x^k in x*(x+1)*(x+2)*....(x+n-1)
5 int Stirling1(int n, int r) {
6     int nn = 1;
7     while(nn < n) nn <= 1;
8
9     for(int i = 0; i < n; ++i) {v[i].push_back(i); v[i].push_back(1);}
10    for(int i = n; i < nn; ++i) v[i].push_back(1);
11
12    for(int j = nn; j > 1; j >= 1) {
13        int hn = j >> 1;
14        for(int i = 0; i < hn; ++i) ntt.multiply(v[i], v[i + hn], v[i]);
15    }
16    return v[0][r];
17 }
18
19 NTT ntt(mod);
20 vector<ll>a,b,res;
21
22 //Stirling2 (n,k) = co-eff of x^k in product of polynomials A & B
23 //where A(i) = (-1)^i / i! and B(i) = i^n / i!
24 int Stirling2(int n, int r) {
25     a.resize(n+1); b.resize(n+1);
26     for(int i = 0; i <= n; i++){
27         a[i] = invfct[i];
28         if(i % 2 == 1) a[i] = mod - a[i];
29     }
30
31     for(int i = 0; i <= n; i++){
32         b[i] = bigMod(i, n, mod);
33         b[i] = (b[i] * invfct[i]) % mod;
34     }
35
36     NTT ntt(mod);
37     ntt.multiply(a,b,res);
38     return res[r];
39 }

```

7.4 nCr mod M

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 #define MAXP 10000000
5 #define ll long long int
6 #define pii pair<int,int>
7 const ll MOD = 10000000000LL;

```

```

8
9 namespace crt{
10     ll egcd(ll a, ll b, ll& x, ll& y){
11         if(!b) {y = 0, x = 1; return a;}
12         ll g = egcd(b, a % b, y, x);
13         y -= ((a / b) * x);
14         return g;
15     }
16
17     ll inv(ll a, ll m){
18         ll x, y;
19         egcd(a, m, x, y);
20         return (x + m) % m;
21     }
22
23     ll chinese_remainder(vector<ll> ar, vector<ll> mods){
24         ll x, y, res = 0, M = 1;
25         for(int i = 0; i < ar.size(); i++) M *= mods[i];
26         for(int i = 0; i < ar.size(); i++){
27             x = M / mods[i], y = inv(x, mods[i]);
28             res = (res + ((x * ar[i]) % M) * y) % M;
29         }
30         return res;
31     }
32 }
33
34 namespace binomial{
35     ll dp[MAXP];
36
37     void build(ll p, ll mod){
38         dp[0] = 1;
39         for(int i = 1; i < mod; i++) dp[i] = (ll)dp[i - 1] * ((i % p) ? i : 1) %
40 mod;
41     }
42
43     ll trailing(ll x, ll p){
44         ll res = 0;
45         while(x) {x /= p; res += x;}
46         return res;
47     }
48
49     ll expo(ll a, ll b, ll m){
50         ll res = 1;
51         while(b){
52             if (b & 1) res = res * a % m;
53             a = (a * a) % m, b >>= 1;
54         }
55         return res;
56     }
57
58     vector<pii> factorize(ll m){
59         vector<pii> factors;
60         for(ll i = 2; i * i <= m; i++){

```

```

61             int c = 0;
62             while (m % i == 0) {c++; m /= i;}
63             if (c) factors.push_back({i, c});
64         }
65         if(m > 1) factors.push_back({m, 1});
66         return factors;
67     }
68
69     ll spf(ll x, ll p, ll mod){
70         //Reference : http://e-maxx.ru/algo/modular\_factorial
71         //Complexity :  $O(\log_p(x))$ 
72         ll res = expo(dp[mod - 1], x / mod, mod);
73         if(x >= p) res = res * spf(x / p, p, mod) % mod;
74         return res * dp[x % mod] % mod;
75     }
76
77     ll C_mod_p_q(ll n, ll k, ll p, ll q){
78         using namespace crt;
79
80         if(k > n || k < 0) return 0;
81         if(n == k || k == 0) return 1;
82
83         ll mod = expo(p, q, 1e18);
84         assert(mod < MAXP);
85
86         ll t = trailing(n, p) - trailing(k, p) - trailing(n - k, p);
87         if(t >= q) return 0;
88
89         build(p, mod); //Complexity :  $O(mod)$ 
90         //This part can be optimized processing same mods at once.
91
92         ll res = spf(n, p, mod) * inv(spf(k, p, mod) * spf(n - k, p, mod) % mod,
93 mod) % mod;
94         res = (res * expo(p, t, mod)) % mod;
95         return res;
96     }
97
98     //Calculate  $C(n, k) \% M$ 
99     //Works when  $M = p_1^{q_1} * p_2^{q_2} * \dots * p_x^{q_x}$ , such that  $p_i^{q_i} < MAXP$ 
100     //Running time :  $O(MAXP)$ 
101     ll C(ll n, ll k, ll m){
102         if(k > n || m == 1 || k < 0) return 0;
103         if(n == k || k == 0) return 1;
104
105         vector<pii> factors = factorize(m);
106         vector<ll> ar, mods;
107         for(auto f : factors){
108             ll x = 1;
109             for (int j = 0; j < f.second; j++) x *= f.first;
110             mods.push_back(x), ar.push_back(C_mod_p_q(n, k, f.first, f.second));
111         }
112         return crt::chinese_remainder(ar, mods);
113     }

```

```

112 //check whether C(n,k) is greater than Lim
113 bool isGreater(ll n, ll k, ll Lim){
114     if(k < 0 || k > n) return false;
115
116     k = min(k, n - k);
117     ll u = n, v = 1;
118     pair <ll, long double> x = make_pair(1, 1);
119
120     for(int i = 0; i < k; i++, u--, v++){
121         x = make_pair(x.first * u, x.second * v);
122         x = make_pair(x.first / v, x.second / v);
123         if(x.first >= Lim || x.second >= (2.0 * Lim)) return true;
124     }
125     return false;
126 }
127 }
128 }

```

7.5 Gaussian Elimination

```

1 //Gaussian Elimination
2 //format : (a[0]*x[0]+a[1]*x[1] ... a[m-1]*x[m-1]) % k = a[m], where 0 <= ai < k
3 //number of solution : k^(number of free variable) = k^(n-rank)
4
5 int gcdExtended(int a, int b, int& x, int& y){
6     if(a==0) {x=0;y=1; return b;}
7     int x1,y1;
8     int gcd = gcdExtended(b%a,a,x1,y1);
9     x=y1-(b/a)*x1;
10    y=x1;
11    return gcd;
12 }
13
14 int modinverse(int x,int y) {int a,b; gcdExtended(x,y,a,b); return a;}
15
16 //n equations (n rows), m variables (m+1 columns)
17 void Gauss(int n,int m,int k){
18     int r,c;
19     for(r=0,c=0;r<n && c<m;c++){
20         for(int i=r+1;i<n;i++) if(abs(A[i][c])>abs(A[r][c])) swap(A[i],A[r]);
21         if(!A[r][c]) continue;
22         int s = modinverse(A[r][c],k);
23         for(int i=r+1;i<n;i++) if(A[i][c]){
24             int w = (s*A[i][c])%k;
25             for(int j=c;j<=m;j++){A[i][j]-=(A[r][j]*w)%k; A[i][j]%=k; if(A[i][j]
26 ]<0) A[i][j]+=k;}
27         }
28         r++;
29     }
30     //Rank = r
31     for(int i=r;i<n;i++) if(A[i][m]) return; //No solution

```

```

32 //Unique Solution for r variables
33 for(int i=r-1;i>=0;i--){
34     x[i]=A[i][m];
35     for(int j=i+1;j<m;j++) {x[i]-=(A[i][j]*x[j])%k; x[i]%=k; if(x[i]<0) x[i]
36 ]+=k;}
37
38     int inv=modinverse(A[i][i],k);
39     x[i]=(x[i]*inv)%k; if(x[i]<0) x[i]+=k;
40 }

```

7.6 Matrix

```

1 //Determinant + Inverse Tested : Codechef - CSTREE
2
3 #include<bits/stdc++.h>
4 using namespace std;
5 #define MAX 105
6 #define ll long long int
7 const ll MOD = 1e9 + 7;
8 const ll MOD2 = MOD * MOD * 3;
9
10 inline ll bigMod(ll a,ll b){
11     ll res=1;
12     while(b){
13         if(b&1) res=(res*a)%MOD;
14         a=(a*a)%MOD; b>>=1;
15     }
16     return res;
17 }
18
19 inline ll inv(ll n) {return bigMod(n,MOD-2);}
20 inline ll Mul(ll a,ll b) {return (a*b)%MOD;}
21 inline ll Div(ll a,ll b) {return Mul(a,inv(b));}
22
23 struct Matrix{
24     int row, col;
25     ll m[MAX][MAX];
26     Matrix() {memset(m,0,sizeof(m));}
27     void Set(int r,int c) {row = r; col = c;}
28     Matrix(int r,int c) {memset(m,0,sizeof(m)); Set(r,c);}
29     void normalize(){
30         for(int i=1; i<=row; i++){
31             for(int j=1; j<=col; j++){
32                 m[i][j] %= MOD;
33                 if(m[i][j] < 0) m[i][j] += MOD;
34             }
35         }
36     }
37 };
38
39 Matrix Multiply(Matrix A,Matrix B){

```

```

40 Matrix ans(A.row,B.col);
41
42 for(int i=1;i<=A.row;i++){
43     for(int j=1;j<=B.col;j++){
44         ans.m[i][j]=0;
45         ll sm = 0;
46         for(int k=1;k<=A.col;k++){
47             sm+=(A.m[i][k]*B.m[k][j]);
48             if(sm >= MOD2) sm -= MOD2;
49         }
50         ans.m[i][j] = sm % MOD;
51     }
52 }
53 return ans;
54 }
55
56 Matrix Power(Matrix mat,ll p){
57     Matrix res(mat.row , mat.col);
58     Matrix ans(mat.row , mat.col);
59
60     int n = ans.row;
61     for(int i=1;i<=n;i++){
62         for(int j=1;j<=n;j++){
63             ans.m[i][j]=0;
64             res.m[i][j]=mat.m[i][j];
65         }
66         ans.m[i][i]=1;
67     }
68
69     while(p){
70         if(p&1) ans=Multiply(ans,res);
71         res=Multiply(res,res);
72         p=p/2;
73     }
74     return ans;
75 }
76
77 ll Det(Matrix mat){
78     assert(mat.row == mat.col);
79     int n = mat.row;
80     mat.normalize();
81
82     ll ret = 1;
83     for(int i = 1; i <= n; i++){
84         for(int j = i + 1; j <= n; j++){
85             while(mat.m[j][i]){
86                 ll t = Div(mat.m[i][i], mat.m[j][i]);
87                 for(int k = i; k <= n; ++k){
88                     mat.m[i][k] -= Mul(mat.m[j][k] , t);
89                     if(mat.m[i][k] < 0) mat.m[i][k] += MOD;
90                     swap(mat.m[j][k], mat.m[i][k]);
91                 }
92             }
93             ret = MOD - ret;

```

```

93     }
94 }
95
96 if(mat.m[i][i] == 0) return 0;
97 ret = Mul(ret, mat.m[i][i]);
98 }
99
100 if(ret < 0) ret += MOD;
101 return ret;
102 }
103
104 ll Tmp[MAX<<1][MAX<<1];
105 Matrix Inverse(Matrix mat){
106     assert(mat.row == mat.col);
107     assert(Det(mat) != 0);
108
109     int n = mat.row;
110     mat.normalize();
111     for(int i=1;i<=n;i++){
112         for(int j=1;j<=n;j++) Tmp[i][j] = mat.m[i][j];
113         for(int j=n+1; j<=2*n; j++) Tmp[i][j] = 0;
114         Tmp[i][i+n] = 1;
115     }
116
117     for(int i=1; i<=n; i++){
118         assert(Tmp[i][i] != 0);
119
120         for(int j=1; j<=n; j++){
121             if(i == j) continue;
122             ll c = Div(Tmp[j][i], Tmp[i][i]);
123             for(int k=i; k<=2*n; k++){
124                 Tmp[j][k] = Tmp[j][k] - Mul(Tmp[i][k], c);
125                 if(Tmp[j][k] < 0) Tmp[j][k] += MOD;
126             }
127         }
128     }
129
130     Matrix Inv(n,n);
131     for(int i=1; i<=n; i++){
132         for(int j = 1; j <= n; j++){
133             Inv.m[i][j] = Div(Tmp[i][j+n], Tmp[i][i]);
134         }
135     }
136     return Inv;
137 }
138
139 //Freivalds algorithm : check whether AB = C
140 //Complexity : O(iteration * n^2)
141 int p[MAX],q[MAX],r[MAX];
142 bool Freivalds(Matrix A,Matrix B,Matrix C){
143     srand(time(NULL));
144     int n=A.row;
145     int iteration=15;

```

```

146 int Yes=0;
147 for(int i=1;i<=iteration;i++){
148     for(int i=1;i<=n;i++) r[i] = rand()%2;
149     for(int i=1;i<=n;i++) p[i] = q[i] = 0;
150
151     for(int i=1;i<=n;i++) for(int j=1;j<=n;j++){
152         p[i] += r[j] * A.m[i][j];
153     }
154     for(int i=1;i<=n;i++) for(int j=1;j<=n;j++){
155         q[i] += p[j] * B.m[i][j];
156     }
157     for(int i=1;i<=n;i++) for(int j=1;j<=n;j++){
158         q[i] -= r[j] * C.m[i][j];
159     }
160     bool All = true;
161     for(int i=1;i<=n;i++) if(q[i]) All=false;
162     if(All) Yes++;
163 }
164 return Yes == iteration;
165 }

```

7.7 Maximum Xor Subset

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define MAX 100005
4 #define ll long long int
5
6 // Gaussian Elimination Online
7 struct maxxor{
8     vector<ll> basis;
9     void init() {basis.clear();}
10
11     void add(ll x){
12         // Keep the basis sorted in increasing order
13         for(ll b : basis) x = min(x, x ^ b);
14         for(ll &b : basis) b = min(b, x ^ b);
15
16         if(x){
17             basis.push_back(x);
18             for(ll i = basis.size() - 1; i > 0 ; i--){
19                 if(basis[i] < basis[i - 1]) swap(basis[i], basis[i - 1]);
20                 else break;
21             }
22         }
23     }
24
25     //returns max subset xor
26 ll getMax(){
27     ll ans=0;
28     for(ll b : basis) ans ^= b;
29     return ans;
30 }

```

```

31 //returns max xor over (k ^ some subset)
32 ll getMax(ll k){
33     ll ans = k;
34     for(ll b : basis) ans = max(ans, ans ^ b);
35     return ans;
36 }
37
38 //returns k-th (0-indexed) smallest distinct subset xor
39 ll getKth(ll k){
40     ll ans = 0;
41     for(ll i = 0; i < basis.size(); i++){
42         if((k >> i) & 1) ans ^= basis[i];
43     }
44     return ans;
45 }
46 }ds;
47
48 // Gaussian Elimination Offline
49 ll a[MAX], n;
50 ll maxxor(){
51     int r = 0; ll ret = 0;
52     for(int c = 63; c >= 0; c--){
53         int idx = -1;
54         for(int i = r; i < n && idx < 0; i++){
55             if(a[i] >> c & 1) idx = i;
56         }
57         if(idx == -1) continue;
58         swap(a[r], a[idx]);
59         for(int i = 0; i < n; i++) if(i != r)
60             if(a[i] >> c & 1) a[i] ^= a[r];
61         r++;
62     }
63     for(int i = 0; i < n; i++) ret = max(ret, ret ^ a[i]);
64     return ret;
65 }

```

7.8 Simplex

```

1 /* Note: Simplex algorithm on augmented matrix a of dimension (m+1)x(n+1)
2  * returns 1 if feasible, 0 if not feasible, -1 if unbounded
3  * returns solution in b[] in original var order, max(f) in ret
4  * form: maximize sum_j(a_mj*x_j)-a_mn s.t. sum_j(a_ij*x_j)<=a_in
5  * To convert into standard form:
6  * 1. if exists equality constraint, then replace by both >= and <=
7  * 2. if variable x doesn't have nonnegativity constraint, then replace by
8  * difference of 2 variables like x1-x2, where x1>=0, x2>=0
9  * 3. for a>=b constraints, convert to -a<=-b
10 * note: watch out for -0.0 in the solution, algorithm may cycle
11 * EPS = 1e-7 may give wrong answer, 1e-10 is better */
12
13 #include <bits/stdc++.h>
14 using namespace std;
15 #define maxM 7

```

```

16 #define maxN 100007
17 #define INF 1000000007
18 #define EPS (1e-12)
19 #define PI acos(-1)
20
21 struct Simplex {
22     void pivot( int m,int n,double A[maxM][maxN+7],int *B,int *N,int r,int c ) {
23         int i,j;
24         swap( N[c],B[r] );
25         A[r][c] = 1/A[r][c];
26         for( j=0; j<=n; j++ ) if( j!=c ) A[r][j] *= A[r][c];
27         for( i=0; i<=m; i++ ) {
28             if( i!=r ) {
29                 for( j=0; j<=n; j++ ) if( j!=c ) A[i][j] -= A[i][c]*A[r][j];
30                 A[i][c] = -A[i][c]*A[r][c];
31             }
32         }
33     }
34
35     int feasible( int m,int n,double A[maxM][maxN+7],int *B,int *N ) {
36         int r,c,i;
37         double p,v;
38         while( 1 ) {
39             for( p=INF,i=0; i<m; i++ ) if( A[i][n]<p ) p = A[r=i][n];
40             if( p > -EPS ) return 1;
41             for( p=0,i=0; i<n; i++ ) if( A[r][i]<p ) p = A[r][c=i];
42             if( p > -EPS ) return 0;
43             p = A[r][n]/A[r][c];
44             for( i=r+1; i<m; i++ ) {
45                 if( A[i][c] > EPS ) {
46                     v = A[i][n]/A[i][c];
47                     if( v<p ) r=i,p=v;
48                 }
49             }
50             pivot( m,n,A,B,N,r,c );
51         }
52     }
53
54     // for all i<m,j<n : a[i][j]*b[i]<=a[i][n]
55     // maximize sum of a[m][j]-a[m][n] where j<n
56
57     int simplex( int m,int n,double A[maxM+7][maxN+7],double *b,double &Ret ) {
58         int B[maxM+7],N[maxN+7],r,c,i;
59         double p,v;
60         for( i=0; i<n; i++ ) N[i] = i;
61         for( i=0; i<m; i++ ) B[i] = n+i;
62         if( !feasible( m,n,A,B,N ) ) return 0;
63         while( 1 ) {
64             for( p=0,i=0; i<n; i++ ) if( A[m][i] > p ) p = A[m][c=i];
65             if( p<EPS ) {
66                 for( i=0; i<n; i++ ) if( N[i]<n ) b[N[i]] = 0;
67                 for( i=0; i<m; i++ ) if( B[i]<n ) b[B[i]] = A[i][n];
68                 Ret = -A[m][n];

```

```

69         return 1;
70     }
71     for( p=INF,i=0; i<m; i++ ) {
72         if( A[i][c] > EPS ) {
73             v = A[i][n]/A[i][c];
74             if( v<p ) p = v,r = i;
75         }
76     }
77     if( p==INF ) return -1;
78     pivot( m,n,A,B,N,r,c );
79 }
80 }
81 };

```

7.9 Thomas Algorithm

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define LD long double
4
5 // Equation of the form: (x_prev * l) + (x_cur * p) + (x_next * r) = rhs
6 struct equation{
7     LD l, p, r, rhs;
8
9     equation(){}
10    equation(LD l, LD p, LD r, LD rhs = 0.0):
11        l(l), p(p), r(r), rhs(rhs){}
12 };
13
14 // Thomas algorithm to solve tri-digonal system of equations in O(n)
15 vector<LD> thomas_algorithm(int n, vector<struct equation> ar){
16     ar[0].r = ar[0].r / ar[0].p;
17     ar[0].rhs = ar[0].rhs / ar[0].p;
18
19     for (int i = 1; i < n; i++){
20         LD v = 1.0 / (ar[i].p - ar[i].l * ar[i - 1].r);
21         ar[i].r = ar[i].r * v;
22         ar[i].rhs = (ar[i].rhs - ar[i].l * ar[i - 1].rhs) * v;
23     }
24     for (int i = n - 2; i >= 0; i--) ar[i].rhs = ar[i].rhs - ar[i].r * ar[i + 1].rhs;
25
26     vector<LD> res;
27     for (int i = 0; i < n; i++) res.push_back(ar[i].rhs);
28     return res;
29 }

```

7.10 Baby Step Giant Step (Any modulo)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int egcd(int a, int b, int& x, int& y){
5     if (!b) {y = 0, x = 1; return a;}
6     int g = egcd(b, a % b, y, x);
7     y -= (a / b) * x;
8     return g;
9 }
10
11 int discrete_log(int g, int h, int p){
12     // returns smallest x such that (g^x) % p = h, -1 if none exists
13     // function returns x, the discrete log of h with respect to g modulo p
14
15     if (h >= p) return -1;
16     if ((g % p) == 0){
17         if (h == 1) return 0; // return -1 if strictly positive integer solution
18         is required
19         else return -1;
20     }
21
22     int i, c, x, y, z, r, m, counter = 0;
23     long long v = 1, d = 1, mul = 1, temp = 1 % p;
24
25     for (int i = 0; i < 100; i++){
26         if (temp == h) return i;
27         temp = (temp * g) % p;
28     }
29
30     while ((v = __gcd(g, p)) > 1){
31         if (h % v) return -1;
32         h /= v, p /= v;
33         d = (d * (g / v)) % p;
34         counter++;
35     }
36
37     m = ceil(sqrt(p)); // try sqrt1()
38     unordered_map <int, int> mp;
39
40     for (i = 0; i < m; i++){
41         if(!mp[mul]) mp[mul] = i + 1;
42         mul = (mul * g) % p;
43     }
44
45     for (i = 0; i < m; i++){
46         z = egcd(d, p, x, y);
47         c = p / z;
48         r = (((long long)x * h) / z) % p + p) % p;
49         if (mp[r]) return ((i * m) + mp[r] + counter - 1);
50         d = (d * mul) % p;
51     }
52     return -1;
53 }

```

```

53
54 int main(){
55     int g, h, p, res;
56     scanf("%d %d %d", &g, &p, &h);
57
58     res = discrete_log(g, h % p, p);
59     if (res == -1) puts("No Solution");
60     else printf("%d\n", res);
61 }

```

7.11 Baby Step Giant Step (Only Prime Modulo)

```

1 //Tested : LightOJ 1325 - Distributing Chocolates
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 #define ll long long int
6
7 //Shanks Baby-Step Giant-Step (Faster)
8 //a^x = b (mod m)
9 ll egcd(ll a,ll b,ll &x,ll &y){
10     if(b == 0) {x = 1; y = 0; return a;}
11     ll g = egcd(b,a%b,y,x);
12     y -= a / b * x;
13     return g;
14 }
15
16 ll InvMod(ll a,ll m){
17     ll x,y;
18     if(egcd(a,m,x,y) == 1) return (x + m) % m;
19     return -1;
20 }
21
22 unordered_map<ll, ll> H;
23 ll baby_step_giant_step(ll a,ll b,ll m){
24     H.clear();
25     int k = (int) sqrt(m)+1;
26
27     //Baby step
28     ll aa = 1;
29     for(int j=0;j<k;j++){
30         if(!H.count(aa)) H[aa] = j;
31         aa = aa * a % m;
32     }
33
34     //Giant step
35     aa = InvMod(a,m);
36     for(int i=0;i<k;i++){
37         if(H.count(b)) return i*k + H[b];
38         b = b * aa % m;
39     }
40     return -1;

```

41 }

7.12 Continued Fraction

```

1  /*
2  * Description: Given N and a real number x >= 0
3  * finds the closest rational approximation p/q with p, q <= N.
4  * It will obey |p/q - x| <= 1 / qN.
5  * For consecutive convergents, p_{k+1} * q_k - q_{k+1} * p_k = (-1)^k
6  * (p_k / q_k alternates between > x and < x)
7  * If x is rational, y eventually becomes \infy;
8  * if x is the root of a degree 2 polynomial the a's eventually become cyclic.
9  * Time: O(log N)
10 */
11
12 #include <bits/stdc++.h>
13 using namespace std;
14
15 #define d double // for N ~ 1e7; long double for N ~ 1e9
16 #define ll long long int
17
18 pair<ll, ll> approximate(d x, ll N) {
19     ll LP = 0, LQ = 1;
20     ll P = 1, Q = 0;
21     ll inf = LLONG_MAX;
22     d y = x;
23
24     while(true){
25         ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf);
26         ll a = (ll) floor(y);
27         ll b = min(a, lim);
28         ll NP = b*P + LP, NQ = b*Q + LQ;
29
30         if(a > b){
31             // If b > a/2, we have a semi-convergent that gives us a
32             // better approximation; if b = a/2, we *may* have one.
33             // Return {P, Q} here for a more canonical approximation.
34             return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
35                 make_pair(NP, NQ) : make_pair(P, Q);
36         }
37
38         if(abs(y = 1/(y - (d) a)) > 3*N) return {NP, NQ};
39         LP = P; P = NP;
40         LQ = Q; Q = NQ;
41     }
42 }
43
44 int main(){
45     auto ret = approximate(0.53846153846, 14);
46     cout<<ret.first<<" "<<ret.second<<endl;
47 }

```

7.13 Diophantine Equation

```

1  //Tested : LightOJ 1306 - Solutions to an Equation
2
3  #include <bits/stdc++.h>
4  using namespace std;
5  #define ll long long int
6
7  // Diophantine equation : a * x + b * y = gcd(a, b)
8  // egcd computes one solution (x, y) for gcd(a,b) = g
9  // Note: computed value g can be negative.
10 // Given one solution (x0, y0), other solutions have form :
11 // xk = x0 + k * b / g and yk = y0 - k * a / g
12
13 ll egcd(ll a, ll b, ll &x, ll &y){
14     if(a == 0) {x = 0; y = 1; return b;}
15     ll x1, y1;
16     ll gcd = egcd(b%a, a, x1, y1);
17     x = y1 - (b / a) * x1; y = x1;
18     return gcd;
19 }
20
21 inline ll Floor(ll p, ll q) {return p>0 ? p/q : p/q - !!(p%q);}
22 inline ll Ceil(ll p, ll q) {return p<0 ? p/q : p/q + !!(p%q);}
23
24 // Number of solutions of Diophantine Eqn : Ax + By = C
25 // A,B,C,x,y integers and X1 <= x <= X2 and Y1 <= y <= Y2
26 inline ll solve(ll A, ll B, ll C, ll X1, ll X2, ll Y1, ll Y2){
27     if(A<0) {A = -A; B = -B; C = -C;}
28     ll G = __gcd(A, B);
29     if(G && C%G) return 0;
30
31     if(A == 0 && B == 0) return (C == 0) ? (X2 - X1 + 1) * (Y2 - Y1 + 1) : 0;
32     if(A == 0) return (Y1 <= C/B && C/B <= Y2) ? (X2 - X1 + 1) : 0;
33     if(B == 0) return (X1 <= C/A && C/A <= X2) ? (Y2 - Y1 + 1) : 0;
34
35     ll x, y;
36     egcd(A, B, x, y);
37     x = x * (C/G); y = y * (C/G);
38
39     ll newX1 = C - B * Y1, newX2 = C - B * Y2;
40     if(newX1 > newX2) swap(newX1, newX2);
41     newX2 = Floor(newX2, A); newX1 = Ceil(newX1, A);
42
43     if(X1 > newX2) return 0;
44     if(X2 < newX1) return 0;
45     X1 = max(X1, newX1); X2 = min(X2, newX2);
46
47     ll div = abs(B/G);
48     if(x < X1) return (X2 - x) / div - (X1 - 1 - x) / div;
49     if(x > X2) return (x - X1) / div - (x - X2 - 1) / div;
50     return 1 + (x - X1) / div + (X2 - x) / div;
51 }

```

7.14 Fast Fibonacci

```

1 // F(n-1) * F(n+1) - F(n) * F(n) = (-1)^n
2 // F(n+k) = F(k) * F(n+1) + F(k-1) * F(n)
3 // gcd(F(m), F(n)) = F(gcd(m,n))
4
5 #define ll long long int
6
7 pair<ll, ll> fib(ll n) {
8     if(n == 0) return {0, 1};
9     ll x, y;
10    if(n & 1) {
11        tie(y, x) = fib(n - 1);
12        return {x, (y + x) % MOD};
13    }
14    else{
15        tie(x, y) = fib(n >> 1);
16        return {(x * y + x * (y - x + MOD)) % MOD, (x * x + y * y) % MOD};
17    }
18 }

```

7.15 Faulhaber's Formua

```

1 //Faulhaber's Formua
2 //faulhaber(n,k) = 1^k + 2^k + 3^k + ... + n^k modulo ANY number
3 //Complexity : O(k^2)
4
5 #include <bits/stdc++.h>
6 using namespace std;
7 #define MAX 1005
8 #define ll long long int
9 const ll MOD = (1LL<<32);
10
11 ll S[MAX][MAX];
12
13 void Stirling2(){
14     S[0][0] = 1;
15     for(int i = 1; i < MAX; i++){
16         S[i][0] = 0;
17         for(int j = 1; j <= i; j++){
18             S[i][j] = ((S[i - 1][j] * j) % MOD + S[i - 1][j - 1]) % MOD;
19         }
20     }
21 }
22
23 ll faulhaber(ll n, int k){
24     if (!k) return n % MOD;
25
26     ll res = 0;
27     for(int j = 0; j <= k; j++){
28         ll Div = j + 1;

```

```

29         ll Mul = 1;
30         ll r = n % Div + 1;
31         if(r == Div) r = 0;
32
33         for(int x = 0; x <= j; x++){
34             if(x == r) Mul *= (n - x + 1) / Div;
35             else Mul *= (n - x + 1);
36             Mul %= MOD;
37         }
38
39         res += (S[k][j] * Mul) % MOD;
40         if(res >= MOD) res -= MOD;
41         if(res < 0) res += MOD;
42     }
43     return res;
44 }

```

7.16 Fraction Binary Search

```

1 /*
2  * Given f and N, finds the smallest fraction p/q in [0, 1]
3  * such that f(p/q) is true, and p, q <= N.
4  * May throw an exception from f if it finds an exact solution,
5  * in which case N can be removed.
6  * Time: O(log(N))
7  */
8
9 #include <bits/stdc++.h>
10 using namespace std;
11 #define ll long long int
12
13 struct Frac{ ll p, q;};
14 template<class F> Frac fracBS(F f, ll N){
15     bool dir = 1, A = 1, B = 1;
16     Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
17     assert(!f(lo));
18     assert(f(hi));
19
20     while (A || B){
21         ll adv = 0, step = 1; // move hi if dir, else lo
22         for (int si = 0; step; (step *= 2) >= si) {
23             adv += step;
24             Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
25             if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
26                 adv -= step; si = 2;
27             }
28         }
29         hi.p += lo.p * adv;
30         hi.q += lo.q * adv;
31         dir = !dir;
32         swap(lo, hi);
33         A = B; B = !!adv;

```

```

34 }
35 return dir ? hi : lo;
36 }
37
38 int main(){
39     Frac ret = fracBS[] (Frac f) {return 3*f.p >= f.q;}, 10); // {1,3}
40     cout<<ret.p<<" "<<ret.q<<endl;
41 }

```

7.17 General CRT

```

1 //Tested : Kattis-generalchineseremainder
2
3 #include<bits/stdc++.h>
4 using namespace std;
5 #define ll long long int
6 #define pll pair<ll, ll>
7
8 ll egcd(ll a, ll b, ll& x, ll& y){
9     if(!b) {y = 0, x = 1; return a;}
10    ll g = egcd(b, a % b, y, x);
11    y -= ((a / b) * x);
12    return g;
13 }
14
15 pll GeneralCRT(pll a, pll b){
16     if(a.second < b.second) swap(a, b);
17     ll x, y; egcd(a.second, b.second, x, y);
18     ll g = a.second * x + b.second * y;
19     ll l = a.second / g * b.second;
20     if((b.first - a.first) % g) return {-1, -1}; // No Solution
21
22     ll c = (b.first - a.first) % b.second;
23     c = (c * x) % b.second;
24     c = c / g * a.second;
25     c += a.first;
26     if(c < 0) c += l;
27     return {c, l};
28 }

```

7.18 Geometric Sum

```

1 //Computes the sum of a^1 + a^2 .... + a^n
2
3 ll GeoSum(ll a, ll n){
4     ll sz = 0;
5     ll ret = 0;
6     ll mul = 1;
7     int MSB = 63 - __builtin_clzll(n);
8

```

```

9     while(MSB >= 0){
10         ret = ret * (1 + mul); mul = (mul * mul) % MOD; sz <= 1;
11         if( (n >> MSB) & 1) {mul = (mul * a) % MOD; ret += mul; sz++;}
12         ret %= MOD; MSB--;
13     }
14     return ret;
15 }

```

7.19 Integration (Romberg)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define LD long double
4 LD eps = 1e-8;
5
6 LD f(LD x) {return sin(x);}
7
8 LD romberg(LD a, LD b){
9     vector<LD>t;
10    LD h = b-a;
11    LD last, curr;
12    int k = 1;
13    int i = 1;
14    t.push_back(h*(f(a)+f(b))/2);
15
16    while(true){
17        last = t.back(); curr=0;
18        LD x = a+h/2;
19        for(int j=0;j<k;j++) curr += f(x), x += h;
20        curr = (t[0] + h*curr) / 2;
21        LD k1 = 4.0/3.0, k2 = 1.0/3.0;
22
23        for(int j=0; j<i; j++){
24            LD temp = k1 * curr - k2 * t[j];
25            t[j] = curr; curr = temp;
26            k2 /= 4*k1 - k2;
27            k1 = k2 + 1;
28        }
29        t.push_back(curr);
30        k *= 2; h /= 2; i++;
31        if(fabs(last-curr) < eps) break;
32    }
33    return t.back();
34 }

```

7.20 Integration (Simpson)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long double LD;

```

```

4 const LD eps = 1e-6;
5
6 LD f(LD x) {return sin(x);}
7
8 inline LD simpson(LD fl, LD fr, LD fmid, LD l, LD r){
9     return (fl + fr + 4.0 * fmid) * (r-l) / 6.0;
10 }
11
12 LD Solve(LD slr, LD fl, LD fr, LD fmid, LD l, LD r){
13     LD mid = (l + r) / 2;
14     LD fml = f((l + mid) / 2);
15     LD fmr = f((mid + r) / 2);
16     LD slm = simpson(fl, fmid, fml, l, mid);
17     LD smr = simpson(fmid, fr, fmr, mid, r);
18     if( fabs(slr-slm-smr) < eps) return slm + smr;
19     return Solve(slm, fl, fmid, fml, l, mid) + Solve(smr, fmid, fr, fmr, mid, r);
20 }
21
22 LD integrate(LD l, LD r){
23     LD mid = (l+r) / 2;
24     LD fl = f(l);
25     LD fr = f(r);
26     LD fmid = f(mid);
27     LD slr = simpson(fl, fr, fmid, l, r);
28     return Solve(slr, fl, fr, fmid, l, r);
29 }

```

7.21 Interpolation (Lagrange - Optimized)

```

1 //Lagrange Polynomial
2 //lagrange(n,k) = 1^k + 2^k + 3^k + ... + n^k modulo Prime
3 //Complexity : O(k log (MOD))
4
5 #include <bits/stdc++.h>
6 using namespace std;
7 #define MAX 1000010          // Upper Limit of k
8 #define MOD 1000000007
9 #define ll long long int
10
11 namespace lgr{
12     short factor[MAX];
13     int P[MAX], S[MAX], ar[MAX], inv[MAX];
14
15     inline int Add(int a, int b) {return a + b >= MOD ? a + b - MOD : a + b;}
16     inline int Sub(int a, int b) {return a - b < 0 ? a - b + MOD : a - b;}
17
18     inline int bigMod(ll a, ll b){
19         ll res = 1;
20         while(b){
21             if(b&1) res = (res * a) % MOD;
22             a = (a * a) % MOD; b >>= 1;
23         }

```

```

24     return res;
25 }
26
27 int lagrange(ll n, int k){
28     if(!k) return (n % MOD);
29
30     if(!inv[0]){
31         int x = 1;
32         for(int i = 2; i < MAX; i++) x = (ll)x * i % MOD;
33         inv[MAX-1] = bigMod(x, MOD-2);
34         for(int i = MAX-2; i >= 0; i--) inv[i] = (ll)inv[i+1] * (i+1) % MOD;
35         for(int i = 0; i < MAX; i++) factor[i] = 0;
36         for(int i = 4; i < MAX; i+=2) factor[i] = 2;
37         for(int i = 3; i * i < MAX; i+=2){
38             if(!factor[i])
39                 for(int j = i*i; j < MAX; j += i + i) factor[j] = i;
40         }
41     }
42
43     k++;
44     ar[0] = 0, ar[1] = 1;
45     for(int i=2; i<=k; i++){
46         if(!factor[i]) ar[i] = bigMod(i, k-1);
47         else ar[i] = (ll) ar[factor[i]] * ar[i/factor[i]] % MOD;
48     }
49     for(int i = 1; i <= k; i++) ar[i] = Add(ar[i], ar[i-1]);
50     if(n <= k) return ar[n];
51
52     P[0]=1, S[k]=1;
53     for(int i = 1; i <= k; i++) P[i] = (n-i+1) % MOD * P[i-1] % MOD;
54     for(int i = k-1; i >= 0; i--) S[i] = (n-i-1) % MOD * S[i+1] % MOD;
55
56     int res = 0;
57     for (int i = 0; i <= k; i++){
58         int x = (ll) ar[i] * P[i] % MOD * S[i] % MOD * inv[k - i] % MOD * inv
59 [i] % MOD;
60         if((k-i) & 1) res = Sub(res, x);
61         else res = Add(res, x);
62     }
63     return res;
64 }
65
66 int main(){
67     ll n; int k;
68     scanf("%lld %d", &n, &k);
69     printf("%d\n", lgr::lagrange(n, k));
70 }

```

7.22 Interpolation (Lagrange)

```

1 #define ll long long int

```

```

2 #define MAXN 1000005
3 #define MOD 1000000007
4
5 ll bigMod(ll n, ll r) {
6     if (r==0) return 1LL;
7     ll ret = bigMod(n, r/2);
8     ret = (ret * ret)% MOD;
9     if (r %2 ) ret = (ret * n) % MOD;
10    return ret;
11 }
12
13 ll Point[MAXN];
14 ll Fact[MAXN];
15
16 // Calculate first k + 1 points (0 to k) on the polynomial
17 // where k = degree of the polynomial
18 // Then find f(x) for any x using interpolation in O(n log(MOD))
19
20 ll interpolate(int n,ll x) {
21     if(x <= n) return Point[x];
22
23     ll num = 1;
24     for(int i=0; i<=n; i++) num=(num * (x-i)) % MOD;
25
26     ll ret = 0;
27     for(int i=0; i<=n; i++) {
28         ll nn = (num * bigMod(x-i, MOD-2)) % MOD;
29         ll dd = (Fact[n-i] * Fact[i]) % MOD;
30
31         if((n-i) & 1) dd = MOD -dd;
32         nn = (Point[i] * nn) % MOD;
33         ret = (ret + nn * bigMod(dd, MOD-2))%MOD;
34     }
35     return ret;
36 }

```

7.23 Interpolation (Newton)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define ll long long int
4 const int maxn = 5005;
5 const int mod = 100019;
6
7 inline ll add(ll x, ll y) {x += y; return x >= mod ? x - mod : x;}
8 inline ll sub(ll x, ll y) {x -= y; return x < 0 ? x + mod : x;}
9 inline ll mul(ll x, ll y) {return (x * y) % mod;}
10 inline ll bigMod(ll x, ll y){
11     ll res = 1;
12     while(y){
13         if(y & 1) res = mul(res, x);
14         y >>= 1; x = mul(x, x);

```

```

15     }
16     return res;
17 }
18 inline ll inv(ll n) {return bigMod(n, mod-2);}
19 inline ll Div(ll a, ll b) {return mul(a, inv(b));}
20
21 struct Newton{
22     ll dp[maxn][maxn];
23     vector<ll> coX, coY;
24     Newton() {coX.clear(); coY.clear();}
25
26     void Insert(ll x, ll y){
27         coX.push_back(x);
28         coY.push_back(y);
29         int v = coX.size() - 1;
30
31         dp[v][v] = y;
32         for(int u=v-1; u>=0; u--){
33             dp[u][v] = sub(dp[u+1][v], dp[u][v-1]);
34             dp[u][v] = Div(dp[u][v], sub(coX[v], coX[u]));
35         }
36     }
37
38     ll Query(ll x){
39         ll pre = 1;
40         ll ans = 0;
41         for(int i=0; i<coX.size(); i++){
42             ans = add(ans, mul(pre, dp[0][i]));
43             pre = mul(pre, sub(x, coX[i]));
44         }
45         return ans;
46     }
47 };

```

7.24 Lehmer Prime Counting

```

1 //Tested : CF 665F - Four Divisors
2
3 /* Let p(j) = j-th prime number.
4 dp(n,j) = Count of k (1 <= k <= n and all prime div of k >= p(j))
5 Then we have : dp(n,1) = n and dp(n,j+1) = dp(n,j) - dp(n/p(j),j)
6 Let p(k) be the smallest prime > sqrt(n), n(n) = dp(n,k) + k - 1
7 Complexity : O(n ^ (2/3)) */
8
9 #include <bits/stdc++.h>
10 using namespace std;
11 #define ll long long int
12
13 const int LIM = 250;
14 const int MAXP = 10000005;
15
16 vector<int> P;

```

```

17 int prec[MAXP];
18 bool prime[MAXP];
19
20 void build(){
21     prime[2] = true;
22     for(int i = 3; i < MAXP; i += 2) prime[i] = true;
23
24     for(int i = 3; i*i < MAXP; i += 2){
25         if(!prime[i]) continue;
26         for(int j = i*i; j < MAXP; j += i+i) prime[j] = false;
27     }
28
29     for(int i=2; i<MAXP; i++) if(prime[i]) P.push_back(i);
30     for(int i=1; i<MAXP; i++) prec[i] = prec[i-1] + prime[i];
31 }
32
33 ll dp[LIM*LIM][LIM];
34 ll rec(ll N, int K){
35     if(N <= 1 || K < 0) return 0;
36     if(N <= P[K]) return N-1;
37     if(N < MAXP && P[K] * 1LL * P[K] > N) return N-1 - prec[N] + prec[P[K]];
38
39     bool ok = N < LIM * LIM && K < LIM;
40     if (ok && dp[N][K]) return dp[N][K];
41
42     ll ret = N/P[K] - rec(N/P[K], K-1) + rec(N, K-1);
43
44     if(ok) dp[N][K] = ret;
45     return ret;
46 }
47
48 ll count_primes(ll N) {
49     if(N < MAXP) return prec[N];
50     int K = prec[(int)sqrt(N) + 1];
51     return N-1 - rec(N, K) + prec[P[K]];
52 }

```

7.25 Maximum Divisors in Range

```

1 //Finds x such that x <= n and has maximum number of divisors
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 #define ULL unsigned long long int
6
7 ULL n, res, idx;
8 int p, primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53,
9     59, 61, 67, 71};
10
11 ULL mul(ULL a, ULL b){
12     ULL res = 0;
13     while (b){

```

```

14         if (b & 1LL) res = (res + a);
15         if (res > n) return 0;
16         a = (a << 1LL);
17         b >>= 1LL;
18     }
19     return res;
20 }
21 void backtrack(int i, int lim, ULL val, ULL r){
22     if ((r > res) || (r == res && val < idx)) res = r, idx = val;
23     if (i == p) return;
24
25     ULL x = val;
26     for (int d = 1; d <= lim; d++){
27         x = mul(x, primes[i]);
28         if (x == 0) return;
29         backtrack(i + 1, d, x, r * (d + 1));
30     }
31 }
32
33 int main(){
34     /* Tested for <= 10^18 */
35     p = sizeof(primes) / sizeof(int);
36     res = 0;
37     scanf("%llu", &n);
38     backtrack(0, 100, 1, 1);
39     printf("%llu = %llu\n", idx, res);
40 }

```

7.26 Pollard's Rho

```

1 //Complexity : O(sqrt(Smallest Prime Factor of n)) = O(n^(1/4))
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define ll long long int
5 #define pii pair<ll,int>
6
7 ll Mul(ll a,ll b,ll Mod){
8     ll Ans=0;
9     while(b){
10         if(b&1) {Ans+=a; if(Ans>=Mod) Ans-=Mod;}
11         a+=a; if(a>=Mod) a-=Mod;
12         b>>=1;
13     }
14     return Ans;
15 }
16
17 ll bigMod(ll n,ll r,ll Mod){
18     if(r==0) return 1LL;
19     ll ret=bigMod(n,r/2,Mod);
20     ret=Mul(ret,ret,Mod);
21     if(r%2==1) ret=Mul(ret,n,Mod);

```

```

22     return ret;
23 }
24
25 //Miller-Rabin
26 bool witness(ll wit,ll n){
27     if(wit>=n) return false;
28
29     int s=0; ll t=n-1;
30     while(t%2==0) s++,t/=2;
31
32     wit=bigMod(wit,t,n);
33     if(wit==1 || wit==n-1) return false;
34
35     for(int i=1;i<s;i++){
36         wit=mul(wit,wit,n);
37         if(wit==1) return true;
38         if(wit==n-1) return false;
39     }
40     return true;
41 }
42
43 //Is n prime?
44 bool miller(ll n){
45     if(n==2) return true;
46     if(n%2==0 || n<2) return false;
47     if(witness(2,n) || witness(7,n) || witness(61,n)) return false;
48     return true;
49 }
50
51 // Pollard's Rho
52 // a must not equal 0 or -2.
53 // returns a divisor, a proper one when succeeded, equal to n if failed
54 // in case of failure, change a
55 ll rho(ll n,ll a) {
56     auto f=[&](ll x) {return (mul(x,x,n)+a)%n; };
57     ll x=2,y=2;
58     for(int i=1;;i++){
59         x=f(x); y=f(f(y));
60         ll d=__gcd(n,abs(x-y));
61         if(d!=1) return d;
62     }
63     return n;
64 }
65
66 ll get_factor(ll n){
67     if(n%2==0) return 2;
68     if(n%3==0) return 3;
69     if(n%5==0) return 5;
70     while(true){
71         ll a=2+rand()%100;
72         ll d=rho(n,a);
73         if(d!=n) return d;
74     }

```

```

75     return n;
76 }
77
78 void factorize(ll n,vector<ll> &x) {
79     if(n==1) return;
80     else if(miller(n)) x.push_back(n);
81     else{
82         ll d=get_factor(n);
83         factorize(d,x);
84         factorize(n/d,x);
85     }
86 }
87
88 vector<ll>factorize(ll n) {vector<ll>x; factorize(n, x); return x;}
89
90 vector<pii>Factors;
91 vector<ll>Divisors;
92 void findDiv(int pos,ll val){
93     if(pos<0) {Divisors.push_back(val); return;}
94     ll Now=1;
95     for(int i=0;i<=Factors[pos].second;i++){
96         findDiv(pos-1,val*Now);
97         Now=Now*Factors[pos].first;
98     }
99 }
100
101 void findAllDiv(ll n){
102     vector<ll>now=factorize(n);
103     sort(now.begin(),now.end());
104
105     Factors.clear();
106     Divisors.clear();
107     int Count=1;
108     for(int i=1;i<now.size();i++){
109         if(now[i]==now[i-1]) Count++;
110         else {Factors.push_back({now[i-1],Count}); Count=1;}
111     }
112     Factors.push_back({now.back(),Count});
113     findDiv(Factors.size()-1,1);
114 }

```

7.27 Primitive root for Prime

```

1 int bigMod(int n, int k, int mod){
2     if(k == 0) return 1;
3     long long re = bigMod(n, (k / 2), mod);
4     re = (re * re) % mod;
5     if(k % 2) re = (re * n) % mod;
6     return re;
7 }
8
9 int phi(int n){

```

```

10 //Find phi(n) in feasible way
11 //return n-1 if n prime
12 }
13
14 int primitive_root(int p){
15     vector<int> factor;
16     int php = phi(p);
17
18     int tmp = php;
19     for(int i = 2; i * i <= tmp; i++){
20         if(tmp % i == 0){
21             factor.push_back(i);
22             while (tmp % i == 0) tmp /= i;
23         }
24     }
25     if(tmp != 1) factor.push_back(tmp);
26
27     for(int root=1; ; root++){
28         bool flag = true;
29         for(int i = 0; i < factor.size(); ++i){
30             if(bigMod(root, php / factor[i], p) == 1){
31                 flag = false;
32                 break;
33             }
34         }
35         if (flag) return root;
36     }
37 }

```

7.28 Range Congruence

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define ll long long int
4
5 // finds minimum x such that L <= (A * x) % P <= R
6 // constraint : 0 < A < P and 0 <= L <= R < P
7 // returns -1 if no solution exists
8
9 ll cdiv(ll x, ll y) {return (x+y-1)/y;}
10 ll get(ll A, ll P, ll L, ll R) {
11     if(A == 0) return L == 0 ? 0 : -1;
12     ll c = cdiv(L, A);
13     if(A*c <= R) return c;
14     ll B = P % A;
15
16     // P = k * A + B, L <= A * (x - K * y) - B * y <= R
17     // => -R <= (B * y) % A <= -L
18     auto y = get(B, A, A - R % A, A - L % A);
19     return y == -1 ? y : cdiv(L + B * y, A) + P / A * y;
20 }

```

7.29 Schreier-Sims Group Test

```

1 // time complexity : O(n^2 log^3 |G| + t n log |G|)
2 // working space : O(n^2 log |G| + tn)
3 // t : number of generator
4 #include <bits/stdc++.h>
5 using namespace std;
6 #define ll long long int
7
8 namespace SchreierSimsAlgorithm{
9     typedef vector<int> Permu;
10     Permu inv(const Permu& p){
11         Permu ret(p.size());
12         for(int i = 0; i < p.size(); i++) ret[p[i]] = i;
13         return ret;
14     }
15
16     Permu operator*( const Permu& a, const Permu& b ){
17         Permu ret(a.size());
18         for(int i = 0 ; i < a.size(); i++) ret[i] = b[a[i]];
19         return ret;
20     }
21
22     int n, m;
23     typedef vector<int> Table;
24     typedef pair<int,int> pii;
25     typedef vector<Permu> Bucket;
26     vector<Table> lookup;
27     vector<Bucket> bkts, bktsInv;
28
29     int fastFilter(const Permu &g, bool addToG = 1){
30         Permu p;
31         n = bkts.size();
32         for(int i = 0; i < n ; i++){
33             int res = lookup[i][p[i]];
34             if(res == -1 ){
35                 if(addToG){
36                     bkts[i].push_back(p);
37                     bktsInv[i].push_back(inv(p));
38                     lookup[i][p[i]] = bkts[i].size()-1;
39                 }
40                 return i;
41             }
42             p = p * bktsInv[i][res];
43         }
44         return -1;
45     }
46
47     ll calcTotalSize(){
48         ll ret = 1;
49         for(int i = 0; i < n; i++) ret *= bkts[i].size();
50         return ret;
51     }

```

```

52 bool inGroup(Permu &g) {return fastFilter(g, false) == -1;}
53
54 void solve(Bucket &gen, int _n){
55     n = _n, m = gen.size(); // m perm[0..n-1]s
56     { //clear all
57         bkts.clear();
58         bktsInv.clear();
59         lookup.clear();
60     }
61     for(int i = 0; i < n ; i++){
62         lookup[i].resize(n);
63         fill(lookup[i].begin(), lookup[i].end(), -1);
64     }
65     Permu id( n );
66     for(int i = 0; i < n ; i++) id[i] = i;
67     for(int i = 0; i < n ; i++){
68         bkts[i].push_back(id);
69         bktsInv[i].push_back(id);
70         lookup[i][i] = 0;
71     }
72     for(int i = 0; i < m ; i++) fastFilter(gen[i]);
73
74     queue< pair<pii,pii> > toUpd;
75     for(int i = 0; i < n ; i++)
76         for(int j = i; j < n ; j++)
77             for(int k = 0; k < bkts[i].size(); k++)
78                 for(int l = 0; l < bkts[j].size(); l++)
79                     toUpd.push({pii(i,k), pii(j,l)});
80
81     while(!toUpd.empty()){
82         pii a = toUpd.front().first;
83         pii b = toUpd.front().second;
84         toUpd.pop();
85         int res = fastFilter(bkts[a.first][a.second] * bkts[b.first][b.second]);
86     }
87
88     if(res == -1) continue;
89     pii newPair(res, (int)bkts[res].size() - 1);
90     for(int i = 0; i < n; i++)
91         for(int j = 0; j < (int)bkts[i].size(); ++j){
92             if(i <= res)
93                 toUpd.push(make_pair(pii(i, j), newPair));
94             if(res <= i)
95                 toUpd.push(make_pair(newPair, pii(i, j)));
96         }
97     }
98 }

```

7.30 Sieve (Linear)

```
1 #include <bits/stdc++.h>
```

```

2 using namespace std;
3 const int MAX = 10000005;
4
5 int phi[MAX], dvc[MAX], sig[MAX], mob[MAX];
6 int least[MAX], lstCnt[MAX], lstSum[MAX];
7 vector<int> primes;
8
9 void RunLinearSieve(int n) {
10     n = max(n, 1);
11     for(int i = 0; i <= n; i++) least[i] = lstCnt[i] = lstSum[i] = 0;
12
13     primes.clear();
14     phi[1] = dvc[1] = sig[1] = mob[1] = 1;
15
16     for(int i = 2; i <= n; i++){
17         if(least[i] == 0){
18             least[i] = i; lstCnt[i] = 1; lstSum[i] = 1 + i;
19             phi[i] = i - 1; dvc[i] = 2; sig[i] = 1 + i; mob[i] = -1;
20             primes.push_back(i);
21         }
22         for(int x : primes){
23             if(x > least[i] || i * x > n) break;
24             least[i * x] = x;
25
26             if(least[i] == x){
27                 lstCnt[i * x] = lstCnt[i] + 1;
28                 lstSum[i * x] = 1 + x * lstSum[i];
29
30                 phi[i * x] = phi[i] * x;
31                 dvc[i * x] = dvc[i] / (lstCnt[i] + 1) * (lstCnt[i * x] + 1);
32                 sig[i * x] = sig[i] / lstSum[i] * lstSum[i * x];
33                 mob[i * x] = 0;
34             }
35             else{
36                 lstCnt[i * x] = 1;
37                 lstSum[i * x] = 1 + x;
38
39                 phi[i * x] = phi[i] * (x - 1);
40                 dvc[i * x] = dvc[i] * 2;
41                 sig[i * x] = sig[i] * (1 + x);
42                 mob[i * x] = -mob[i];
43             }
44         }
45     }
46 }
47
48 int main(){
49     int n = 100;
50     RunLinearSieve(n);
51     for(int i=1;i<=n;i++) cout<<i<<" "<<phi[i]<<" "<<dvc[i]<<" "<<sig[i]<<" "<<mob[i]<<endl;
52 }

```

7.31 FFT (Faster)

```

1 //Fast FFT
2 //Complex Class Hard Coded
3 struct base {
4     typedef double T;
5     T re, im;
6     base() :re(0), im(0) {}
7     base(T re) :re(re), im(0) {}
8     base(T re, T im) :re(re), im(im) {}
9     base op + (const base& o) const {
10         return base(re + o.re, im + o.im);
11     }
12     base op - (const base& o) const {
13         return base(re - o.re, im - o.im);
14     }
15     base op * (const base& o) const {
16         return base(re * o.re - im * o.im, re * o.im + im * o.re);
17     }
18     base op * (ld k) const {return base(re * k, im * k) ;}
19     base conj() const { return base(re, -im);}
20 };
21 const ll N = 21;
22 const ll MAXN = (1<<N);
23 base w[MAXN];
24 base f1[MAXN];
25 ll rev[MAXN];
26 void build_rev(ll k) {
27     static ll rk = -1;
28     if( k == rk )return ;
29     rk = k;
30     for(ll i=1; i<=(1<<k); i++) {
31         ll j = rev[i-1], t = k-1;
32         while( t >= 0 && ((j>>t)&1) ) {j ^= 1 << t;--t;}
33         if( t >= 0 ) { j ^= 1 << t; --t;}
34         rev[i] = j;
35     }
36 }
37 void fft(base *a, ll k) {
38     build_rev(k);
39     ll n = 1 << k;
40     for(ll i=0; i<n; i++) if( rev[i] > i ) swap(a[i], a[rev[i]]);
41     for(ll l = 2, llo = 1; l <= n; l += l, llo += llo) {
42         if( w[llo].re == 0 && w[llo].im == 0 ) {
43             ld angle = M_PI / llo;
44             base ww( cosl(angle), sinl(angle) );
45             if( llo > 1 ) for(ll j = 0; j < llo; ++j) {
46                 if( j & 1 ) w[llo + j] = w[(llo+j)/2] * ww;
47                 else w[llo + j] = w[(llo+j)/2];
48             }
49             else w[llo] = base(1, 0);
50         }
51         for(ll i = 0; i < n; i += l) {

```

```

52             for(ll j=0; j<llo; j++) {
53                 base v = a[i + j], u = a[i + j + llo] * w[llo + j];
54                 a[i + j] = v + u;
55                 a[i + j + llo] = v - u;
56             }
57         }
58     }
59 }
60 void Multiply(vector<ll>& a, vector<ll>& b,vector<ll>& res) {
61     ll k = 1;
62     while( (1<<k) < (a.size() + b.size()) ) ++k;
63     ll n = (1<<k);
64     for(ll i=0; i<n; i++) f1[i] = base(0,0);
65     for(ll i=0; i<a.size(); i++) f1[i] = f1[i] + base(a[i], 0);
66     for(ll i=0; i<b.size(); i++) f1[i] = f1[i] + base(0, b[i]);
67     fft(f1, k);
68     for(ll i=0; i<1+n/2; i++) {
69         base p = f1[i] + f1[(n-i)%n].conj();
70         base _q = f1[(n-i)%n] - f1[i].conj();
71         base q(_q.im, _q.re);
72         f1[i] = (p * q) * 0.25;
73         if( i > 0 ) f1[(n - i)] = f1[i].conj();
74     }
75     for(ll i=0; i<n; i++) f1[i] = f1[i].conj();
76     fft(f1, k);
77     res.resize(a.size() + b.size());
78     for(ll i=0; i<res.size(); i++) {
79         if(fabs(f1[i].re) < eps) res[i]=0;
80         else res[i] = f1[i].re / fabs(f1[i].re) * ll (abs(f1[i].re / n) + 0.5);
81     }
82 }

```

7.32 FFT (Slow, but short)

```

1 //Slow, but short FFT
2 //Using STL Complex Class
3
4 typedef vector<int> VI;
5 typedef complex<double> CN;
6
7 const double PI = acos(-1);
8 const double eps = 1e-6;
9
10 void FFT(vector<CN>&a,bool invert){
11     int n=a.size();
12     for(int i=1,j=0;i<n;i++){
13         int bit=n>>1;
14         for(;j>=bit;bit>>=1) j-=bit ;
15         j+=bit ;
16         if(i<j) swap(a[i],a[j]);
17     }
18 }

```

```

19     for(int len=2;len<=n;len<<=1){
20         double ang=2*PI/len*(invert ? -1:1);
21         CN wlen(cos(ang),sin(ang));
22         for(int i=0;i<n;i+=len){
23             CN w(1) ;
24             for(int j=0;j<len/2;j++){
25                 CN u=a[i+j],v=a[i+j+len/2]*w;
26                 a[i+j]=u+v;
27                 a[i+j+len/2]=u-v;
28                 w*=wlen;
29             }
30         }
31     }
32     if(invert) for(int i=0;i<n;i++) a[i]/=n;
33 }
34
35 inline ll Round(double x){
36     if(abs(x)<eps) return 0;
37     else return (x/abs(x)) * (ll)(abs(x)+0.5);
38 }
39
40 void multiply ( vector<ll> & a, vector<ll> & b, vector<ll> & res) {
41     ll sq=sqrt(MOD);
42     int n = 1;
43     while (n < max (a.size(), b.size())) n <<= 1;
44     n <<= 1;
45
46     vector<CN> a1(n),a2(n),b1(n),b2(n);
47
48     a.resize(n);
49     b.resize(n);
50     for(int i=0;i<n;i++){
51         a1[i]=a[i]/sq; a2[i]=a[i]%sq;
52         b1[i]=b[i]/sq; b2[i]=b[i]%sq;
53     }
54
55     FFT(a1, false), FFT(a2, false);
56     FFT(b1, false), FFT(b2, false);
57     vector<CN>c1(n),c2(n),c3(n);
58
59     for(int i=0;i<n;i++){
60         c1[i] = a1[i]*b1[i];
61         c2[i] = a1[i]*b2[i]+b1[i]*a2[i];
62         c3[i] = a2[i]*b2[i];
63     }
64
65     FFT(c1,true);
66     FFT(c2,true);
67     FFT(c3,true);
68     res.resize(n);
69
70     for(int i=0; i<n; ++i){

```

```

72         res[i] = ( (sq*sq) %MOD * (Round(c1[i].real()) % MOD) ) %MOD
73                 + ( sq * (Round(c2[i].real()) % MOD) ) %MOD
74                 + ( Round(c3[i].real()) %MOD );
75         res[i] %= MOD;
76     }
77 }

```

7.33 FWHT

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define MOD 1000000007
4  #define ll long long int
5
6  #define OR 0
7  #define AND 1
8  #define XOR 2
9  #define LOG 17
10 #define MAX (1 << 17)
11
12 //Fast Walsh-Hadamard Transformation
13 //Complexity : O(n log n)
14 //OR, AND works for any modulo, XOR works for only prime
15 struct fwht{
16     ll P1[MAX], P2[MAX];
17     inline ll cMOD(ll x) {return x>=MOD ? x-MOD : x;}
18     inline ll Add(ll a,ll b) {return cMOD(a+b);}
19     inline ll Sub(ll a,ll b) {return Add(a,MOD-b);}
20     inline ll Half(ll a) {return (a & 1) ? (a+MOD)/2 : a/2;}
21
22     void walsh_transform(ll* ar,int n,int flag = XOR){
23         for(int m=1; m<n; m<<=1)
24             for(int k=0; k<n; k+=m<<1)
25                 for(int i=k; i<k+m; i++){
26                     ll x = ar[i], y = ar[i + m];
27                     if (flag == OR) ar[i] = x, ar[i + m] = Add(x,y);
28                     if (flag == AND) ar[i] = Add(x,y), ar[i + m] = y;
29                     if (flag == XOR) ar[i] = Add(x,y), ar[i + m] = Sub(x,y);
30                 }
31     }
32
33     void inverse_walsh_transform(ll* ar, int n, int flag = XOR){
34         for(int m=1; m<n; m<<=1)
35             for(int k=0; k<n; k+=m<<1)
36                 for(int i=k; i<k+m; i++){
37                     ll x = ar[i], y = ar[i + m];
38                     if (flag == OR) ar[i] = x, ar[i + m] = Sub(y,x);
39                     if (flag == AND) ar[i] = Sub(x,y), ar[i + m] = y;
40                     if (flag == XOR) ar[i] = Half(Add(x,y)), ar[i + m] = Half(Sub
41                                     (x, y));
42                 }
43     }

```

```

43 vector<ll> convolution(int n, ll* A, ll* B, int flag = XOR){
44     assert(__builtin_popcount(n) == 1); // n must be a power of 2
45     for (int i = 0; i < n; i++) P1[i] = A[i] % MOD;
46     for (int i = 0; i < n; i++) P2[i] = B[i] % MOD;
47
48     walsh_transform(P1, n, flag);
49     walsh_transform(P2, n, flag);
50     for (int i = 0; i < n; i++) P1[i] = (P1[i] * P2[i]) % MOD;
51     inverse_walsh_transform(P1, n, flag);
52     return vector<ll> (P1, P1 + n);
53 }
54
55 // For i = 0 to n - 1, j = 0 to n - 1 : v[i or j] += A[i] * B[j]
56 vector<ll> or_convolution(int n, ll* A, ll* B) {return convolution(n, A, B,
57 OR);}
58
59 // For i = 0 to n - 1, j = 0 to n - 1 : v[i and j] += A[i] * B[j]
60 vector<ll> and_convolution(int n, ll* A, ll* B) {return convolution(n, A, B,
61 AND);}
62
63 // For i = 0 to n - 1, j = 0 to n - 1 : v[i xor j] += A[i] * B[j]
64 vector<ll> xor_convolution(int n, ll* A, ll* B) {return convolution(n, A, B,
65 XOR);}
66
67 // For i = 0 to n - 1, j = 0 to n - 1 : v[i or j] += A[i] * B[j] when i & j ==
68 0
69 int PopCount[MAX];
70 ll F1[LOG+1][MAX];
71 ll F2[LOG+1][MAX];
72 ll G[LOG+1][MAX];
73
74 vector<ll> subset_or_convolution(int n, ll* A, ll* B){
75     for(int i=1;i<MAX;i++) PopCount[i] = PopCount[i>>1] + (i&1);
76     for(int i=0;i<n;i++) F1[PopCount[i]][i] = A[i] % MOD;
77     for(int i=0;i<n;i++) F2[PopCount[i]][i] = B[i] % MOD;
78
79     for(int i=0;i<=LOG;i++) walsh_transform(F1[i],n,OR);
80     for(int i=0;i<=LOG;i++) walsh_transform(F2[i],n,OR);
81
82     for(int i=0;i<=LOG;i++) for(int j=0;j<=i;j++) for(int k=0;k<n;k++){
83         G[i][k]+=(F1[j][k]*F2[i-j][k])%MOD;
84         if(G[i][k]>=MOD) G[i][k]-=MOD;
85     }
86     for(int i=0;i<=LOG;i++) inverse_walsh_transform(G[i],n,OR);
87     for(int i=0;i<n;i++) F1[0][i]=G[PopCount[i]][i];
88     return vector<ll> (F1[0], F1[0] + n);
89 }
90
91 fwht fft;
92
93 int main(){

```

```

92 //Parameter n (Size of vector) must be -
93 //1. Greater than input vector size
94 //2. Power of 2 with additional 0 padding (2 --> 4, 3 --> 4, 4 --> 8)
95 }

```

7.34 KaratSuva

```

1  /* Polynomial multiplication
2   Algorithm: Karatsuva multiplication
3   Order: n^1.58
4   Note : 1) All the number should be kept in reverse order in vector
5           2) initially for two numbers length should be power of 2,
6              otherwise padded extra field with zero */
7
8  #include<bits/stdc++.h>
9  using namespace std;
10 #define ll long long int
11
12 // to check if all value of A is zero
13 bool AllZero( vector<ll> &A ){
14     for(int i=0;i<A.size();i++ ) if(A[i]) return false;
15     return true;
16 }
17
18 // multiple to polynomial A and B , result in Ret
19 // A should be represented as A1*x^(n/2) + A0
20 // B should be represented as B1*x^(n/2) + B0
21 void KaratSuva( vector<ll> &A,vector<ll> &B,vector<ll> &Ret ){
22     ll i,j,n = A.size();
23     Ret.resize(2*n);
24
25     // check if any polynomial is zero
26     if( AllZero( A ) || AllZero( B ) ) return;
27
28     /* normal multiplication for n lower value
29     limit should be power of 2 ( 16,32,64 )
30     any value can be used but 32 seems to be better */
31     if(n<=32){
32         for(i=0;i<n;i++) for(j=0;j<n;j++){
33             Ret[i+j] += A[i]*B[j];
34         }
35     }
36
37     vector<ll> A1( A.begin()+n/2,A.end());
38     vector<ll> A0( A.begin(),A.begin()+n/2 );
39     vector<ll> B1( B.begin()+n/2,B.end());
40     vector<ll> B0( B.begin(),B.begin()+n/2 );
41
42     vector<ll> A1_B1,A0_B0;
43     KaratSuva( A1,B1,A1_B1 );
44     KaratSuva( A0,B0,A0_B0 );
45

```

```

46     for(i=0;i<n/2;i++) {A1[i] += A0[i]; B1[i] += B0[i];}
47
48     vector<ll> A_B;
49     KaratSuva( A1,B1,A_B );
50     for(i=0;i<n;i++) A_B[i] -= A1_B1[i] + A0_B0[i];
51
52     for(i=0;i<n;i++){
53         Ret[i] += A0_B0[i];
54         Ret[i+n/2] += A_B[i];
55         Ret[i+n] += A1_B1[i];
56     }
57 }
58
59 int main(){
60     int n;
61     scanf("%d",&n);
62
63     vector<ll> A, B;
64     A.resize(n+1); B.resize(n+1);
65
66     for(int i=0;i<=n;i++) scanf("%lld",&A[i]);
67     for(int i=0;i<=n;i++) scanf("%lld",&B[i]);
68     n++;
69     while(n&(n-1)) A.push_back(0), B.push_back(0), n++;
70
71     vector<ll> Ret;
72     KaratSuva(A,B,Ret);
73     while(Ret.back() == 0) Ret.pop_back();
74 }

```

7.35 NTT Applications

```

1  /*****
2  * polyinv(a, b, n) calculates b: b(z)a(z) = 1 (mod z^n)
3  * polysqrt(a, b, n) calculates b: b(z)^2 = a(z) (mod z^n)
4  * make sure ta[], tb[] has 2n spaces to be used.
5  * in polysqrt(), if a[0] != 1 then discrete sqrt function is needed
6  */
7
8  int ta[N], tb[N], tc[N];
9  void prepare(int n) { init(n >> 1); }
10 void ntt(int *a, int n, ll f){
11     for(int i=0; i<n; i++) A[i] = a[i];
12     ntransform(A, f);
13     for(int i=0; i<n; i++) a[i] = A[i];
14 }
15
16 void polyinv(int *a, int *b, int n) {
17     if(n == 1) return void(b[0] = Pow(a[0], mod - 2));
18     polyinv(a, b, n >> 1);
19     for(int i = 0; i < n; ++i)
20         ta[i] = a[i], tb[i] = b[i];

```

```

21     for(int i = n; i < (n << 1); ++i)
22         ta[i] = tb[i] = 0;
23     n <<= 1; prepare(n);
24     ntt(ta, n, 0), ntt(tb, n, 0);
25     for(int i = 0; i < n; ++i)
26         b[i] = (ll) tb[i] * (2 + mod - (ll) ta[i] * tb[i] % mod) % mod;
27     ntt(b, n, 1);
28     fill(b + (n >> 1), b + n, 0);
29 }
30
31 int inv2 = Pow(2, mod - 2);
32 void polysqrt(int *a, int *b, int n) {
33     if(n == 1) return void(b[0] = 1); // b[0] = x: x^2 \equiv a[0]
34     polysqrt(a, b, n >> 1);
35     polyinv(b, tb, n);
36     for(int i = 0; i < n; ++i)
37         ta[i] = a[i];
38     for(int i = n; i < (n << 1); ++i)
39         ta[i] = tb[i] = 0;
40     n <<= 1; prepare(n);
41     ntt(ta, n, 0); ntt(tb, n, 0);
42     for(int i = 0; i < n; ++i)
43         ta[i] = (ll) ta[i] * tb[i] % mod;
44     ntt(ta, n, 1);
45     for(int i = 0; i < n; ++i)
46         b[i] = (ll) inv2 * (ta[i] + b[i]) % mod;
47     fill(b + (n >> 1), b + n, 0);
48 }

```

7.36 NTT

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define ll long long int
4  const ll mod = 786433;
5  //For 10^9 + 7 : {7340033, 415236097, 463470593}
6
7  inline ll add(ll x, ll y) {x += y; return x >= mod ? x - mod : x;}
8  inline ll sub(ll x, ll y) {x -= y; return x < 0 ? x + mod : x;}
9  inline ll mul(ll x, ll y) {return (x * y) % mod;}
10 inline ll Pow(ll x, ll y){
11     ll res = 1;
12     while(y){
13         if(y & 1) res = (res * x) % mod;
14         y >>= 1, x = (x * x) % mod;
15     }
16     return res;
17 }
18
19 struct NTT{
20     vector<ll> A,B;
21     vector<ll> root, rev;

```

```

22 ll P,M,G;
23
24 NTT(ll mod) {P = mod; G = 10;}
25 void init(int n){
26     M = 1; int z = 0;
27     while(M < n) M <= 1, z++;
28     M <= 1; z++;
29
30     A.resize(M); B.resize(M);
31     root.resize(M); rev.resize(M);
32     for(int i = 1; i < M; i++) rev[i] = rev[i >> 1] >> 1 | (i & 1) << (z - 1);
33
34     int x = Pow(G, (P-1) / M);
35     root[0] = 1;
36     for(int i = 1; i < M; i++) root[i] = mul(root[i-1], x);
37 }
38
39 void ntransform(vector<ll> &a,ll f){
40     if(f) reverse(a.begin() + 1, a.end());
41     for (ll i = 0; i < M; i++) if(i < rev[i]) swap(a[i],a[rev[i]]);
42     for (ll i = 1; i < M; i <= 1){
43         for (ll j = 0, t = M / (i << 1); j < M; j += i << 1){
44             for (ll k = 0, l = 0; k < i; k++, l += t){
45                 ll x = a[j + k];
46                 ll y = mul(a[j + k + i], root[l]);
47                 a[j + k] = add(x, y);
48                 a[j + k + i] = sub(x, y);
49             }
50         }
51     }
52
53     if(f){
54         ll inv = Pow(M, P-2);
55         for(ll i = 0; i < M; i++) a[i] = mul(a[i], inv);
56     }
57 }
58
59 void multiply( vector<ll> &X, vector<ll> &Y, vector<ll> &res){
60     init(max(X.size(), Y.size()));
61     for (ll i = 0; i < M; i++) A[i] = B[i] = 0;
62     for (ll i = 0; i < X.size(); i++) A[i] = X[i];
63     for (ll i = 0; i < Y.size(); i++) B[i] = Y[i];
64     ntransform(A, 0); ntransform(B, 0);
65     res.clear(); res.resize(M);
66     for (ll i = 0; i < M; i++) res[i] = mul(A[i], B[i]);
67     ntransform(res, 1);
68 }
69 } ntt(mod);

```

7.37 Online FFT Trick

```

1 //Online FFT Calculation
2 //a[i] = Sum of a[j]*b[i-j-1] for j in range [0,i-1]
3 //Problem Link : https://www.codechef.com/problems/MANCBST
4
5 const ll MAX = 1e5 + 9;
6 ll a[2*MAX+10], b[2*MAX+10];
7 vector<ll> vec1,vec2,res;
8
9 int main(){
10     b[0]=b[1]=1;
11     for(int i=2;i< MAX;i++) b[i]=bigMod(i,i-1);
12
13     a[0]=1;
14     for(int i = 0; i < MAX; i++){
15         //+1 because of the format of the recurrence
16         a[i + 0 + 1] += (a[i] * b[0]) % mod;
17         a[i + 1 + 1] += (a[i] * b[1]) % mod;
18         if(a[i + 0 + 1] >= mod) a[i + 0 + 1]-=mod;
19         if(a[i + 1 + 1] >= mod) a[i + 1 + 1]-=mod;
20
21         ll cc = 1;
22         ll tmp = i;
23         while(tmp && (tmp & 1) == 0){
24             tmp = tmp / 2; cc = cc * 2;
25             vec1.clear(); vec2.clear();
26
27             for(int j=i-cc;j<i;j++) vec1.push_back(a[j]);
28             for(int j=cc;j<cc+cc;j++) vec2.push_back(b[j]);
29             ntt.multiply(vec1, vec2, res);
30
31             ll Beg = i+1; //+1 because of the format of the recurrence
32             for(ll j = 0; j < res.size(); j++){
33                 a[Beg+j] += res[j];
34                 if(a[Beg+j]>=mod) a[Beg+j]-=mod;
35             }
36         }
37     }
38 }

```

7.38 Polynomial Root Check

```

1 //Check whether x=k is a root of polynomial f(x)
2 //Check whether f(k) = 0 efficiently
3
4 #define ll long long int
5 bool divisible_by_k(vector<ll>arr,int k){
6     int n = arr.size();
7     if(arr[0] % k != 0) return false;
8     for(int i=1;i<n;i++){
9         arr[i] += arr[i-1]/k;
10        if(arr[i] % k != 0) return false;
11    }

```

```

12     return (arr[n-1] == 0);
13 }

```

7.39 Polynomial

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long int ll;
4  const ll p = 998244353, gen = 3;
5
6  inline ll add(ll x, ll y) {x += y; return x >= p ? x - p : x;}
7  inline ll sub(ll x, ll y) {x -= y; return x < 0 ? x + p : x;}
8  inline ll mul(ll x, ll y) {return (x * y) % p;}
9  inline ll Pow(ll x, ll y){
10     ll res = 1;
11     while(y){
12         if(y & 1) res = mul(res, x);
13         y >>= 1, x = mul(x, x);
14     }
15     return res;
16 }
17
18 namespace Polynom{
19     vector<ll> rev, root;
20     vector<ll> operator + (vector<ll> f, vector<ll> g) {
21         ll n = max(f.size(), g.size());
22         f.resize(n); g.resize(n);
23         for(ll i = 0; i < n; i++) f[i] = add(f[i], g[i]);
24         return f;
25     }
26
27     vector<ll> operator - (vector<ll> f, vector<ll> g) {
28         ll n = max(f.size(), g.size());
29         f.resize(n); g.resize(n);
30         for(ll i = 0; i < n; i++) f[i] = sub(f[i], g[i]);
31         return f;
32     }
33
34     void getRevRoot(ll n) {
35         ll m = 0; while( (1<<m) < n) m++;
36
37         rev.resize(n); root.resize(n);
38         for(ll i = 1; i < n; i++) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (m -
39 1));
40         for(ll len = 1; len < n; len <= 1) {
41             ll uni = Pow(gen, (p ^ 1) / (len << 1));
42             root[len] = 1;
43             for(ll i = 1; i < len; i++) root[i + len] = mul(root[i + len - 1],
44 uni);
45         }
46     }
47 }

```

```

46 void ntransform(vector<ll> &f, ll n, bool t){
47     if(t) reverse(f.begin() + 1, f.end());
48     f.resize(n);
49
50     for(ll i = 0; i < n; i++) { if (i < rev[i]) {swap(f[i], f[rev[i]]);} }
51     for(ll len = 1; len < n; len <= 1) {
52         for(ll i = 0; i < n; i += len << 1) {
53             for(ll j = 0; j < len; j++) {
54                 ll x = f[i + j], y = mul(f[i + j + len], root[j + len]);
55                 f[i + j] = add(x, y); f[i + j + len] = sub(x, y);
56             }
57         }
58     }
59 }
60
61 // Given a n-degree polynomial f and m-degree polynomial g
62 // returns a (n + m) degree polynomial f * g (not in modular field)
63 vector<ll> operator * (vector<ll> f, vector<ll> g) {
64     ll n = 1, m = f.size() + g.size() - 1;
65     while (n < m) n <= 1;
66     ll invN = Pow(n, p - 2);
67
68     getRevRoot(n);
69     ntransform(f, n, false); ntransform(g, n, false);
70     for(ll i = 0; i < n; i++) f[i] = mul(f[i], g[i]);
71     ntransform(f, n, true);
72     for(ll i = 0; i < n; i++) f[i] = mul(f[i], invN);
73     f.resize(m);
74     return f;
75 }
76
77 vector<ll> polyInv(vector<ll> f, ll m){
78     vector<ll> g, h; f.resize(m);
79     if (m == 2) g.push_back(Pow(f[0], p - 2));
80     else g = polyInv(f, m >> 1);
81
82     ll n = m << 1, invN = Pow(n, p - 2);
83     g.resize(m); h.resize(m);
84     for(ll i = (m >> 1) - 1; ~i; i--) h[i] = add(g[i], g[i]);
85
86     getRevRoot(n);
87     ntransform(f, n, false); ntransform(g, n, false);
88     for(ll i = 0; i < n; i++) f[i] = mul(mul(f[i], g[i]), g[i]);
89     ntransform(f, n, true);
90     for(ll i = 0; i < m; i++) h[i] = sub(h[i], mul(f[i], invN));
91     return h;
92 }
93
94 // given a n-degree polynomials f
95 // returns w such that f * w = 1 (mod z^n)
96 vector<ll> operator ~ (vector<ll> f){
97     assert(f[0] != 0);
98     ll n = 1, m = (ll) f.size();

```

```

99     while(n < m) n <= 1;
100     if(n == 1) f[0] = Pow(f[0], p - 2);
101     else f = polyInv(f, n);
102     f.resize(m);
103     return f;
104 }
105
106 vector<ll> dvf, dvf, dvr;
107 // given 2 n-degree polynomials f and g
108 // returns w such that g * w = f (mod z^n)
109 vector<ll> operator / (vector<ll> f, vector<ll> g) {
110     if (f == dvf && g == dvf) return dvr;
111     dvf = f; dvf = g;
112     ll n = f.size(), m = g.size(); n = n - m + 1;
113     reverse(f.begin(), f.end()); f.resize(n);
114     reverse(g.begin(), g.end()); g.resize(n);
115     dvr = f * (~g); dvr.resize(n); reverse(dvr.begin(), dvr.end());
116     return dvr;
117 }
118
119 vector<ll> fac, inv, one = {1};
120 void getFacInv(ll n){
121     if (n <= fac.size()) return;
122     fac.resize(n); inv.resize(n);
123     for(ll i = fac[0] = 1; i < n; i++) fac[i] = mul(fac[i - 1], i);
124     inv[n - 1] = Pow(fac[n - 1], p - 2);
125     for(ll i = n - 1; i; i--) inv[i - 1] = mul(inv[i], i);
126 }
127
128 vector<ll> operator >> (vector<ll> f, ll m){
129     ll n = f.size(); m = min(n, m); getFacInv(n);
130     for(ll i = m; i < n; i++) f[i - m] = mul(mul(f[i], fac[i]), inv[i - m]);
131     f.resize(n - m);
132     return f;
133 }
134
135 vector<ll> operator << (vector<ll> f, ll m){
136     ll n = f.size(); f.resize(n + m); getFacInv(n + m);
137     for(ll i = n + m - 1; i >= m; i--) f[i] = mul(mul(f[i - m], fac[i - m]),
138     inv[i]);
139     fill(f.begin(), f.begin() + m, 0);
140     return f;
141 }
142
143 vector<ll> polyLn(vector<ll> f){
144     ll n = f.size();
145     f = (f >> 1) * (~f) << 1;
146     f.resize(n);
147     return f;
148 }
149
150 vector<ll> polyExp(vector<ll> f, ll n){
151     vector<ll> g, h; f.resize(n);

```

```

151     if (n == 2) g.push_back(1);
152     else g = polyExp(f, n >> 1);
153     g.resize(n);
154     return g * (one - polyLn(g) + f);
155 }
156
157 vector<ll> polyExp(vector<ll> f){
158     ll n = 1, m = (ll) f.size();
159     while(n < m) n <= 1;
160     f = polyExp(f, n);
161     f.resize(m);
162     return f;
163 }
164
165 // Given a n-degree polynomial f
166 // returns w such that w = f ^ x (mod z^n)
167 vector<ll> operator ^ (vector<ll> f, ll x){
168     ll idx = 0;
169     while(idx < f.size() && f[idx] == 0) idx++;
170     if(idx == f.size()) return f;
171
172     vector<ll> g;
173     for(int i = idx; i < f.size(); i++) g.push_back(f[i]);
174
175     ll n = g.size();
176     ll inv = Pow(g[0], p-2), ext = Pow(g[0], x);
177     for(ll i = 0; i < n; i++) g[i] = mul(g[i], inv);
178     g = polyLn(g);
179     for(ll i = 0; i < n; i++) g[i] = mul(g[i], x);
180     g = polyExp(g);
181     for(ll i = 0; i < n; i++) g[i] = mul(g[i], ext);
182
183     f.clear();
184     for(ll i = 0; i < idx * x; i++) f.push_back(0);
185     for(ll i = 0; i < n; i++){
186         if(f.size() == n + idx) break;
187         f.push_back(g[i]);
188     }
189     return f;
190 }
191 }
192 using namespace Polynom;
193
194 int main(){
195     vector<ll> a{43431,1231,435,43340,430,10};
196     vector<ll> b = ~a;
197     vector<ll> c = a * b * a;
198     for(ll x : c) cout<<x<<" "; cout<<endl;
199
200     vector<ll> d = a^3;
201     vector<ll> e = a * a * a;
202     for(ll x : d) cout<<x<<" "; cout<<endl;
203     for(ll x : e) cout<<x<<" "; cout<<endl;

```

```

204     e.resize(a.size());
205
206     vector<ll> f = e / d;
207     vector<ll> g = f * d;
208     for(ll x : g) cout<<x<<" ";
209 }

```

8 Matroid

8.1 colorGraphicMatroid

```

1  /**
2   * Description: computes a set of maximum size which is independent
3   * in both graphic and colorful matroids, aka a spanning forest where
4   * no two edges are of the same color
5   * Time:  $O(GI^{1.5})$  calls to oracles, where  $G$  is the size of the ground set
6   * and  $I$  is the size of the independent set
7   * Source: https://github.com/bqi343/USACO/blob/master/Implementations/content/
8   * Verification: https://www.urionlinejudge.com.br/judge/en/problems/view/2128
9   * Verification: https://www.spoj.com/problems/COIN/
10  */
11  int cs, N, M, R;
12  const int maxn = 307;
13  const int maxm = 307;
14  const int maxc = 307;
15  typedef pair<int,int> pi;
16  typedef vector<int> vi;
17  #define FOR(i,j) for(int i = 0; i < j ; i++)
18  #define sz(x) x.size()
19  #define f first
20  #define s second
21  #define trav(u,x) for(auto& u : x)
22  #define MOD 1e9+7
23  #define pb push_back
24  struct DSU {
25     vi e; void init(int n) { e = vi(n,-1); }
26     int get(int x) { return e[x] < 0 ? x : e[x] = get(e[x]); }
27     bool sameSet(int a, int b) { return get(a) == get(b); }
28     bool unite(int x, int y) { // union-by-rank
29         x = get(x), y = get(y); if (x == y) return 0;
30         if (e[x] > e[y]) swap(x,y);
31         e[x] += e[y]; e[y] = x;
32         return 1;
33     }
34 };
35 map<int,int> m;
36 struct Element {
37     pi ed; int col;
38     bool in_independent_set = 0;
39     int independent_set_position;

```

```

40     Element(int u, int v, int c) { ed = {u,v}; col = c; }
41 };
42 vi independent_set; vector<Element> ground_set; bool col_used[maxc];
43 struct GBasis {
44     DSU D;
45     void reset() { D.init(sz(m)); }
46     void add(pi v) { assert(D.unite(v.f,v.s)); }
47     bool independent_with(pi v) { return !D.sameSet(v.f,v.s); }
48 };
49 GBasis basis, basis_wo[maxm];
50 bool graph_oracle(int inserted) { return basis.independent_with(ground_set[inserted].ed); }
51 bool graph_oracle(int inserted, int removed) {
52     int wi = ground_set[removed].independent_set_position;
53     return basis_wo[wi].independent_with(ground_set[inserted].ed);
54 }
55 void prepare_graph_oracle() {
56     basis.reset();
57     FOR(i,sz(independent_set)) basis_wo[i].reset();
58     FOR(i,sz(independent_set)) {
59         pi v = ground_set[independent_set[i]].ed; basis.add(v);
60         FOR(j,sz(independent_set)) if (i != j) basis_wo[j].add(v);
61     }
62 }
63 bool colorful_oracle(int ins) { ins = ground_set[ins].col; return !col_used[ins]; }
64 bool colorful_oracle(int ins, int rem) {
65     ins = ground_set[ins].col;
66     rem = ground_set[rem].col;
67     return !col_used[ins] || ins == rem;
68 }
69 void prepare_colorful_oracle() {
70     FOR(i,R) col_used[i] = 0;
71     trav(t,independent_set) col_used[ground_set[t].col] = 1;
72 }
73 bool augment() {
74     prepare_graph_oracle(); prepare_colorful_oracle();
75     vi par(sz(ground_set),MOD); queue<int> q;
76     FOR(i,sz(ground_set)) if (colorful_oracle(i)) {
77         assert(!ground_set[i].in_independent_set);
78         par[i] = -1; q.push(i);
79     }
80     int lst = -1;
81     while (sz(q)) {
82         int cur = q.front(); q.pop();
83         if (ground_set[cur].in_independent_set) {
84             FOR(to,sz(ground_set)) if (par[to] == MOD) {
85                 if (!colorful_oracle(to,cur)) continue;
86                 par[to] = cur; q.push(to);
87             }
88         } else {
89             if (graph_oracle(cur)) { lst = cur; break; }
90             trav(to,independent_set) if (par[to] == MOD) {

```

```

91     if (!graph_oracle(cur,to)) continue;
92     par[to] = cur; q.push(to);
93 }
94 }
95 }
96 if (lst == -1) return 0;
97 do {
98     ground_set[lst].in_independent_set ^= 1;
99     lst = par[lst];
100 } while (lst != -1);
101 independent_set.clear();
102 FOR(i,sz(ground_set)) if (ground_set[i].in_independent_set) {
103     ground_set[i].independent_set_position = sz(independent_set);
104     independent_set.pb(i);
105 }
106 return 1;
107 }
108 void solve() {
109     cs++;
110     m.clear(); ground_set.clear(); independent_set.clear();
111     FOR(i,M) {
112         int a, b, c; cin >> a >> b >> c; c--;
113         ground_set.pb(Element(a,b,c)); m[a] = m[b] = 0;
114     }
115     int co = 0; trav(t,m) t.s = co++;
116     trav(t,ground_set) t.ed.f = m[t.ed.f], t.ed.s = m[t.ed.s];
117     while(augment()); //// keep increasing size of independent set
118     //Matroid Intersection is stored in Independent Set
119 }
120 int main(){
121     ios_base::sync_with_stdio(false); cin.tie(NULL);
122     while( cin >> N >> M >> R ) solve();
123 }

```

8.2 colorLinearMatroid

```

1  /*
2  source : https://codeforces.com/blog/entry/69287
3          [Tutorial] Matroid intersection in simple words by ATSTNG
4  */
5  #include <bits/stdc++.h>
6
7  using namespace std;
8
9  typedef long long int Z2Vector;
10
11 struct GroundSetElement {
12     int color;
13     Z2Vector vector;
14     bool in_independent_set;
15     int independent_set_position;
16     int bfs_parent;

```

```

17 };
18
19 struct Z2Basis {
20     vector<Z2Vector> vectors;
21
22     void add_vector(Z2Vector new_vector) {
23         vectors.push_back(new_vector);
24     }
25
26     void reset() {
27         vectors.clear();
28     }
29
30     int size() {
31         return vectors.size();
32     }
33
34     void gaussian_elimination() {
35         for (int i = 0; i < size(); i++) {
36             for (int j = i; j < size(); j++) {
37                 if (vectors[i] < vectors[j]) swap(vectors[i], vectors[j]);
38             }
39
40             for (int j = i+1; j < size(); j++) {
41                 vectors[j] = min(vectors[j], vectors[j] ^ vectors[i]);
42             }
43         }
44     }
45
46     bool independent_with(Z2Vector new_vector) {
47         for (auto v : vectors) new_vector = min(new_vector, new_vector ^ v);
48
49         return new_vector > 0;
50     }
51 };
52
53 int n;
54 int m;
55
56 int color_count;
57 bool color_used[60+60];
58
59 vector<GroundSetElement> ground_set;
60 vector<int> independent_set; /// as indexes of ground set
61 Z2Basis basis; /// of independent set
62 Z2Basis basis_without[60+60];
63
64 bool linear_oracle(int inserted) {
65     return basis.independent_with(ground_set[inserted].vector);
66 }
67
68 bool linear_oracle(int inserted, int removed) {
69     int wi = ground_set[removed].independent_set_position;

```

```

70     return basis_without[wi].independent_with(ground_set[inserted].vector);
71 }
72
73
74 void prepare_linear_oracle() {
75     basis.reset();
76
77     for (int i = 0; i < independent_set.size(); i++) {
78         basis_without[i].reset();
79     }
80
81     for (int i = 0; i < independent_set.size(); i++) {
82         basis.add_vector(ground_set[independent_set[i]].vector);
83         for (int j = 0; j < independent_set.size(); j++) {
84             if (i != j) basis_without[j].add_vector(ground_set[independent_set[i]
85             ].vector);
86         }
87     }
88     basis.gaussian_elimination();
89     for (int i = 0; i < independent_set.size(); i++) {
90         basis_without[i].gaussian_elimination();
91     }
92 }
93
94 bool colorful_oracle(int inserted) {
95     int inserted_color = ground_set[inserted].color;
96
97     return !color_used[inserted_color];
98 }
99
100 bool colorful_oracle(int inserted, int removed) {
101     int inserted_color = ground_set[inserted].color;
102     int removed_color = ground_set[removed].color;
103
104     if (!color_used[inserted_color]) return true;
105
106     return inserted_color == removed_color;
107 }
108
109 void prepare_colorful_oracle() {
110     for (int c = 0; c < color_count; c++) {
111         color_used[c] = false;
112     }
113
114     for (auto idx : independent_set) {
115         color_used[ground_set[idx].color] = true;
116     }
117 }
118
119 bool augment() {
120     prepare_linear_oracle();
121     prepare_colorful_oracle();

```

```

122
123     const int SOURCE = -1;
124     const int NOT_VISITED = -2;
125     const int NOT_FOUND = -3;
126
127     for (int i = 0; i < ground_set.size(); i++) {
128         ground_set[i].bfs_parent = NOT_VISITED;
129     }
130
131     int endpoint = NOT_FOUND;
132
133     queue<int> q;
134
135     for (int i = 0; i < ground_set.size(); i++) {
136         if (colorful_oracle(i)) {
137             ground_set[i].bfs_parent = SOURCE;
138             q.push(i);
139         }
140     }
141
142     while (q.size()) {
143         int cur = q.front();
144         q.pop();
145
146         if (ground_set[cur].in_independent_set) {
147             for (int to = 0; to < ground_set.size(); to++) {
148                 if (ground_set[to].bfs_parent != NOT_VISITED) continue;
149                 if (!colorful_oracle(to, cur)) continue;
150
151                 ground_set[to].bfs_parent = cur;
152                 q.push(to);
153             }
154         } else {
155             if (linear_oracle(cur)) {
156                 endpoint = cur;
157                 break;
158             }
159
160             for (auto to : independent_set) {
161                 if (ground_set[to].bfs_parent != NOT_VISITED) continue;
162                 if (!linear_oracle(cur, to)) continue;
163
164                 ground_set[to].bfs_parent = cur;
165                 q.push(to);
166             }
167         }
168     }
169
170     if (endpoint == NOT_FOUND) return false;
171
172     do {
173         ground_set[endpoint].in_independent_set ^= true;
174         endpoint = ground_set[endpoint].bfs_parent;

```

```

175     } while (endpoint != SOURCE);
176
177     independent_set.clear();
178     for (int i = 0; i < ground_set.size(); i++) {
179         if (ground_set[i].in_independent_set) {
180             ground_set[i].independent_set_position = independent_set.size();
181             independent_set.push_back(i);
182         }
183     }
184
185     return true;
186 }
187
188 void solve() {
189     cin >> n;
190     for (int i = 0; i < n; i++) {
191         ground_set.emplace_back();
192         ground_set.back().color = i;
193         ground_set.back().in_independent_set = false;
194
195         cin >> ground_set.back().vector;
196     }
197
198     cin >> m;
199     for (int i = 0; i < m; i++) {
200         int k;
201         cin >> k;
202         while (k--) {
203             ground_set.emplace_back();
204             ground_set.back().color = n+i;
205             ground_set.back().in_independent_set = false;
206
207             cin >> ground_set.back().vector;
208         }
209     }
210
211     color_count = n+m;
212
213     while (augment()) ;
214
215     if (independent_set.size() < color_count) {
216         cout << "-1\n";
217         return;
218     }
219
220     for (int i = n; i < ground_set.size(); i++) {
221         if (ground_set[i].in_independent_set) cout << ground_set[i].vector << "\n";
222     }
223 }
224
225 int main() {
226     solve();

```

```

227 }

```

8.3 graphicGraphicMatroid

```

1 // courtesy : https://discuss.codechef.com/t/cnnct2-editorial/40191
2 #include <bits/stdc++.h>
3 using namespace std;
4 const int MX = 300;
5 int n, m;
6 bool enabled[MX];
7 struct graph_t {
8     int u[MX], v[MX];
9     void read();
10    vector<int> getNonBridges();
11 } G1, G2;
12 void graph_t::read() {
13     for (int i = 0; i < m; i++) {
14         ignore = scanf("%d %d", u + i, v + i);
15         u[i]--;
16         v[i]--;
17     }
18 }
19 vector<int> graph_t::getNonBridges() {
20     static vector<pair<int, int>> G[MX];
21     for (int i = 0; i < n; i++) G[i].clear();
22     for (int i = 0; i < m; i++) {
23         if (enabled[i] == false) continue;
24         G[u[i]].emplace_back(v[i], i);
25         G[v[i]].emplace_back(u[i], i);
26     }
27     static bool vis[MX];
28     static int up[MX], dep[MX];
29     fill(vis, vis + n, false);
30     vector<int> result;
31     function<void(int, int, int)> dfs = [&](int v, int p, int d) {
32         vis[v] = true;
33         dep[v] = d;
34         up[v] = d;
35         for (auto& e : G[v]) {
36             if (e.second == p) continue;
37
38             if (vis[e.first] == false) {
39                 dfs(e.first, e.second, d + 1);
40                 if (up[e.first] <= d) result.push_back(e.second);
41             }
42             else {
43                 if (dep[e.first] < d) result.push_back(e.second);
44             }
45
46             up[v] = min(up[v], up[e.first]);
47         }
48     };

```

```

49  dfs(0, -1, 0);
50  return result;
51 }
52 namespace ExchangeGraph {
53  vector<int> G[MX];
54  void clear() { for (int i = 0; i < m; i++) G[i].clear(); }
55  void addEdge(int u, int v) { G[u].push_back(v); }
56  bool findAugmentalPath(vector<int> from, vector<int> to) {
57      vector<int> par(m, -1), dist(m), queue;
58      for (int v : from) {
59          queue.push_back(v); par[v] = -2; dist[v] = 0;
60      }
61      for (size_t i = 0; i < queue.size(); i++) {
62          int v = queue[i];
63          for (int u : G[v]) {
64              if (par[u] == -1) {
65                  queue.push_back(u); par[u] = v; dist[u] = dist[v] + 1;
66              }
67          }
68      }
69      int target = -1;
70      for (int v : to) {
71          if (par[v] == -1) continue;
72          if (target == -1 || dist[v] < dist[target]) target = v;
73      }
74      if (target != -1) {
75          while (target != -2) {
76              enabled[target] = not enabled[target]; target = par[target];
77          }
78          return true;
79      }
80      return false;
81  }
82 }
83 int main() {
84     int T;
85     ignore = scanf("%d", &T);
86     while (T--) {
87         ignore = scanf("%d %d", &n, &m);
88         G1.read();
89         G2.read();
90         fill(enabled, enabled + m, true);
91         int ans = m;
92         while (true) {
93             ExchangeGraph::clear();
94             auto Y1 = G1.getNonBridges();
95             auto Y2 = G2.getNonBridges();
96             if (ExchangeGraph::findAugmentalPath(Y1, Y2)) { ans--; continue; }
97             for (int e = 0; e < m; e++) {
98                 if (enabled[e]) continue;
99                 enabled[e] = true;
100                 auto edgesTo = G1.getNonBridges();
101                 auto edgesFrom = G2.getNonBridges();

```

```

102         enabled[e] = false;
103         for (int v : edgesTo) ExchangeGraph::addEdge(e, v);
104         for (int v : edgesFrom) ExchangeGraph::addEdge(v, e);
105     }
106     if (ExchangeGraph::findAugmentalPath(Y1, Y2)) { ans--; }
107     else { break; }
108 }
109 printf("%d\n", ans);
110 }
111 return 0;
112 }

```

9 Notes

9.1 Bernoulli

```

1 // Bernoulli Number : We have -
2 // B(k) = sum over m in [0,k] (-1)^m * m! / (m+1) * Str2(k,m)

```

9.2 Catalan

```

1 /*
2 * Total number of paths from point P(x1, y1)
3 to point Q(x2, y2) where x2 >= x1 and y2 >= y1 :
4 Let x = x2 - x1 and y = y2 - y1. Then ans = C(x+y, x).
5
6 * Total number of paths from point P(x1, y1) to point Q(x2, y2)
7 where x2 >= x1 and y2 >= y1 without crossing the line X = Y + c :
8 Let x = x2 - x1 and y = y2 - y1. Then ans = C(x+y, x) - C(x+y, x+c-1).
9 Special Case : x = n, y = n, c = 0, then ans = C(2n, n) - C(2n, n-1) [Catalan]
10
11 * Catalan triangle : Total number of permutation having n X and k Y
12 so that Count(X)-Count(Y)>=0 in any prefix (Non-negative Partial Sum) :
13 ans = C(n+k,k) - C(n+k, k-1)
14
15 * Catalan trapezoid : Total number of permutation having n X and k Y
16 so that Count(Y) - Count(X) < m in any prefix, then :
17 when 0 <= k < m, ans = C(n+k,k)
18 when m <= k <= n+m-1, ans = C(n+k,k) - C(n+k,k-m)
19 when k > n+m-1, ans = 0
20 */

```

9.3 Eulerian

```

1 * Eulerian number of the first kind :
2
3 * A1(n,k) is the number of permutations of 1 to n in which

```

```

4 exactly k elements are greater than their previous element.
5 Then :  $A1(n,k) = (n-k) * A1(n-1,k-1) + (k+1) * A1(n-1,k)$ .
6
7 * Eulerian number of the second kind :
8
9 * Number of permutations of the multiset  $\{1,1,2,2,\dots,n,n\}$ 
10 such that for each k, all the numbers appearing between the
11 two occurrences of k are greater than k =  $(2n - 1) !$ 
12
13 *  $A2(n,k)$  is the number of such permutations with k ascents.
14 Then :  $A2(n,k) = (2n-k-1) * A2(n-1, k-1) + (k+1) * A2(n-1,k)$ 

```

9.4 Stirling

```

1 /*
2 * Number of permutations of n elements with k disjoint cycles
3 =  $Str1(n,k) = (n-1) * Str1(n-1,k) + Str1(n-1,k-1)$ 
4
5 *  $n! = \text{Sum}(Str1(n,k))$  (for all  $0 \leq k \leq n$ ).
6
7 * Ways to partition n labelled objects into k unlabelled
8 subsets =  $Str2(n,k) = k * Str2(n-1,k) + Str2(n-1,k-1)$ 
9
10 * Parity of  $Str2(n,k)$  :  $((n-k) \& \text{Floor}((k-1)/2)) == 0$ 
11
12 * Ways to partition n labelled objects into k unlabelled
13 subsets, with each subset containing at least r elements :
14  $SR(n,k) = k * SR(n-1,k) + C(n-1,r-1) * SR(n-r,k-1)$ 
15
16 * Number of ways to partition n labelled objects 1,2,3, ... n
17 into k non-empty subsets so that for any integers i and j in a
18 given subset  $|i-j| \geq d$  :  $Str2(n-d+1, k-d+1)$ ,  $n \geq k \geq d$ 
19 */

```

10 String

10.1 Aho Corasick

```

1 // Aho-Corasick
2 // Complexity :  $|Text| + \text{Sum of all } |Pattern| + O(\text{number of Occurrences})$ 
3 // if occurrence positions needed, Worst Case Complexity :  $(\text{SumLen}) \text{ Root } (\text{SumLen})$ 
4 #include <bits/stdc++.h>
5 using namespace std;
6 const int MAXT = 1000005; // Length of Text
7 const int MAXP = 1000005; // Sum of all |Pattern|
8 const int MAXQ = 1000005; // Number of Patterns
9
10 int n;
11 map<char, int> Next[MAXP];

```

```

12 int Root; // AC automaton Root
13 int Nnode; // Total node count
14 int Link[MAXP]; // failure links
15 int Len[MAXP]; // Len[i] = length of i-th pattern
16 vector<int> End[MAXP]; // End[i] = indices of patterns those end in node i
17 // vector<int> Occ[MAX]; // Occ[i] = occurrences of i-th pattern
18 vector<int> edgeLink[MAXP];
19 vector<int> perNodeText[MAXP];
20
21 int in[MAXQ], out[MAXQ];
22 int euler[MAXT];
23 int Time;
24
25 void Clear(int node){
26     Next[node].clear();
27     End[node].clear();
28     edgeLink[node].clear();
29     perNodeText[node].clear();
30 }
31
32 void init(){
33     Time = 0;
34     Root = Nnode = 0;
35     Clear(Root);
36 }
37
38 void insertword(string p,int ind){
39     int len = p.size();
40     int now = Root;
41     for(int i=0; i<len; i++){
42         if(!Next[now][p[i]]){
43             Next[now][p[i]] = ++Nnode;
44             Clear(Nnode);
45         }
46         now = Next[now][p[i]];
47     }
48     End[now].push_back(ind);
49 }
50
51 void push_links(){
52     queue<int> q;
53     Link[0] = -1;
54     q.push(0);
55     while(!q.empty()){
56         int u = q.front();
57         q.pop();
58         for(auto edge : Next[u]){
59             char ch = edge.first;
60             int v = edge.second;
61             int j = Link[u];
62
63             while(j != -1 && !Next[j][ch]) j = Link[j];
64             if(j != -1) Link[v] = Next[j][ch];

```

```

65         else Link[v] = 0;
66
67         q.push(v);
68         edgeLink[Link[v]].push_back(v);
69         // for(int x : End[Link[v]]) End[v].push_back(x);
70     }
71 }
72 }
73
74 void traverse(string s){
75     int len = s.size();
76     int now = Root;
77     for(int i = 0; i < len; i++) {
78         while(now != -1 && !Next[now][s[i]]) now = Link[now];
79         if(now != -1) now = Next[now][s[i]];
80         else now = 0;
81         perNodeText[now].push_back(i+1); // using 1 based indexing for text
82         indices
83         // for(int x=0;x<End[now].size();x++) Occ[End[now][x]].push_back(i);
84     }
85 }
86 // After dfs, the occurrence of ith query string will be the count of
87 // all the occurrence of the subtree under the endNode of ith string
88 void dfs(int pos){
89     for(int q : End[pos]) in[q] = Time + 1;
90     for(int val : perNodeText[pos]) euler[++Time] = val;
91     for(int to : edgeLink[pos]) dfs(to);
92     for(int q : End[pos]) out[q] = Time;
93 }
94
95 int main(){
96     // init();
97     // push_links();
98     // traverse(s);
99     // dfs(Root);
100 }

```

10.2 Dynamic Aho-Corasick

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define pii pair<int,int>
4 #define MAX 300009
5 const int LOG = 19;
6 typedef long long ll;
7
8 struct Aho{
9     //Aho-Corasick
10    //Complexity : |Text| + Sum of all |Pattern| + O(number of Occurrences)
11    //if occurrence positions needed, Worst Case Complexity : (SumLen) Root (SumLen)
12    int Size;

```

```

13 map<char,int>node[MAX];
14 int Root; //AC automaton Root
15 int Nnode; //Total node count
16 int Link[MAX]; //failure links
17 pii Level[MAX]; //Level[i] = Depth of node i
18 int Len[MAX]; //Len[i] = length of i-th query string
19 //vector<int>End[MAX]; //End[i] = indices of strings those end in node i
20 //vector<int>Occ[MAX]; //Occ[i] = occurrences of i-th query string
21 vector < string > vec;
22 int EndCounter[MAX];
23
24 Aho(){init();}
25
26 void init(){
27     Root=0;
28     Nnode=0;
29     node[Root].clear();
30     vec.clear();
31     Size = 0;
32     EndCounter[Root] = 0;
33 }
34
35 void insertword(string p,int ind){
36     vec.push_back(p);
37     Size++;
38     int len=p.size();
39     int now=Root;
40     for(int i=0;i<len;i++){
41         if(!node[now][p[i]]){
42             node[now][p[i]]=++Nnode;
43             Level[Nnode]=make_pair(Level[now].first+1,Nnode);
44             node[Nnode].clear();
45             //End[Nnode].clear();
46             EndCounter[Nnode] = 0;
47         }
48         now=node[now][p[i]];
49     }
50     //End[now].push_back(ind);
51     EndCounter[now]++;
52 }
53
54 void push_links(){
55     queue<int>q;
56     Link[0]=-1;
57     q.push(0);
58     while(!q.empty()){
59         int u=q.front();
60         q.pop();
61
62         for(auto edge : node[u]){
63             char ch=edge.first;
64             int v=edge.second;
65             int j=Link[u];

```

```

66         while(j!=-1 && !node[j][ch]) j=Link[j];
67         if(j!=-1) Link[v]=node[j][ch];
68         else Link[v]=0;
69         q.push(v);
70         //for(int x : End[Link[v]]) End[v].push_back(x);
71         EndCounter[v] += EndCounter[Link[v]];
72     }
73 }
74
75 ll traverse(string s){
76     if(Size == 0) return 0;
77     int len=s.size();
78     int now=Root;
79     ll ans = 0;
80     for(int i=0;i<len;i++){
81         while(now!=-1 && !node[now][s[i]]) now=Link[now];
82         if(now!=-1) now=node[now][s[i]];
83         else now=0;
84         //for(int x=0;x<End[now].size();x++) Occ[End[now][x]].push_back(i);
85         ans += EndCounter[now];
86     }
87     return ans;
88 }
89
90 };
91
92 //Complexity : (Complexity of Aho-Corasick) * LOG
93 struct DynamicAho{
94     Aho aho[LOG];
95     DynamicAho() {init();}
96     void init() {for(ll i = 0; i < LOG; i++) aho[i].init();}
97
98     void Insert(string str){
99         int k = 0;
100         for(k = 0; k < LOG && aho[k].Size > 0; k++);
101         aho[k].insertword(str, 0);
102
103         for(int i = 0; i < k; i++){
104             vector < string > vec = aho[i].vec;
105             aho[i].init();
106             for(auto s : vec) aho[k].insertword(s, 0);
107         }
108         aho[k].push_links();
109     }
110
111     ll getCount(string s){
112         ll ans = 0;
113         for(ll i = 0; i < LOG; i++) ans += aho[i].traverse(s);
114         return ans;
115     }
116 }
117
118 };

```

```

119 DynamicAho dn_aho[2];
120
121 int main(){
122     //init();
123     //Must clear Occ vector before starting every test case
124     //insert identical query strings only once and handle others keeping map
125     //push_links();
126     //traverse(s);
127     //Find the occurrences from Occ array and use them.
128     ios::sync_with_stdio(false);
129     ll n, tp;
130     string str;
131
132     cin >> n;
133     for(ll i = 1; i <= n; i++){
134         cin >> tp >> str;
135         if(tp == 3){
136             ll ans1 = dn_aho[0].getCount(str) ;
137             ll ans2 = dn_aho[1].getCount(str);
138             cout << ans1 - ans2 << '\n';
139             fflush(stdout);
140         }
141         else dn_aho[tp - 1].Insert(str);
142     }
143 }
144

```

10.3 Hashing (1D)

```

1 // Hashing
2 // Hashvalue(l...r) = hsh[l] - hsh[r + 1] * base ^ (r - l + 1);
3 // Must call preprocess
4 #include<bits/stdc++.h>
5 using namespace std;
6 typedef long long ll;
7 const int MAX = 100009;
8 ll mods[2] = {1000000007, 1000000009};
9 //Some back-up primes: 1072857881, 1066517951, 1040160883
10 ll bases[2] = {137, 281};
11 ll pwbase[3][MAX];
12
13 void Preprocess(){
14     pwbase[0][0] = pwbase[1][0] = 1;
15     for(ll i = 0; i < 2; i++){
16         for(ll j = 1; j < MAX; j++){
17             pwbase[i][j] = (pwbase[i][j - 1] * bases[i]) % mods[i];
18         }
19     }
20 }
21
22 struct Hashing{
23     ll hsh[2][MAX];

```

```

24 string str;
25
26 Hashing(){}
27 Hashing(string _str) {str = _str; memset(hsh, 0, sizeof(hsh)); build();}
28
29 void Build(){
30     for(ll i = str.size() - 1; i >= 0; i--){
31         for(int j = 0; j < 2; j++){
32             hsh[j][i] = (hsh[j][i + 1] * bases[j] + str[i]) % mods[j];
33             hsh[j][i] = (hsh[j][i] + mods[j]) % mods[j];
34         }
35     }
36 }
37
38 pair<ll,ll> GetHash(ll i, ll j){
39     assert(i <= j);
40     ll tmp1 = (hsh[0][i] - (hsh[0][j + 1] * pwbase[0][j - i + 1]) % mods[0])
41     % mods[0];
42     ll tmp2 = (hsh[1][i] - (hsh[1][j + 1] * pwbase[1][j - i + 1]) % mods[1])
43     % mods[1];
44     if(tmp1 < 0) tmp1 += mods[0];
45     if(tmp2 < 0) tmp2 += mods[1];
46     return make_pair(tmp1, tmp2);
47 }

```

10.4 Hashing (2D)

```

1 #define MAX 2005
2 #define ll long long int
3
4 int Base[2] = {773,709};
5 int Mod[2] = {281559881,398805713};
6 int InvBase[2][2];
7
8 int Pow[MAX][2][2], Inv[MAX][2][2];
9 int PowCell[MAX][MAX][2], InvCell[MAX][MAX][2];
10
11 int A[MAX][MAX], B[MAX][MAX];
12 int cA[MAX][MAX][2], cB[MAX][MAX][2];
13
14 inline int cMod(int x,int mod){
15     while(x >= mod) x-=mod;
16     while(x < 0) x+=mod;
17     return x;
18 }
19
20 int bigMod(int n,int r,int mod){
21     if(r==0) return 1;
22     ll ret = bigMod(n, r/2, mod);
23     ret = (ret*ret) % mod;
24     if(r&1) ret = (ret*n)% mod;

```

```

25     return ret;
26 }
27 int invMod(int n,int mod) {return bigMod(n,mod-2,mod);}
28
29 void Preprocess(){
30     for(int b=0;b<2;b++) for(int m=0;m<2;m++)
31         Pow[0][b][m] = Inv[0][b][m] = 1;
32     for(int b=0;b<2;b++) for(int m=0;m<2;m++) for(int i=1;i<MAX;i++)
33         Pow[i][b][m] = (Pow[i-1][b][m]*1LL*Base[b]) % Mod[m];
34     for(int m=0;m<2;m++) for(int i=0;i<MAX;i++) for(int j=0;j<MAX;j++)
35         PowCell[i][j][m] = (Pow[i][0][m]*1LL*Pow[j][1][m]) % Mod[m];
36
37     for(int b=0;b<2;b++) for(int m=0;m<2;m++)
38         InvBase[b][m] = invMod(Base[b],Mod[m]);
39     for(int b=0;b<2;b++) for(int m=0;m<2;m++) for(int i=1;i<MAX;i++)
40         Inv[i][b][m] = (Inv[i-1][b][m]*1LL*InvBase[b][m]) % Mod[m];
41     for(int m=0;m<2;m++) for(int i=0;i<MAX;i++) for(int j=0;j<MAX;j++)
42         InvCell[i][j][m] = (Inv[i][0][m]*1LL*Inv[j][1][m]) % Mod[m];
43 }
44
45 void Build(int M[MAX][MAX],int cM[MAX][MAX][2],int r,int c){
46     for(int m=0;m<2;m++){
47         for(int i=1;i<=r;i++){
48             for(int j=1;j<=c;j++){
49                 int Now=(M[i][j]*1LL*PowCell[i][j][m])%Mod[m];
50                 cM[i][j][m]=cM[i-1][j][m]+cM[i][j-1][m]-cM[i-1][j-1][m]+Now;
51                 cM[i][j][m] = cMod(cM[i][j][m], Mod[m]);
52             }
53         }
54     }
55 }
56
57 pair<int,int> Query(int cM[MAX][MAX][2],int x1,int y1,int x2,int y2){
58     ll Sum[2];
59     for(int m=0;m<2;m++){
60         Sum[m] = cB[x2][y2][m]-cB[x2][y1-1][m]-cB[x1-1][y2][m]+cB[x1-1][y1-1][m];
61         Sum[m] = cMod(Sum[m],Mod[m]);
62         Sum[m] = (InvCell[x1-1][y1-1][m]*1LL*Sum[m]) % Mod[m];
63     }
64     return {Sum[0],Sum[1]};
65 }

```

10.5 KMP Fail + Z- Algo

```

1 //0-indexed string s[0...n-1]
2 //fail[i] = maximum x such that s[0...x-1] matches with s[i-x+1...i]
3 int fail[MAX];
4 void build_failure(string s){
5     int n=s.size();
6     for(int i=1;i<n;i++){
7         int j=fail[i-1];
8         while(j>0 && s[i]!=s[j]) j=fail[j-1];

```

```

9         if(s[i]==s[j]) j++;
10        fail[i]=j;
11    }
12 }
13
14 int KMPSearch(string txt, string pat){
15     int Count = 0;
16     int i = 0;
17     int j = 0;
18     while(i < txt.size()){
19         if(txt[i] == pat[j]) i++, j++;
20
21         if(j == pat.size()) Count++, j = fail[j - 1];
22         else if(i < txt.size() && txt[i] != pat[j]){
23             if(j != 0) j = fail[j - 1];
24             else i++;
25         }
26     }
27     return Count;
28 }
29
30 //0-indexed string s[0...n-1]
31 //z[i] = maximum x such that s[0...x-1] matches with s[i..i+x-1]
32 int z[MAX];
33 void zAlgo(string s){
34     int L=0,R=0;
35     int n=s.size();
36     for(int i=1;i<n;i++){
37         if(i>R) {L=R=i; while(R<n && s[R-L]==s[R]) R++; z[i]=R-L; R--;}
38         else{
39             int k=i-L;
40             if(z[k]<R-i+1) z[i]=z[k];
41             else {L=i; while(R<n && s[R-L]==s[R]) R++; z[i]=R-L; R--;}
42         }
43     }
44 }

```

10.6 Minimum Lexicographical Rotation

```

1 int minimumExpression(string s) {
2     s = s + s;
3     int i = 0, j = 1, k = 0, len = s.size();
4     while(i + k < len && j + k < len) {
5         if(s[i + k] == s[j + k]) k++;
6         else if(s[i + k] < s[j + k]) { j = max(j + k + 1, i + 1); k = 0; }
7         else { i = max(i + k + 1, j + 1); k = 0; }
8     }
9     return min(i, j);
10 }

```

10.7 Palindromic Tree

```

1 //Palindromic Tree
2 //Complexity : O(n) memory and O(n) time
3
4 #include<bits/stdc++.h>
5 using namespace std;
6 #define INF 1e9
7 #define MAX 300005
8 #define ff first
9 #define ss second
10 #define pii pair<int,int>
11
12 struct node{
13     int next[26];
14     int minPos;           //Position in string where this node ff occurs
15     int occurrence;       //Total number of occurrence of this node
16     int len;              //Length of the palindrome represented by current node
17     int power;            //Number of palindromic suffixes of the palindrome
18                           //represented by current node
19     int suffLink;         //Links such node w such that the palindrome represented by
20                           //node w is the largest
21                           //palindrome which is a proper suffix of the palindrome
22                           //represented by current node
23
24     //Used for palindrome factorization
25     int diff;
26     pii seriesAns;
27     int serialLink;
28
29     void Init(){
30         memset(next, 0, sizeof(next));
31         len = power = minPos = occurrence = suffLink = 0;
32         diff = serialLink = 0; seriesAns = {INT_MAX, INT_MAX};
33     }
34 };
35
36 struct eerTree{
37     string s;
38     pii DP[MAX];
39     node Tree[MAX];
40     int suff;             // Points to current (initially previous) node in
41                           // palindromic tree
42     int num;              // num = Total number of node in palindromic tree
43                           // num = 1 --> root 1 (len -1), num = 2 --> root 2 (len
44                           // 0)
45
46     void initTree(){
47         num=2; suff=2;
48         Tree[1].Init(); Tree[2].Init();
49         Tree[1].len=-1; Tree[1].suffLink=1;
50         Tree[2].len=0; Tree[2].suffLink=1;
51         DP[0] = {INF, 0};

```

```

47     }
48
49     bool addLetter(int pos){
50         int curr = suff, curLen = 0;
51         int let=s[pos]-'a';
52
53         while(true){
54             curLen=Tree[curr].len;
55             if(pos-1>=curLen && s[pos-1-curLen]==s[pos]) break;
56             curr=Tree[curr].suffLink;
57         }
58
59         if(Tree[curr].next[let]) {
60             suff=Tree[curr].next[let];
61             Tree[suff].occurrence++;
62             return false;
63         }
64
65         suff = ++num;
66         Tree[suff].Init();
67         Tree[suff].minPos = pos;
68         Tree[suff].len = Tree[curr].len+2;
69         Tree[curr].next[let] = suff;
70
71         if(Tree[num].len == 1) Tree[num].suffLink=2;
72         else{
73             while(true){
74                 curr = Tree[curr].suffLink;
75                 curLen = Tree[curr].len;
76                 if(pos-1 >= curLen && s[pos-1-curLen] == s[pos]){
77                     Tree[num].suffLink=Tree[curr].next[let];
78                     break;
79                 }
80             }
81         }
82         Tree[num].power = 1+Tree[Tree[num].suffLink].power;
83         Tree[num].occurrence++;
84
85         //update for palindromic factorization Begin
86         int xx = Tree[suff].suffLink;
87         Tree[suff].diff = Tree[suff].len - Tree[xx].len;
88         if(Tree[suff].diff == Tree[xx].diff){
89             Tree[suff].serialLink = Tree[xx].serialLink;
90         }
91         else Tree[suff].serialLink = suff;
92         //update for palindromic factorization Ended
93
94         return true;
95     }
96
97     void update(int pos){
98         DP[pos]={INF,INF};
99         for(int j=suff; Tree[j].len > 0;){

```

```

100         int sfl = Tree[j].suffLink;
101         int srl = Tree[j].serialLink;
102         int slv = Tree[srl].suffLink;
103         Tree[j].seriesAns = DP[pos - (Tree[slv].len + Tree[j].diff)];
104
105         if(Tree[j].diff == Tree[sfl].diff){
106             Tree[j].seriesAns.ff = min(Tree[j].seriesAns.ff, Tree[sfl].
seriesAns.ff);
107             Tree[j].seriesAns.ss = min(Tree[j].seriesAns.ss, Tree[sfl].
seriesAns.ss);
108         }
109
110         auto val = Tree[j].seriesAns;
111         DP[pos].ff = min(DP[pos].ff, val.ss + 1);
112         DP[pos].ss = min(DP[pos].ss, val.ff + 1);
113         j = slv;
114     }
115 }
116 };
117
118 int main(){
119     // eerTree T; T.initTree();
120     // cin>>s; int n=s.num(); s="$"+s;
121     // for(int i=1;i<=n;i++) addLetter(i, s[i-1]), update(i);
122     // distinct palindrome Count = num - 2
123 }

```

10.8 Suffix Array (O(n logn))

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define MAX 1000005
4
5 int wa[MAX],wb[MAX],wv[MAX],Ws[MAX];
6 int cmp(int *r,int a,int b,int l) {return r[a]==r[b] && r[a+l]==r[b+l];}
7
8 //(1-indexed) sa[i] = starting position (0...n-1) of ith lexicographically
smallest suffix in s
9 //(0-indexed) Rank[i] = lexicographical rank of s[i...n-1] ((i+1)th suffix by
position)
10 //LCP[i] = longest common prefix of sa[i] & sa[i-1]
11 int sa[MAX],Rank[MAX],LCP[MAX];
12
13 //Suffix Array (O(nlogn))
14 //m = maximum possible ASCII value of a string character (alphabet size)
15 //also, m = maximum number of distinct character in string (when compressed)
16 void buildSA(string s,int* sa,int n,int m){
17     int i,j,p,*x=wa,*y=wb,*t;
18     for(i=0; i<m; i++) Ws[i]=0;
19     for(i=0; i<n; i++) Ws[x[i]=s[i]]++;
20     for(i=1; i<m; i++) Ws[i]+=Ws[i-1];
21     for(i=n-1; i>=0; i--) sa[--Ws[x[i]]]=i;

```

```

22     for (j=1,p=1; p<n; j<=1,m=p){
23         for(p=0,i=n-j; i<n; i++) y[p++]=i;
24         for(i=0; i<n; i++) if(sa[i]>=j) y[p++]=sa[i]-j;
25         for(i=0; i<n; i++) wv[i]=x[y[i]];
26         for(i=0; i<m; i++) Ws[i]=0;
27         for(i=0; i<n; i++) Ws[wv[i]]++;
28         for(i=1; i<m; i++) Ws[i]+=Ws[i-1];
29         for(i=n-1; i>=0; i--) sa[--Ws[wv[i]]]=y[i];
30         for(t=x,x=y,y=t,p=1,x[sa[0]]=0,i=1; i<n; i++)
31             x[sa[i]]=cmp(y,sa[i-1],sa[i],j) ? p-1 : p++;
32     }
33 }
34
35 //Kasai's LCP algorithm (O(n))
36 void buildLCP(string s,int *sa,int n){
37     int i,j,k=0;
38     for(i=1; i<=n; i++) Rank[sa[i]]=i;
39     for(i=0; i<n; LCP[Rank[i+1]]=k)
40         for(k?k--:0, j=sa[Rank[i]-1]; s[i+k]==s[j+k]; k++);
41     return;
42 }
43
44 int main(){
45     string s="abababab";
46     int n=s.size();
47     buildSA(s,sa,n+1,130); //Important
48     buildLCP(s,sa,n);
49     for(int i=1;i<=n;i++) cout<<sa[i]<<" "; cout<<endl;
50     for(int i=0;i<n;i++) cout<<Rank[i]<<" "; cout<<endl;
51     for(int i=1;i<=n;i++) cout<<LCP[i]<<" ";
52 }

```

10.9 Suffix Automata

```

1 // Tested : HDU 4270 - Dynamic Lover
2 // Tested : SPOJ SUBLEX, SPOJ LCS, CF 128B
3
4 #include<bits/stdc++.h>
5 using namespace std;
6 #define ll long long int
7 const int ALPHA = 28;
8 const int MAXLEN = 300005;
9 const int LOG = 20;
10 int TotalLen, Size;
11 int Root, Last;
12
13 // Dstnct_substr can be calculated online as St[pos].Dstnct_substr = St[pos].Len
14 // - St[St[pos].link].Len
15 // Occurrence can be calculated online,
16 // First, Every node pos except a clone node, St[pos].Occurrence = 1
17 // then, St[pos].Occurrence = sum of St[i].Occurrence where St[i].Link = pos|
18 // Must make isvliid array 0 for more than one test case

```

```

18 struct Node{
19     int Link, Len;
20     ll Occurrence; // How many times each state (endpos) occurs
21     ll Word; // How many substrings can be reached from this node
22     ll Dstnct_substr; // How many distinct substrings can be reached from this
23     node
24     int FirstPos, version, baseID;
25     int Next[ALPHA];
26     void Clear(){
27         Len = Occurrence = Word = Dstnct_substr = 0;
28         Link = baseID = FirstPos = version = -1;
29         memset(Next, 0, sizeof(Next));
30     }
31 };
32
33 Node St[MAXLEN*2];
34 bool isValid[MAXLEN*2];
35 vector<int>CurrList;
36 vector<int>LastList;
37 vector<int> linkTree[2 * MAXLEN];
38 int Par[2 * MAXLEN], P[2 * MAXLEN][LOG];
39 int perPrefNode[MAXLEN]; ///[i] = node created after inserting ith position in
40 SAM
41
42 inline void CreateNode(int dep){
43     St[Size].Clear();
44     St[Size].Len = dep;
45     St[Size].FirstPos = dep;
46     St[Size].baseID = Size;
47     isValid[Size] = true;
48 }
49
50 inline void init(){
51     Size = 0;
52     Root = Last = TotalLen = 0;
53     St[Root].Clear();
54     CurrList.clear();
55     LastList.clear();
56     for(int i = 0; i < 2 * MAXLEN; i++) linkTree[i].clear();
57 }
58
59 inline bool has(int u, int ch){
60     int x = St[u].Next[ch];
61     return isValid[St[x].baseID];
62 }
63
64 inline void SAM(int pos, int ch){
65     TotalLen++;
66     int Curr = ++Size;
67     CreateNode(St[Last].Len + 1);
68     int p = Last;

```

```

69 while(p != -1 && !has(p, ch)){
70     St[p].Next[ch] = Curr;
71     p = St[p].Link;
72 }
73
74 if(p == -1) {
75     St[Curr].Link = Root;
76 }
77 else{
78     int q = St[p].Next[ch];
79     if(St[q].Len == St[p].Len + 1) St[Curr].Link = q;
80     else{
81         int Clone = ++Size;
82         CreateNode(St[p].Len + 1);
83
84         St[Clone].FirstPos = St[q].FirstPos;
85         memcpy(St[Clone].Next, St[q].Next, sizeof(St[q].Next));
86         St[Clone].Link = St[q].Link;
87         St[Clone].baseID = St[q].baseID;
88         St[q].Link = St[Curr].Link = Clone;
89
90         while(p != -1 && St[p].Next[ch] == q) St[p].Next[ch] = Clone, p = St[
91 p].Link;
92         linkTree[St[Clone].Link].push_back(Clone);
93         Par[Clone] = St[Clone].Link;
94     }
95     perPrefNode[pos] = Curr; //pass pos as parameter
96     linkTree[St[Curr].Link].push_back(Curr);
97     Par[Curr] = St[Curr].Link;
98     CurrList.push_back(Curr);
99     LastList.push_back(Last);
100     Last = Curr;
101 }
102
103 inline void Del(int len){
104     if(!len) return;
105
106     for(int i = 0; i < len; i++){
107         isValid[St[CurrList.back()].baseID] = false;
108         CurrList.pop_back();
109         Last = LastList.back();
110         LastList.pop_back();
111         TotalLen--;
112     }
113 }
114
115 inline void MarkTerminal(int u, int v = 1){
116     while(u != -1) St[u].version = v, u = St[u].Link, St[u].Occurrence = 1;
117 }
118
119 // Returns Smallest Substring with length len
120 // If i + len > s.size(), consider substring(i, s.size())

```

```

121 int LexSmallestSubStr(int len, int idx){
122     MarkTerminal(Last, idx);
123
124     int cur = Root;
125     for(int i = 0; i < len; i++){
126         if(cur > Root && St[cur].version == idx) return TotalLen - i + 1;
127         for(int ch = 0; ch < ALPHA; ch++){
128             if(!has(cur, ch)) continue;
129             cur = St[cur].Next[ch];
130             break;
131         }
132     }
133     return St[cur].FirstPos - len + 1;
134 }
135
136 // Returns longest common substring of 2 strings
137 int LCS(char * s1, char * s2){
138     int len1 = strlen(s1), len2 = strlen(s2);
139     init();
140     for(int i = 0; i < len1; i++) SAM(i, s1[i] - 'a');
141
142     int curNode = 0, curLen = 0, ans = 0;
143     for(int i = 0; i < len2; i++){
144         int ch = s2[i] - 'a';
145         while(curNode > -1 && St[curNode].Next[ch] == 0)
146             curNode = St[curNode].Link, curLen = St[curNode].Len;
147
148         if(curNode == -1) curNode = 0;
149         if(St[curNode].Next[ch]) curNode = St[curNode].Next[ch], curLen++;
150         ans = max(ans, curLen);
151     }
152     return ans;
153 }
154
155 // Must call MarkTerminal Before
156 void dfs_sam(int pos){
157     if(St[pos].Dstnct_substr) return;
158     int resDstnct = 1;
159     for(int i = 0; i < ALPHA; i++) if(St[pos].Next[i]){
160         int to = St[pos].Next[i];
161         dfs_sam(to);
162         resDstnct += St[to].Dstnct_substr;
163         St[pos].Occurrence += St[to].Occurrence;
164         St[pos].Word += St[to].Word;
165     }
166     St[pos].Dstnct_substr = resDstnct;
167     St[pos].Word += St[pos].Occurrence;
168 }
169
170 void PrintKthLexSubstr(int k){
171     // Must call dfs_sam before
172     int cur = Root;
173     while(k > 0){

```

```

174     int tmp = 0;
175     for(int i = 0; i < 26; i++){
176         if(St[cur].Next[i] == 0) continue;
177         int to = St[cur].Next[i];
178         if(tmp + St[to].Word >= k){
179             k -= (tmp + St[to].Occurrence), cur = to;
180             printf("%c", 'a' + i);
181             break;
182         }
183         tmp += St[to].Word;
184     }
185 }
186 }
187
188 void PrintKthLexDstSubstr(int k){
189     // Must call dfs_sam before
190     int cur = Root;
191     while(k > 0){
192         int tmp = 0;
193         for(int i = 0; i < 26; i++){
194             if(St[cur].Next[i] == 0) continue;
195             int to = St[cur].Next[i];
196             if(tmp + St[to].Dstnct_substr >= k){
197                 k -= (tmp + 1), cur = to;
198                 printf("%c", 'a' + i);
199                 break;
200             }
201             tmp += St[to].Dstnct_substr;
202         }
203     }
204 }
205
206 void BuildSparse(int n){
207     // Using 0 for uninitialized parent (Remember and use carefully)
208     for(int i=1; i<=n; i++) for(int j=0; j<LOG; j++) P[i][j] = 0;
209
210     for(int i=1; i<=n; i++) P[i][0] = Par[i];
211     for(int j=1; j<LOG; j++){
212         for(int i=1; i<=n; i++){
213             if(P[i][j-1] != 0){
214                 int x = P[i][j-1];
215                 P[i][j] = P[x][j-1];
216             }
217         }
218     }
219 }
220
221 int getNodeSubstring(int l, int r){
222     int cur = perPrefNode[r], curLen = r - l + 1;
223     if(St[St[cur].Link].Len + 1 <= curLen) return cur;
224
225     for(int i = LOG - 1; i >= 0; i--){
226         if(P[cur][i] == 0) continue;

```

```

227         int ncur = P[cur][i];
228         if(St[ncur].Len >= curLen) cur = ncur;
229     }
230 }
231 // must call init() at the start

```